

Phương pháp chia đôi

1 Cơ sở lý thuyết: Phương pháp chia đôi (Bisection Method)

1.1 1. Ý tưởng và Định lý cơ sở

Phương pháp chia đôi dựa trên **Định lý Bolzano-Cauchy** (Hệ quả của định lý giá trị trung gian đối với hàm liên tục):

Nếu hàm số $f(x)$ liên tục trên đoạn kín $[a, b]$ và thỏa mãn điều kiện $f(a) \cdot f(b) < 0$ (nghĩa là $f(x)$ đổi dấu trên khoảng này), thì tồn tại ít nhất một nghiệm $x^* \in (a, b)$ sao cho $f(x^*) = 0$.

Giả sử phương trình $f(x) = 0$ có nghiệm duy nhất trên $[a, b]$. Phương pháp sẽ liên tiếp chia đôi khoảng chứa nghiệm để thu hẹp dần phạm vi tìm kiếm.

1.2 2. Thuật toán chi tiết

Cho khoảng phân ly nghiệm $[a, b]$ và sai số cho phép ε .

- Bước 1:** Tính trung điểm $c = \frac{a+b}{2}$.
- Bước 2:** Tính giá trị hàm tại trung điểm $f(c)$.
- Bước 3:** Đánh giá để thu hẹp khoảng:
 - Nếu $f(c) = 0$ (hoặc $|f(c)|$ đủ nhỏ): c chính là nghiệm đúng x^* , thuật toán dừng.
 - Nếu $f(a) \cdot f(c) < 0$: Nghiệm x^* nằm trong nửa khoảng bên trái $[a, c]$. Gán $b \leftarrow c$.
 - Nếu $f(c) \cdot f(b) < 0$: Nghiệm x^* nằm trong nửa khoảng bên phải $[c, b]$. Gán $a \leftarrow c$.
- Bước 4:** Lặp lại từ Bước 1 cho đến khi điều kiện dừng được thỏa mãn.

1.3 3. Đánh giá sai số và Điều kiện dừng

Gọi $[a_n, b_n]$ là khoảng chứa nghiệm ở bước lặp thứ n , với $a_0 = a$ và $b_0 = b$. Nghiệm xấp xỉ ở bước thứ n là $x_n = \frac{a_n+b_n}{2}$.

- Chiều dài khoảng:** Ở mỗi bước, độ dài khoảng giảm đi một nửa:

$$b_n - a_n = \frac{b - a}{2^n} \quad (1)$$

- Sai số tuyệt đối (Đánh giá tiên nghiệm):** Vì nghiệm đúng x^* và nghiệm gần đúng x_n đều nằm trong đoạn $[a_n, b_n]$, khoảng cách giữa chúng không vượt quá nửa độ dài đoạn này:

$$|x_n - x^*| \leq \frac{b_n - a_n}{2} = \frac{b - a}{2^{n+1}} \quad (2)$$

- **Số bước lặp tối thiểu:** Để đạt được sai số ε , ta cần $\frac{b-a}{2^{n+1}} \leq \varepsilon \implies 2^{n+1} \geq \frac{b-a}{\varepsilon}$. Từ đó:

$$n \geq \log_2 \left(\frac{b-a}{\varepsilon} \right) - 1 \quad (3)$$

- **Điều kiện dừng thực hành:** Có thể dùng $|b_n - a_n| \leq \varepsilon$ hoặc $|f(x_n)| \leq \varepsilon_f$.

1.4 4. Tính hội tụ

- **Sự hội tụ:** Thuật toán *chắc chắn hội tụ toàn cục* đối với mọi hàm liên tục đổi dấu trên $[a, b]$.
- **Tốc độ hội tụ:** Tuyến tính (Linear convergence) với hằng số tiệm cận $C = \frac{1}{2}$, nên tốc độ khá chậm so với Newton-Raphson hay dãy cung.

2 Ví dụ minh họa chi tiết

2.1 Ví dụ 1: Áp dụng phương pháp chia đôi cơ bản

Đề bài: Tìm nghiệm xấp xỉ của phương trình $f(x) = x^3 - x - 1 = 0$ trên khoảng phân ly nghiệm $[1, 2]$ với sai số cho phép $\varepsilon = 0.2$.

Lời giải:

1. Kiểm tra điều kiện tồn tại nghiệm:

Hàm số $f(x)$ đa thức nên liên tục trên $[1, 2]$.

Tính giá trị tại hai đầu mút:

- $f(1) = 1^3 - 1 - 1 = -1 < 0$
- $f(2) = 2^3 - 2 - 1 = 5 > 0$

Vì $f(1) \cdot f(2) = -5 < 0$, theo định lý Bolzano-Cauchy, phương trình có ít nhất một nghiệm $x^* \in (1, 2)$.

2. Đánh giá số bước lặp tối thiểu (Tiên nghiệm):

Áp dụng công thức tính số bước lặp:

$$n \geq \log_2 \left(\frac{b-a}{\varepsilon} \right) - 1 = \log_2 \left(\frac{2-1}{0.2} \right) - 1 = \log_2(5) - 1 \approx 2.32 - 1 = 1.32$$

Do n nguyên, cần thực hiện tối thiểu $n = 2$ bước lặp (đạt tới x_2).

3. Quá trình lặp:

• Bước 0 ($n = 0$):

Khoảng ban đầu: $[a_0, b_0] = [1, 2]$.

Nghiệm xấp xỉ: $x_0 = \frac{1+2}{2} = 1.5$.

Giá trị hàm: $f(x_0) = f(1.5) = 1.5^3 - 1.5 - 1 = 0.875 > 0$.

Xét dấu: $f(a_0) \cdot f(x_0) = f(1) \cdot f(1.5) = (-1) \cdot 0.875 < 0$.

Kết luận: Nghiệm nằm trong nửa khoảng trái. Thu hẹp khoảng thành $[a_1, b_1] = [1, 1.5]$.

• **Bước 1** ($n = 1$):

Khoảng hiện tại: $[a_1, b_1] = [1, 1.5]$.

Nghiệm xấp xỉ: $x_1 = \frac{1+1.5}{2} = 1.25$.

Giá trị hàm: $f(x_1) = f(1.25) = 1.25^3 - 1.25 - 1 = -0.296875 < 0$.

Xét dấu: $f(x_1) \cdot f(b_1) = f(1.25) \cdot f(1.5) = (-0.296875) \cdot 0.875 < 0$.

Kết luận: Nghiệm nằm trong nửa khoảng phải. Thu hẹp khoảng thành $[a_2, b_2] = [1.25, 1.5]$.

• **Bước 2** ($n = 2$):

Khoảng hiện tại: $[a_2, b_2] = [1.25, 1.5]$.

Nghiệm xấp xỉ: $x_2 = \frac{1.25+1.5}{2} = 1.375$.

Đánh giá sai số thực tế:

$$|x_2 - x^*| \leq \frac{b_2 - a_2}{2} = \frac{1.5 - 1.25}{2} = 0.125$$

Vì $0.125 \leq 0.2$ (thỏa mãn điều kiện ε), thuật toán dừng tại đây.

Kết luận: Nghiệm xấp xỉ của phương trình là $x \approx 1.375$, với sai số tuyệt đối mắc phải không vượt quá 0.125.

3 Các dạng toán thường gặp

3.1 Dạng 1: Tìm khoảng phân ly nghiệm

Đề bài: Tìm các khoảng phân ly nghiệm của phương trình $f(x) = x^3 - x - 1 = 0$.

Phương pháp giải: Lập bảng biến thiên (sử dụng đạo hàm $f'(x)$) hoặc thử các giá trị nguyên liên tiếp để tìm những khoảng (a, b) sao cho $f(a)f(b) < 0$. $f(1) = -1, f(2) = 5 \implies$ Có nghiệm thuộc $(1, 2)$.

3.2 Dạng 2: Trình bày thuật toán và tính toán bằng tay (N bước)

Đề bài: Cho hàm $f(x) = \cos(x) - x$. Áp dụng phương pháp chia đôi tìm nghiệm trong đoạn $[0, 1]$, thực hiện $n = 3$ bước lặp và đánh giá sai số.

Phương pháp giải: Lập bảng các cột: $n, a_n, b_n, x_n, f(a_n), f(x_n), f(a_n)f(x_n)$ và tính toán cẩn thận, sử dụng công thức sai số $\Delta = \frac{b-a}{2^{n+1}}$.

3.3 Dạng 3: Bài toán dự báo số bước lặp (Đánh giá tiên nghiệm)

Đề bài: Cần chia đôi ít nhất bao nhiêu lần để tìm nghiệm của phương trình $e^x - 2x - 2 = 0$ trên đoạn $[1, 2]$ với sai số $\varepsilon = 10^{-4}$?

Phương pháp giải: Sử dụng công thức $n \geq \log_2 \left(\frac{b-a}{\varepsilon} \right) - 1$. Thay số: $n \geq \log_2 \left(\frac{2-1}{10^{-4}} \right) - 1 \approx 12.28 \implies n = 13$ bước lặp.

3.4 Dạng 4: Bài toán ứng dụng (Mô hình hóa phương trình)

Đề bài: Một quả bóng rơi tự do có phương trình vận tốc $v(t) = \frac{gm}{c}(1 - e^{-ct/m})$. Biết $g = 9.8$, $m = 68.1$, $v = 40$ tại $t = 10$. Hãy dùng phương pháp chia đôi tìm hệ số cản c trong khoảng $[10, 20]$ sao cho sai số nhỏ hơn 0.1.

Phương pháp giải: Xây dựng hàm mục tiêu $f(c) = \frac{9.8 \times 68.1}{c}(1 - e^{-10c/68.1}) - 40$. Kiểm tra $f(10)f(20) < 0$ và tiến hành lập trình/tính tay thuật toán chia đôi.

4 Mã giả (Pseudocode) và Thuật toán

Quy trình lập của phương pháp chia đôi được trình bày một cách hệ thống qua đoạn mã giả dưới đây. Thuật toán đòi hỏi đầu vào bao gồm hàm mục tiêu $f(x)$ liên tục, khoảng phân ly nghiệm $[a, b]$ thỏa mãn điều kiện $f(a) \cdot f(b) < 0$, dung sai sai số ε và số vòng lặp tối đa N nhằm đảm bảo chương trình không rơi vào vòng lặp vô hạn.

Algorithm 1 Phương pháp Chia đôi (Bisection Method)

Require: Hàm số $f(x)$ liên tục, các điểm mút a và b , dung sai ε , số vòng lặp tối đa N

Ensure: Nghiệm xấp xỉ c hoặc thông báo lỗi

```
1: if  $f(a) \cdot f(b) \geq 0$  then                                ▷ Kiểm tra tính hợp lệ của khoảng phân ly
2:   return "Lỗi: Hàm số không đổi dấu trên đoạn  $[a, b]$ "
3: end if
4:  $n \leftarrow 1$ 
5: while  $n \leq N$  do
6:    $c \leftarrow \frac{a+b}{2}$                                        ▷ Tính tọa độ trung điểm
7:   if  $f(c) = 0$  hoặc  $\frac{b-a}{2} < \varepsilon$  then                     ▷ Đánh giá tiêu chí hội tụ
8:     return  $c$ 
9:   end if
10:  if  $f(a) \cdot f(c) < 0$  then                                ▷ Định vị khoảng chứa nghiệm
11:     $b \leftarrow c$ 
12:  else
13:     $a \leftarrow c$ 
14:  end if
15:   $n \leftarrow n + 1$ 
16: end while
17: return "Thuật toán không hội tụ sau  $N$  vòng lặp"
```

Giải Phương Trình $f(x) = 0$ Bằng Phương Pháp Dây Cung (Secant Method)

Tài liệu học tập

Ngày 18 tháng 6 năm 2026

1 Lý thuyết và Khái niệm

Phương pháp dây cung (Secant Method) là một thuật toán lặp tìm nghiệm gần đúng của phương trình phi tuyến $f(x) = 0$. Thuật toán này có thể xem là một biến thể của phương pháp Newton-Raphson. Trong phương pháp Newton, ta cần tính đạo hàm $f'(x)$ tại mỗi bước lặp. Tuy nhiên, trong thực tế, nhiều hàm số rất khó tính đạo hàm giải tích hoặc tốn kém tài nguyên tính toán.

Phương pháp dây cung khắc phục nhược điểm này bằng cách xấp xỉ đạo hàm $f'(x_n)$ thông qua độ dốc của cát tuyến (dây cung) đi qua hai điểm gần nhất trên đồ thị là $(x_{n-1}, f(x_{n-1}))$ và $(x_n, f(x_n))$:

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}} \quad (1)$$

Thay biểu thức xấp xỉ này vào công thức lặp Newton-Raphson $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$, ta thu được công thức của phương pháp dây cung.

Công thức lặp

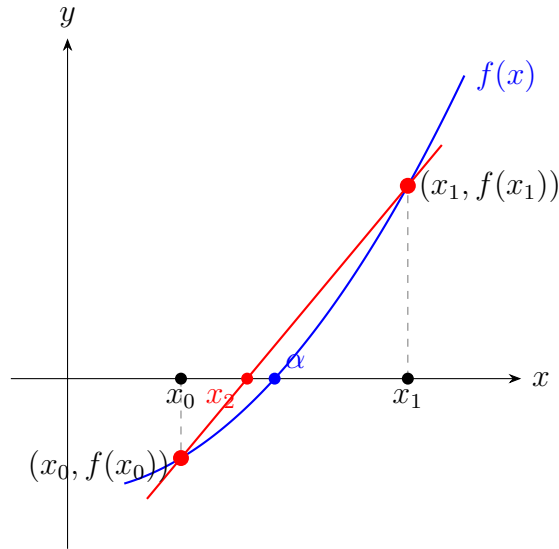
$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} \quad (2)$$

Đặc điểm:

- Thuật toán cần hai giá trị khởi tạo ban đầu là x_0 và x_1 . Hai giá trị này không nhất thiết phải bao quanh nghiệm (khác với phương pháp chia đôi).
- Tốc độ hội tụ của phương pháp dây cung là *siêu tuyến tính* (superlinear), với bậc hội tụ $\alpha \approx 1.618$ (tỷ lệ vàng). Tốc độ này nhanh hơn phương pháp chia đôi nhưng chậm hơn phương pháp Newton (bậc 2).

1.1 Ý nghĩa hình học

Về mặt hình học, giao điểm của đường thẳng nối hai điểm $(x_{n-1}, f(x_{n-1}))$ và $(x_n, f(x_n))$ với trục hoành chính là giá trị xấp xỉ tiếp theo x_{n+1} . Quá trình này được thực hiện lặp đi lặp lại để các điểm tiến dần đến nghiệm thực của phương trình.



Hình 1: Minh họa ý nghĩa hình học của Phương pháp dây cung.

2 Mã giả (Pseudocode) và Thuật toán

Quy trình thực hiện thuật toán dây cung được trình bày rõ ràng qua đoạn mã giả dưới đây. Phương pháp này yêu cầu cung cấp hàm mục tiêu f , hai giá trị dự đoán ban đầu x_0, x_1 , dung sai ϵ và số vòng lặp tối đa N để tránh lặp vô hạn nếu thuật toán phân kỳ.

Thuật toán 1 Phương pháp Dây cung (Secant Method)

Đầu vào: Hàm số $f(x)$, các điểm khởi tạo x_0, x_1 , dung sai ϵ , số vòng lặp tối đa N

Đầu ra: Nghiệm gần đúng x hoặc thông báo lỗi

- 1: $i \leftarrow 1$
 - 2: $q_0 \leftarrow f(x_0)$
 - 3: $q_1 \leftarrow f(x_1)$
 - 4: **Trong khi** $i \leq N$ **thực hiện**
 - 5: **Nếu** $|q_1 - q_0| < 10^{-14}$ **thì** ▷ Tránh lỗi chia cho 0
 - 6: **Trả về** "Lỗi: Mẫu số quá nhỏ"
 - 7: **Kết thúc Nếu**
 - 8: $x_2 \leftarrow x_1 - q_1 \frac{x_1 - x_0}{q_1 - q_0}$ ▷ Tính điểm tiếp theo
 - 9: **Nếu** $|x_2 - x_1| < \epsilon$ **hoặc** $|f(x_2)| < \epsilon$ **thì**
 - 10: **Trả về** x_2 ▷ Thỏa mãn điều kiện hội tụ
 - 11: **Kết thúc Nếu**
 - 12: $i \leftarrow i + 1$
 - 13: $x_0 \leftarrow x_1$ ▷ Cập nhật lại giá trị cho vòng lặp sau
 - 14: $q_0 \leftarrow q_1$
 - 15: $x_1 \leftarrow x_2$
 - 16: $q_1 \leftarrow f(x_2)$
 - 17: **Kết thúc Trong khi**
 - 18: **Trả về** "Thuật toán không hội tụ sau N vòng lặp"
-

3 Ví dụ minh họa

Bài toán: Áp dụng phương pháp dây cung để tìm một nghiệm của phương trình sau với sai số $\epsilon = 0.01$:

$$f(x) = x^3 - x - 1 = 0$$

Chọn hai điểm dự đoán ban đầu là $x_0 = 1$ và $x_1 = 2$.

Giải:

Ta có $f(x) = x^3 - x - 1$. Tính các giá trị hàm tại hai điểm đầu tiên:

$$f(x_0) = f(1) = 1^3 - 1 - 1 = -1$$

$$f(x_1) = f(2) = 2^3 - 2 - 1 = 5$$

Vòng lặp 1 ($n = 1$):

$$\begin{aligned}x_2 &= x_1 - f(x_1) \frac{x_1 - x_0}{f(x_1) - f(x_0)} \\&= 2 - 5 \frac{2 - 1}{5 - (-1)} = 2 - \frac{5}{6} \approx 1.16667\end{aligned}$$

$$f(x_2) = (1.16667)^3 - 1.16667 - 1 \approx -0.57870$$

Vòng lặp 2 ($n = 2$):

$$\begin{aligned}x_3 &= x_2 - f(x_2) \frac{x_2 - x_1}{f(x_2) - f(x_1)} \\&= 1.16667 - (-0.57870) \frac{1.16667 - 2}{-0.57870 - 5} \\&\approx 1.16667 + 0.57870 \frac{-0.83333}{-5.57870} \approx 1.25311\end{aligned}$$

$$f(x_3) = (1.25311)^3 - 1.25311 - 1 \approx -0.28536$$

Vòng lặp 3 ($n = 3$):

$$\begin{aligned}x_4 &= x_3 - f(x_3) \frac{x_3 - x_2}{f(x_3) - f(x_2)} \\&= 1.25311 - (-0.28536) \frac{1.25311 - 1.16667}{-0.28536 - (-0.57870)} \\&\approx 1.33721\end{aligned}$$

(Kiểm tra: $|x_4 - x_3| = 0.0841 > 0.01$, tiếp tục lặp)

Tiếp tục quá trình này, thuật toán sẽ hội tụ rất nhanh về nghiệm thực của phương trình là $x \approx 1.32472$ sau vài vòng lặp tiếp theo.

Phương pháp tiếp tuyến (Newton-Raphson)

June 19, 2026

1 Phương pháp tiếp tuyến (Newton-Raphson)

1.1 Cơ sở lý thuyết

Phương pháp tiếp tuyến (hay phương pháp Newton-Raphson) là một thuật toán tìm nghiệm xấp xỉ của phương trình phi tuyến $f(x) = 0$. Thuật toán sử dụng khai triển Taylor bậc nhất để tuyến tính hóa hàm số tại lân cận của một điểm xấp xỉ, từ đó thiết lập một chuỗi hội tụ tiệm cận đến nghiệm thực của hệ thống.

Về mặt hình học, tại mỗi bước lặp k , phương pháp thiết lập phương trình tiếp tuyến của đồ thị hàm số $y = f(x)$ tại điểm có hoành độ x_k . Giao điểm của tiếp tuyến này với trục hoành ($y = 0$) sẽ được xác định là giá trị xấp xỉ tiếp theo x_{k+1} .

Phương trình tiếp tuyến của $f(x)$ tại x_k được biểu diễn giải tích như sau:

$$y - f(x_k) = f'(x_k)(x - x_k) \quad (1)$$

Thiết lập $y = 0$ để tìm giao điểm với trục hoành, ta thu được công thức truy hồi Newton-Raphson:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \quad (k = 0, 1, 2, \dots) \quad (2)$$

Định lý 1 (Điều kiện hội tụ Fourier). *Giả sử hàm số $f(x)$ liên tục và có đạo hàm đến cấp 2 liên tục trên khoảng phân ly nghiệm $[a, b]$, đồng thời thỏa mãn các điều kiện:*

- Đạo hàm bậc nhất $f'(x)$ và đạo hàm bậc hai $f''(x)$ không đổi dấu trên toàn bộ khoảng $[a, b]$.*
- Điểm khởi tạo $x_0 \in [a, b]$ được chọn sao cho điều kiện Fourier được thỏa mãn:*

$$f(x_0) \cdot f''(x_0) > 0 \quad (3)$$

Khi đó, dãy xấp xỉ $\{x_k\}$ được định nghĩa bởi công thức truy hồi sẽ hội tụ đơn điệu về nghiệm duy nhất x^ của phương trình $f(x) = 0$ trên $[a, b]$.*

1.1.1 Thuật toán định lượng

Hệ thống tiến hành xác định nghiệm xấp xỉ thông qua các bước logic sau:

- Xác định khoảng phân ly nghiệm $[a, b]$ chứa nghiệm duy nhất x^* .
- Lựa chọn điểm khởi tạo $x_0 \in [a, b]$ thỏa mãn $f(x_0)f''(x_0) > 0$.
- Tính toán xấp xỉ tại chu kỳ $k + 1$ theo công thức: $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$.
- Kiểm tra tiêu chuẩn hội tụ. Thuật toán dừng lại khi thỏa mãn điều kiện sai số: $|x_{k+1} - x_k| \leq \varepsilon$ hoặc $|f(x_{k+1})| \leq \varepsilon$, trong đó ε là dung sai cho trước.

1.1.2 Mã giả (Pseudocode)

Algorithm 1 Thuật toán Newton-Raphson

```
1: Input: Hàm số  $f(x)$ , đạo hàm  $f'(x)$ , điểm khởi tạo  $x_0$ , sai số cho phép  $\varepsilon$ , số vòng lặp tối đa  $N_{max}$ .
2: Output: Nghiệm xấp xỉ  $x^*$  hoặc thông báo không hội tụ.
3:  $k \leftarrow 0$ 
4: while  $k < N_{max}$  do
5:   if  $f'(x_k) == 0$  then
6:     Return "Lỗi: Đạo hàm triệt tiêu, thuật toán gián đoạn."
7:   end if
8:    $x_{k+1} \leftarrow x_k - \frac{f(x_k)}{f'(x_k)}$ 
9:   if  $|x_{k+1} - x_k| < \varepsilon$  or  $|f(x_{k+1})| < \varepsilon$  then
10:    Return  $x_{k+1}$ 
11:  end if
12:   $x_k \leftarrow x_{k+1}$ 
13:   $k \leftarrow k + 1$ 
14: end while
15: Return "Không hội tụ sau  $N_{max}$  bước lặp."
```

1.1.3 Ví dụ minh họa

Bài toán: Xác định nghiệm xấp xỉ của phương trình $f(x) = x^3 - x - 1 = 0$ trên khoảng $[1, 2]$ với sai số $\varepsilon = 10^{-4}$.

Lời giải: Tiến hành khảo sát các đạo hàm của hàm số:

$$\begin{aligned}f(x) &= x^3 - x - 1 \\f'(x) &= 3x^2 - 1 \\f''(x) &= 6x\end{aligned}$$

Trên khoảng $[1, 2]$, ta có:

- $f(1) = -1 < 0$ và $f(2) = 5 > 0 \implies f(1)f(2) < 0$ (tồn tại nghiệm trong khoảng).
- $f'(x) = 3x^2 - 1 > 0, \forall x \in [1, 2]$ (hàm số đồng biến, nghiệm duy nhất).
- $f''(x) = 6x > 0, \forall x \in [1, 2]$ (hàm số lồi).

Xác định điểm khởi tạo x_0 : Lựa chọn $x_0 = 2$ do $f(2) \cdot f''(2) = 5 \cdot 12 = 60 > 0$. Áp dụng công thức truy hồi:

$$x_{k+1} = x_k - \frac{x_k^3 - x_k - 1}{3x_k^2 - 1} = \frac{2x_k^3 + 1}{3x_k^2 - 1}$$

Tiến hành các bước lặp tính toán:

- **Bước $k = 0$:** $x_0 = 2$
- **Bước $k = 1$:** $x_1 = 2 - \frac{f(2)}{f'(2)} = 2 - \frac{5}{11} \approx 1.54545$
- **Bước $k = 2$:** $x_2 = 1.54545 - \frac{f(1.54545)}{f'(1.54545)} \approx 1.35961$
- **Bước $k = 3$:** $x_3 = 1.35961 - \frac{f(1.35961)}{f'(1.35961)} \approx 1.32580$

- **Bước $k = 4$:** $x_4 = 1.32580 - \frac{f(1.32580)}{f'(1.32580)} \approx 1.32472$

- **Bước $k = 5$:** $x_5 = 1.32472 - \frac{f(1.32472)}{f'(1.32472)} \approx 1.32472$

Đánh giá sai số: $|x_5 - x_4| \approx 0 < 10^{-4}$. Hệ thống đạt trạng thái hội tụ. Nghiệm xấp xỉ của phương trình là: $x^* \approx 1.32472$.

Phương pháp lặp đơn (Fixed-Point Iteration)

June 19, 2026

1 Phương pháp lặp đơn (Fixed-Point Iteration)

1.1 Cơ sở lý thuyết

Phương pháp lặp đơn là một kỹ thuật số trị để giải phương trình phi tuyến $f(x) = 0$ thông qua việc biến đổi đại số phương trình gốc về dạng tương đương:

$$x = g(x) \quad (1)$$

Nghiệm x^* của phương trình $f(x) = 0$ chính là điểm bất động (fixed point) của hàm số $g(x)$, tức là $x^* = g(x^*)$.

Bằng cách thiết lập một giá trị khởi tạo x_0 , phương pháp xây dựng một dãy lặp $\{x_k\}$ dựa trên hệ thức truy hồi:

$$x_{k+1} = g(x_k) \quad (k = 0, 1, 2, \dots) \quad (2)$$

Định lý 1 (Điều kiện hội tụ của phương pháp lặp đơn). *Giả sử hàm số $g(x)$ liên tục trên đoạn $[a, b]$ và thỏa mãn hai điều kiện sau:*

1. *Điều kiện nội tại: Định lý giá trị trung gian đảm bảo $g(x) \in [a, b], \forall x \in [a, b]$.*
2. *Điều kiện co (Contraction mapping): Tồn tại một hằng số $q \in (0, 1)$ sao cho đạo hàm của hàm lặp thỏa mãn:*

$$|g'(x)| \leq q < 1, \quad \forall x \in (a, b) \quad (3)$$

Khi đó:

- Phương trình $x = g(x)$ có một nghiệm duy nhất $x^* \in [a, b]$.
- Dãy lặp $x_{k+1} = g(x_k)$ hội tụ về nghiệm x^* với mọi giá trị khởi tạo $x_0 \in [a, b]$.
- Đánh giá sai số hậu nghiệm tại chu kỳ k được xác định bởi:

$$|x_k - x^*| \leq \frac{q}{1 - q} |x_k - x_{k-1}| \quad (4)$$

1.1.1 Thuật toán định lượng

Quy trình xác định nghiệm xấp xỉ bằng phương pháp lặp đơn tuân thủ các bước giải tích sau:

1. Phân ly nghiệm: Khảo sát và cô lập nghiệm thực của $f(x) = 0$ trong khoảng $[a, b]$.
2. Xây dựng hàm lặp: Thiết lập $x = g(x)$ sao cho $|g'(x)| \leq q < 1$ trên toàn miền $[a, b]$. Giá trị q càng nhỏ, hệ thống hội tụ càng nhanh.

3. Lựa chọn điểm khởi tạo $x_0 \in [a, b]$.
4. Tính toán dãy xấp xỉ: $x_{k+1} = g(x_k)$.
5. Kiểm tra tiêu chuẩn hội tụ: Thuật toán dừng lại khi sai số đạt ngưỡng dung sai ε định trước:

$$|x_k - x_{k-1}| \leq \frac{1-q}{q} \varepsilon \quad (5)$$

(Hoặc sử dụng tiêu chuẩn đơn giản hóa $|x_k - x_{k-1}| \leq \varepsilon$ trong các trường hợp thực hành hệ số co q xấp xỉ 0.5).

1.2 Mã giả (Pseudocode)

Algorithm 1 Thuật toán lặp đơn

```

1: Input: Hàm lặp  $g(x)$ , điểm khởi tạo  $x_0$ , sai số cho phép  $\varepsilon$ , hệ số co  $q$ , số vòng lặp tối đa  $N_{max}$ .
2: Output: Nghiệm xấp xỉ  $x^*$  hoặc thông báo không hội tụ.
3:  $k \leftarrow 0$ 
4:  $tol \leftarrow \varepsilon \cdot \frac{1-q}{q}$  ▷ Ngưỡng hội tụ lý thuyết
5: while  $k < N_{max}$  do
6:    $x_{k+1} \leftarrow g(x_k)$ 
7:   if  $|x_{k+1} - x_k| \leq tol$  then
8:     Return  $x_{k+1}$ 
9:   end if
10:   $x_k \leftarrow x_{k+1}$ 
11:   $k \leftarrow k + 1$ 
12: end while
13: Return "Trạng thái phân kỳ hoặc không hội tụ sau  $N_{max}$  bước lặp."

```

1.2.1 Ví dụ minh họa

Bài toán: Xác định nghiệm xấp xỉ của phương trình $f(x) = x^3 - x - 1 = 0$ trên khoảng $[1, 2]$ với sai số $\varepsilon = 10^{-4}$.

Lời giải: Tiến hành thiết lập phương trình tương đương. Rút x theo cấu trúc có đạo hàm giới nội:

$$x^3 = x + 1 \implies x = \sqrt[3]{x+1}$$

Xác định hàm lặp $g(x) = (x+1)^{\frac{1}{3}}$.

Kiểm tra điều kiện hội tụ trên khoảng $[1, 2]$:

- Tính đạo hàm: $g'(x) = \frac{1}{3}(x+1)^{-\frac{2}{3}} = \frac{1}{3\sqrt[3]{(x+1)^2}}$.
- Trên đoạn $[1, 2]$, hàm $g'(x)$ là hàm nghịch biến và nhận giá trị dương. Do đó:

$$\max_{x \in [1, 2]} |g'(x)| = g'(1) = \frac{1}{3\sqrt[3]{4}} \approx 0.210$$

- Hệ số co $q = 0.210 < 1$. Điều kiện hội tụ được thỏa mãn tuyệt đối.

Xác định ngưỡng hội tụ hiệu chỉnh cho thuật toán: $tol = \varepsilon \frac{1-q}{q} = 10^{-4} \times \frac{1-0.210}{0.210} \approx 3.76 \times 10^{-4}$.

Lựa chọn điểm khởi tạo $x_0 = 1.5$. Áp dụng công thức lặp $x_{k+1} = \sqrt[3]{x_k + 1}$:

- **Bước $k = 0$:** $x_0 = 1.5$
- **Bước $k = 1$:** $x_1 = \sqrt[3]{1.5 + 1} \approx 1.35721$
- **Bước $k = 2$:** $x_2 = \sqrt[3]{1.35721 + 1} \approx 1.33086$
- **Bước $k = 3$:** $x_3 = \sqrt[3]{1.33086 + 1} \approx 1.32588$
- **Bước $k = 4$:** $x_4 = \sqrt[3]{1.32588 + 1} \approx 1.32494$
- **Bước $k = 5$:** $x_5 = \sqrt[3]{1.32494 + 1} \approx 1.32476$

Kiểm tra sai số tại bước $k = 5$:

$$|x_5 - x_4| = |1.32476 - 1.32494| = 0.00018 \leq 3.76 \times 10^{-4} \text{ (Thỏa mãn tiêu chuẩn)}$$

Hệ thống đạt trạng thái hội tụ. Nghiệm xấp xỉ của phương trình là $x^* \approx 1.32476$ (đối chiếu với nghiệm từ phương pháp tiếp tuyến 1.32472, kết quả hoàn toàn nhất quán trong giới hạn dung sai).

Giải tích số: Phương pháp Gauss và Gauss-Jordan giải hệ $Ax = B$

1 Điều kiện áp dụng

Hệ phương trình tuyến tính $Ax = B$ chỉ có nghiệm duy nhất khi và chỉ khi ma trận hệ số A là ma trận vuông cấp n (n phương trình, n ẩn) và không suy biến ($\det(A) \neq 0$). Khi thuật toán vận hành, nếu gặp toàn bộ một cột bằng 0 từ vị trí đường chéo chính trở xuống, hệ suy biến và thuật toán dừng.

2 Phương pháp khử Gauss (Gaussian Elimination)

2.1 Thuật toán chi tiết

Mục tiêu: Đưa ma trận mở rộng $\bar{A} = [A|B]$ về dạng ma trận tam giác trên, sau đó giải ngược từ dưới lên.

1. **Quá trình thuận (Khử tiến):** Lặp k từ 1 đến $n - 1$:

- *Chọn chốt (Pivoting):* Tìm phần tử có trị tuyệt đối lớn nhất trong cột k (từ dòng k đến dòng n). Hoán vị dòng chứa phần tử đó với dòng k . Phần tử chốt mới là a_{kk} .
- *Khử ẩn:* Với mỗi dòng i từ $k + 1$ đến n , tính hệ số nhân $m = \frac{a_{ik}}{a_{kk}}$. Thực hiện phép biến đổi dòng: $D_i \leftarrow D_i - m \cdot D_k$. Lúc này, các phần tử dưới a_{kk} sẽ biến thành 0.

2. **Quá trình nghịch (Thế ngược):** Hệ đã trở thành dạng tam giác trên.

- Tính nghiệm cuối: $x_n = \frac{b_n}{a_{nn}}$.
- Thế ngược lên trên: $x_i = \frac{b_i - \sum_{j=i+1}^n a_{ij}x_j}{a_{ii}}$ với $i = n - 1, n - 2, \dots, 1$.

2.2 Ví dụ chi tiết: Phương pháp Gauss

$$\text{Giải hệ: } \begin{cases} 2x + y - z = 8 \\ -3x - y + 2z = -11 \\ -2x + y + 2z = -3 \end{cases}$$

Bước 1: Lập ma trận mở rộng $[A|B]$

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array} \right]$$

Bước 2: Quá trình thuận Tại cột 1, phần tử chốt lớn nhất là $|-3|$. Đổi dòng 1 và dòng 2:

$$\left[\begin{array}{ccc|c} -3 & -1 & 2 & -11 \\ 2 & 1 & -1 & 8 \\ -2 & 1 & 2 & -3 \end{array} \right]$$

Khử cột 1: $D_2 \leftarrow D_2 - \left(\frac{2}{-3}\right) D_1$; $D_3 \leftarrow D_3 - \left(\frac{-2}{-3}\right) D_1$:

$$\left[\begin{array}{ccc|c} -3 & -1 & 2 & -11 \\ 0 & 1/3 & 1/3 & 2/3 \\ 0 & 5/3 & 2/3 & 13/3 \end{array} \right]$$

Tại cột 2, chốt lớn nhất là $5/3$. Đổi dòng 2 và 3:

$$\left[\begin{array}{ccc|c} -3 & -1 & 2 & -11 \\ 0 & 5/3 & 2/3 & 13/3 \\ 0 & 1/3 & 1/3 & 2/3 \end{array} \right]$$

Khử cột 2: $D_3 \leftarrow D_3 - \left(\frac{1/3}{5/3}\right) D_2$:

$$\left[\begin{array}{ccc|c} -3 & -1 & 2 & -11 \\ 0 & 5/3 & 2/3 & 13/3 \\ 0 & 0 & 1/5 & -1/5 \end{array} \right]$$

Bước 3: Quá trình thế ngược

- $z = \frac{-1/5}{1/5} = -1$
- $y = \frac{13/3 - (2/3)(-1)}{5/3} = 3$
- $x = \frac{-11 - (-1)(3) - (2)(-1)}{-3} = 2$

Nghiệm: $x = 2, y = 3, z = -1$.

3 Mã giả (Pseudocode) và Thuật toán

Quy trình thực hiện phương pháp giải hệ phương trình tuyến tính được trình bày chặt chẽ qua các đoạn mã giả dưới đây. Thuật toán yêu cầu đầu vào là ma trận vuông hệ số A không suy biến cấp n và vector hằng số B . Các kỹ thuật chọn chốt từng phần (partial pivoting) được tích hợp nhằm giảm thiểu sai số làm tròn và tránh lỗi chia cho không trong quá trình vận hành.

3.1 Thuật toán Khử Gauss

Algorithm 1 Phương pháp Khử Gauss với kỹ thuật chọn chốt từng phần (Partial Pivoting)

Require: Ma trận hệ số A cấp $n \times n$, vector hằng số B cấp $n \times 1$

Ensure: Vector nghiệm X cấp $n \times 1$ hoặc thông báo lỗi

```
1: Khởi tạo ma trận mở rộng  $M \leftarrow [A|B]$  kích thước  $n \times (n + 1)$ 
2: for  $k = 1, 2, \dots, n - 1$  do ▷ Quá trình thuận (Khử tiến)
3:    $p \leftarrow k$ 
4:    $max\_val \leftarrow |M_{kk}|$ 
5:   for  $i = k + 1, \dots, n$  do ▷ Tìm phần tử chốt có trị tuyệt đối lớn nhất
6:     if  $|M_{ik}| > max\_val$  then
7:        $max\_val \leftarrow |M_{ik}|$ 
8:        $p \leftarrow i$ 
9:     end if
10:  end for
11:  if  $max\_val = 0$  then ▷ Đánh giá điều kiện suy biến
12:    return "Lỗi: Ma trận suy biến"
13:  end if
14:  if  $p \neq k$  then ▷ Hoán vị dòng chứa chốt lên vị trí  $k$ 
15:    Hoán vị dòng  $k$  và dòng  $p$  của  $M$ 
16:  end if
17:  for  $i = k + 1, \dots, n$  do ▷ Khử các phần tử nằm dưới chốt
18:     $m \leftarrow \frac{M_{ik}}{M_{kk}}$ 
19:    for  $j = k, \dots, n + 1$  do
20:       $M_{ij} \leftarrow M_{ij} - m \cdot M_{kj}$ 
21:    end for
22:  end for
23: end for
24: if  $M_{nn} = 0$  then ▷ Kiểm tra phần tử chốt tại vị trí cuối cùng
25:   return "Lỗi: Ma trận suy biến"
26: end if
27:  $X_n \leftarrow \frac{M_{n,n+1}}{M_{nn}}$  ▷ Quá trình nghịch (Thế ngược)
28: for  $i = n - 1, n - 2, \dots, 1$  do
29:    $S \leftarrow 0$ 
30:   for  $j = i + 1, \dots, n$  do
31:      $S \leftarrow S + M_{ij} \cdot X_j$ 
32:   end for
33:    $X_i \leftarrow \frac{M_{i,n+1} - S}{M_{ii}}$ 
34: end for
35: return  $X$ 
```

3.2 Thuật toán Gauss-Jordan

Algorithm 2 Phương pháp Gauss-Jordan

Require: Ma trận hệ số A cấp $n \times n$, vector hằng số B cấp $n \times 1$

Ensure: Vector nghiệm X cấp $n \times 1$ hoặc thông báo lỗi

```
1: Khởi tạo ma trận mở rộng  $M \leftarrow [A|B]$  kích thước  $n \times (n + 1)$ 
2: for  $k = 1, 2, \dots, n$  do ▷ Quá trình khử toàn phần
3:    $p \leftarrow k$ 
4:    $max\_val \leftarrow |M_{kk}|$ 
5:   for  $i = k + 1, \dots, n$  do ▷ Tìm phần tử chốt lớn nhất
6:     if  $|M_{ik}| > max\_val$  then
7:        $max\_val \leftarrow |M_{ik}|$ 
8:        $p \leftarrow i$ 
9:     end if
10:  end for
11:  if  $max\_val = 0$  then
12:    return "Lỗi: Ma trận suy biến"
13:  end if
14:  if  $p \neq k$  then ▷ Hoán vị dòng để đưa chốt lên đường chéo chính
15:    Hoán vị dòng  $k$  và dòng  $p$  của  $M$ 
16:  end if
17:   $pivot \leftarrow M_{kk}$  ▷ Chuẩn hóa dòng chốt về đơn vị
18:  for  $j = k, \dots, n + 1$  do
19:     $M_{kj} \leftarrow \frac{M_{kj}}{pivot}$ 
20:  end for
21:  for  $i = 1, \dots, n$  do ▷ Khử các phần tử ở cả trên và dưới chốt
22:    if  $i \neq k$  then
23:       $factor \leftarrow M_{ik}$ 
24:      for  $j = k, \dots, n + 1$  do
25:         $M_{ij} \leftarrow M_{ij} - factor \cdot M_{kj}$ 
26:      end for
27:    end if
28:  end for
29: end for
30: for  $i = 1, \dots, n$  do ▷ Trích xuất nghiệm từ khối ma trận đường chéo
31:    $X_i \leftarrow M_{i,n+1}$ 
32: end for
33: return  $X$ 
```

4 Phương pháp Gauss-Jordan (Giải hệ $Ax = B$)

4.1 Thuật toán chi tiết

Mục tiêu: Đưa ma trận mở rộng $\bar{A} = [A|B]$ về dạng ma trận đường chéo (khối A thành ma trận đơn vị I). Khối B lúc đó chính là nghiệm x .

1. Lặp k từ 1 đến n :

- Chọn chốt (*Pivoting*) tương tự như Gauss.

- *Chuẩn hóa dòng chốt*: Chia toàn bộ dòng k cho phần tử chốt a_{kk} để a_{kk} biến thành 1. ($D_k \leftarrow \frac{D_k}{a_{kk}}$).
- *Khử TẤT CẢ các dòng khác*: Với mọi dòng $i \neq k$ (cả trên và dưới k), biến đổi $D_i \leftarrow D_i - a_{ik} \cdot D_k$. (Vì a_{kk} đã bằng 1, hệ số nhân chính là phần tử cần khử a_{ik}).

2. Kết thúc vòng lặp, cột mở rộng cuối cùng chính là vector nghiệm. Không cần thế ngược.

4.2 Ví dụ chi tiết: Gauss-Jordan

(Lấy lại kết quả sau khi chọn chốt và chuẩn hóa dòng đầu của hệ trên)

$$\left[\begin{array}{ccc|c} 1 & 1/3 & -2/3 & 11/3 \\ 2 & 1 & -1 & 8 \\ -2 & 1 & 2 & -3 \end{array} \right]$$

Thay vì chỉ khử bên dưới, ở bước chốt thứ 2 (khi cột 2 có phần tử chốt là 1), ta dùng nó để khử cho **cả dòng trên nó (dòng 1)** và **dòng dưới nó (dòng 3)**. Cuối cùng ma trận sẽ có dạng:

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & -1 \end{array} \right]$$

Cột cuối cùng chính là nghiệm: $x = 2, y = 3, z = -1$.

Giải tích số: Phương pháp phân rã LU và Cholesky

1 Phương pháp phân rã LU (Doolittle)

1.1 1. Cơ sở lý thuyết

Mục tiêu của phương pháp là phân tích ma trận hệ số vuông A thành tích của hai ma trận: $A = LU$, trong đó:

- L (Lower) là ma trận tam giác dưới, với các phần tử trên đường chéo chính bằng 1 (phương pháp Doolittle): $l_{ii} = 1$.
- U (Upper) là ma trận tam giác trên.

Điều kiện tồn tại: Phân rã LU không cần hoán vị dòng (pivoting) tồn tại và duy nhất nếu mọi định thức con chính (leading principal minors) của A đều khác 0.

1.2 2. Thuật toán phân rã

Các phần tử của L và U được tính lần lượt theo thứ tự: Hàng 1 của U , Cột 1 của L , Hàng 2 của U , Cột 2 của L ,...

- Tính hàng i của U ($j \geq i$):

$$u_{ij} = a_{ij} - \sum_{k=1}^{i-1} l_{ik} u_{kj} \quad (1)$$

- Tính cột j của L ($i > j$):

$$l_{ij} = \frac{1}{u_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} u_{kj} \right) \quad (2)$$

Algorithm 1 Phương pháp Phân rã LU (Doolittle)

Require: Ma trận vuông A cấp $n \times n$

Ensure: Ma trận tam giác dưới L , ma trận tam giác trên U

```
1: Khởi tạo  $L \leftarrow$  ma trận đơn vị cấp  $n$  ( $l_{ii} = 1$ )
2: Khởi tạo  $U \leftarrow$  ma trận không cấp  $n$ 
3: for  $s = 1, 2, \dots, n$  do
4:   for  $j = s, \dots, n$  do ▷ Tính các phần tử thuộc hàng  $s$  của  $U$ 
5:      $S_U \leftarrow 0$ 
6:     for  $k = 1, \dots, s - 1$  do
7:        $S_U \leftarrow S_U + l_{sk} \cdot u_{kj}$ 
8:     end for
9:      $u_{sj} \leftarrow a_{sj} - S_U$ 
10:  end for
11:  if  $u_{ss} = 0$  then ▷ Đánh giá điều kiện suy biến chốt
12:    return "Lỗi: Không thể phân rã LU (yêu cầu hoán vị dòng)"
13:  end if
14:  for  $i = s + 1, \dots, n$  do ▷ Tính các phần tử thuộc cột  $s$  của  $L$ 
15:     $S_L \leftarrow 0$ 
16:    for  $k = 1, \dots, s - 1$  do
17:       $S_L \leftarrow S_L + l_{ik} \cdot u_{ks}$ 
18:    end for
19:     $l_{is} \leftarrow \frac{a_{is} - S_L}{u_{ss}}$ 
20:  end for
21: end for
22: return  $L, U$ 
```

1.3 3. Giải hệ phương trình $Ax = B$

Hệ $Ax = B \iff LUx = B$. Đặt $Ux = y$, ta có 2 hệ tam giác giải rất nhanh:

1. Giải hệ tam giác dưới $Ly = B$ bằng quá trình thế thuận (từ trên xuống) để tìm y .
2. Giải hệ tam giác trên $Ux = y$ bằng quá trình thế ngược (từ dưới lên) để tìm x .

1.4 4. Ví dụ phân rã LU

Cho ma trận $A = \begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix}$. Phân rã $A = LU$:

- **Hàng 1 của U :** $u_{11} = 2, u_{12} = 1, u_{13} = -1$.
- **Cột 1 của L :** $l_{21} = -3/2 = -1.5, l_{31} = -2/2 = -1$.
- **Hàng 2 của U :** $u_{22} = a_{22} - l_{21}u_{12} = -1 - (-1.5)(1) = 0.5$
 $u_{23} = a_{23} - l_{21}u_{13} = 2 - (-1.5)(-1) = 0.5$
- **Cột 2 của L :** $l_{32} = \frac{a_{32} - l_{31}u_{12}}{u_{22}} = \frac{1 - (-1)(1)}{0.5} = 4$
- **Hàng 3 của U :** $u_{33} = a_{33} - (l_{31}u_{13} + l_{32}u_{23}) = 2 - ((-1)(-1) + (4)(0.5)) = 2 - 3 = -1$.

Kết quả: $L = \begin{bmatrix} 1 & 0 & 0 \\ -1.5 & 1 & 0 \\ -1 & 4 & 1 \end{bmatrix}, U = \begin{bmatrix} 2 & 1 & -1 \\ 0 & 0.5 & 0.5 \\ 0 & 0 & -1 \end{bmatrix}.$

2 Phương pháp Cholesky (LL^T)

2.1 1. Cơ sở lý thuyết

Phương pháp Cholesky phân rã ma trận A thành $A = LL^T$, trong đó L là ma trận tam giác dưới, và L^T là chuyển vị của nó. **Điều kiện áp dụng nghiêm ngặt:** Ma trận A phải là ma trận **đối xứng** ($A = A^T$) và **xác định dương** ($x^T Ax > 0, \forall x \neq 0$, hoặc mọi trị riêng đều dương, hoặc mọi định thức con chính đều > 0).

2.2 2. Thuật toán phân rã

Tính các phần tử của L từng cột một, từ trái qua phải, từ trên xuống dưới:

- Phần tử trên đường chéo chính:

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2} \quad (3)$$

- Các phần tử dưới đường chéo ($i > j$):

$$l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right) \quad (4)$$

Algorithm 2 Phương pháp Phân rã Cholesky

Require: Ma trận vuông đối xứng A cấp $n \times n$

Ensure: Ma trận tam giác dưới L

```
1: Khởi tạo  $L \leftarrow$  ma trận không cấp  $n$ 
2: for  $j = 1, 2, \dots, n$  do
3:    $S_D \leftarrow 0$ 
4:   for  $k = 1, \dots, j - 1$  do
5:      $S_D \leftarrow S_D + l_{jk}^2$ 
6:   end for
7:    $D \leftarrow a_{jj} - S_D$ 
8:   if  $D \leq 0$  then ▷ Kiểm tra tính xác định dương của ma trận
9:     return "Lỗi: Ma trận không xác định dương"
10:  end if
11:   $l_{jj} \leftarrow \sqrt{D}$  ▷ Tính phần tử trên đường chéo chính
12:  for  $i = j + 1, \dots, n$  do ▷ Tính các phần tử dưới đường chéo
13:     $S_L \leftarrow 0$ 
14:    for  $k = 1, \dots, j - 1$  do
15:       $S_L \leftarrow S_L + l_{ik} \cdot l_{jk}$ 
16:    end for
17:     $l_{ij} \leftarrow \frac{a_{ij} - S_L}{l_{jj}}$ 
18:  end for
19: end for
20: return  $L$ 
```

Hệ $Ax = B \iff LL^T x = B$. Ta cũng giải $Ly = B$ (thuận) và $L^T x = y$ (nghịch). Phương pháp này tiết kiệm 50% bộ nhớ và số phép tính so với LU thông thường.

3 Các dạng toán thường gặp

3.1 Dạng 1: Phân rã ma trận và giải hệ

Đề bài: Phân rã LU (hoặc Cholesky) cho ma trận A , từ đó giải hệ $Ax = B$.

Phương pháp: Thực hiện các bước tìm L , U (hoặc L , L^T). Sau đó giải hệ tam giác $Ly = B$ tìm y , và giải $Ux = y$ (hoặc $L^T x = y$) tìm x .

3.2 Dạng 2: Tính định thức của ma trận bằng phân rã

Đề bài: Tính $\det(A)$ dựa vào phân rã LU hoặc Cholesky.

Phương pháp: Dựa vào tính chất $\det(AB) = \det(A)\det(B)$.

- **LU:** $\det(A) = \det(L)\det(U) = 1 \cdot \prod_{i=1}^n u_{ii}$. (Chỉ cần nhân các phần tử trên đường chéo chính của U).
- **Cholesky:** $\det(A) = \det(L)\det(L^T) = (\det(L))^2 = (\prod_{i=1}^n l_{ii})^2$.

3.3 Dạng 3: Tìm ma trận nghịch đảo A^{-1}

Đề bài: Dùng LU (hoặc Cholesky) tìm A^{-1} .

Phương pháp: Giải n hệ phương trình $Ax_i = e_i$ với e_i là cột thứ i của ma trận đơn vị I .

Vector x_i tìm được chính là cột thứ i của ma trận nghịch đảo A^{-1} . Quá trình này rất nhanh vì ta chỉ cần phân rã $A = LU$ một lần duy nhất, sau đó thay đổi các vector về phải e_i .

3.4 Dạng 4: Kiểm tra tính xác định dương của ma trận

Đề bài: Ma trận A có phải là ma trận xác định dương không?

Phương pháp: Áp dụng thuật toán Cholesky. Nếu trong quá trình tính toán, biểu thức trong căn bậc hai $(a_{jj} - \sum l_{jk}^2) \leq 0$, thì thuật toán dừng và kết luận A **không** xác định dương. Nếu phân rã thành công, A xác định dương.

4 Cơ sở lý thuyết: Giải hệ phương trình qua phân rã

Sau khi phân rã ma trận hệ số A thành tích của hai ma trận tam giác (ví dụ $A = LU$ hoặc $A = LL^T$), hệ phương trình ban đầu $Ax = B$ được viết lại thành $LUx = B$. Bằng cách đặt $Ux = y$ (một vector trung gian), bài toán được chia làm hai giai đoạn giải hệ tam giác rất đơn giản:

1. Giải hệ tam giác dưới $Ly = B$ để tìm vector y (Quá trình thế thuận).
2. Giải hệ tam giác trên $Ux = y$ để tìm vector nghiệm x (Quá trình thế ngược).

4.1 1. Quá trình Thế thuận (Forward Substitution)

Hệ phương trình $Ly = B$ có dạng ma trận như sau:

$$\begin{bmatrix} l_{11} & 0 & \cdots & 0 \\ l_{21} & l_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & \cdots & l_{nn} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

Cách giải: Vì đây là ma trận tam giác dưới, phương trình đầu tiên chỉ chứa 1 ẩn y_1 . Ta giải y_1 trước, sau đó *thế thuận* (thế dần xuống các phương trình bên dưới) để tìm $y_2, y_3, \dots, y_n$.

- Phương trình 1: $l_{11}y_1 = b_1 \implies y_1 = \frac{b_1}{l_{11}}$. (Trong LU Doolittle, $l_{11} = 1$).
- Phương trình i ($i = 2, \dots, n$): $l_{i1}y_1 + l_{i2}y_2 + \cdots + l_{ii}y_i = b_i$.
Do đã biết $y_1, \dots, y_{i-1}$, ta suy ra:

$$y_i = \frac{b_i - \sum_{j=1}^{i-1} l_{ij}y_j}{l_{ii}} \quad (5)$$

Algorithm 3 Thuật toán Thế thuận giải hệ $Ly = B$

Require: Ma trận tam giác dưới L cấp $n \times n$, vector hằng số B cấp $n \times 1$

Ensure: Vector nghiệm Y cấp $n \times 1$

- 1: Khởi tạo vector Y kích thước n
 - 2: **for** $i = 1, 2, \dots, n$ **do**
 - 3: $S \leftarrow 0$
 - 4: **for** $j = 1, \dots, i - 1$ **do**
 - 5: $S \leftarrow S + l_{ij} \cdot y_j$
 - 6: **end for**
 - 7: $y_i \leftarrow \frac{b_i - S}{l_{ii}}$
 - 8: **end for**
 - 9: **return** Y
-

4.2 2. Quá trình Thế ngược (Backward Substitution)

Sau khi có vector y , ta giải tiếp hệ $Ux = y$:

$$\begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Cách giải: Phương trình cuối cùng (dòng n) chỉ chứa 1 ẩn x_n . Ta giải x_n trước, sau đó *thế ngược* (thế dần lên các phương trình bên trên) để tìm $x_{n-1}, x_{n-2}, \dots, x_1$.

- Phương trình n : $u_{nn}x_n = y_n \implies x_n = \frac{y_n}{u_{nn}}$.
- Phương trình i (đi từ $n-1$ lùi về 1): $u_{ii}x_i + u_{i,i+1}x_{i+1} + \cdots + u_{in}x_n = y_i$.
Do đã biết $x_{i+1}, \dots, x_n$, ta suy ra:

$$x_i = \frac{y_i - \sum_{j=i+1}^n u_{ij}x_j}{u_{ii}} \quad (6)$$

Algorithm 4 Thuật toán Thế ngược giải hệ $Ux = y$

Require: Ma trận tam giác trên U cấp $n \times n$, vector trung gian Y cấp $n \times 1$

Ensure: Vector nghiệm X cấp $n \times 1$

```
1: Khởi tạo vector  $X$  kích thước  $n$ 
2: for  $i = n, n-1, \dots, 1$  do
3:    $S \leftarrow 0$ 
4:   for  $j = i+1, \dots, n$  do
5:      $S \leftarrow S + u_{ij} \cdot x_j$ 
6:   end for
7:    $x_i \leftarrow \frac{y_i - S}{u_{ii}}$ 
8: end for
9: return  $X$ 
```

4.3 3. Ví dụ chi tiết

Giải sử từ phân rã LU ta có hệ $Ly = B$:

$$\begin{bmatrix} 1 & 0 & 0 \\ -1.5 & 1 & 0 \\ -1 & 4 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 8 \\ -11 \\ -3 \end{bmatrix}$$

Thế thuận tìm y :

- Hàng 1: $1 \cdot y_1 = 8 \implies y_1 = 8$.
- Hàng 2: $-1.5 \cdot y_1 + 1 \cdot y_2 = -11 \implies -1.5(8) + y_2 = -11 \implies -12 + y_2 = -11 \implies y_2 = 1$.
- Hàng 3: $-1 \cdot y_1 + 4 \cdot y_2 + 1 \cdot y_3 = -3 \implies -1(8) + 4(1) + y_3 = -3 \implies -4 + y_3 = -3 \implies y_3 = 1$.

Vậy $y = [8, 1, 1]^T$.

Giải tiếp hệ $Ux = y$:

$$\begin{bmatrix} 2 & 1 & -1 \\ 0 & 0.5 & 0.5 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 8 \\ 1 \\ 1 \end{bmatrix}$$

Thế ngược tìm x :

- Hàng 3: $-1 \cdot x_3 = 1 \implies x_3 = -1$.
- Hàng 2: $0.5 \cdot x_2 + 0.5 \cdot x_3 = 1 \implies 0.5 \cdot x_2 + 0.5(-1) = 1 \implies 0.5 \cdot x_2 = 1.5 \implies x_2 = 3$.
- Hàng 1: $2 \cdot x_1 + 1 \cdot x_2 - 1 \cdot x_3 = 8 \implies 2 \cdot x_1 + 3 - (-1) = 8 \implies 2 \cdot x_1 + 4 = 8 \implies x_1 = 2$.

Nghiệm cuối cùng: $x = [2, 3, -1]^T$.

Giải tích số: Phương pháp lặp đơn và Jacobi giải hệ $Ax = B$

1 Phương pháp Lặp đơn tổng quát

1.1 1. Ý tưởng thuật toán

Để giải hệ phương trình tuyến tính $Ax = B$, ta biến đổi tương đương hệ này về dạng:

$$x = Tx + C \quad (1)$$

Trong đó T là ma trận lặp (kích thước $n \times n$) và C là vector cột (kích thước $n \times 1$). Từ đây, ta xây dựng dãy lặp bắt đầu từ một vector đoán nhận ban đầu $x^{(0)}$:

$$x^{(k+1)} = Tx^{(k)} + C \quad (k = 0, 1, 2, \dots) \quad (2)$$

1.2 2. Điều kiện hội tụ và Chuẩn

Dãy lặp $x^{(k)}$ sẽ hội tụ về nghiệm đúng x^* với mọi vector ban đầu $x^{(0)}$ nếu tồn tại một chuẩn ma trận (matrix norm) sao cho:

$$\|T\| < 1 \quad (3)$$

Các chuẩn ma trận thông dụng thường dùng để kiểm tra:

- **Chuẩn hàng (Chuẩn vô cùng):** $\|T\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |t_{ij}|$ (Max tổng trị tuyệt đối các hàng).
- **Chuẩn cột (Chuẩn 1):** $\|T\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |t_{ij}|$ (Max tổng trị tuyệt đối các cột).

1.3 3. Đánh giá sai số

Gọi $q = \|T\| < 1$. Ta có các công thức đánh giá sai số (thường dùng chuẩn vô cùng $\|\cdot\|_\infty$ cho vector, tức là max trị tuyệt đối các phần tử):

- **Sai số tiên nghiệm (Dự báo số bước lặp):**

$$\|x^{(k)} - x^*\| \leq \frac{q^k}{1 - q} \|x^{(1)} - x^{(0)}\| \quad (4)$$

- **Sai số hậu nghiệm (Điều kiện dừng thực tế):**

$$\|x^{(k)} - x^*\| \leq \frac{q}{1 - q} \|x^{(k)} - x^{(k-1)}\| \quad (5)$$

2 Phương pháp lặp Jacobi

2.1 1. Ý tưởng thuật toán

Phương pháp Jacobi là một trường hợp cụ thể và phổ biến nhất của phương pháp lặp đơn. Từ hệ $Ax = B$, tại phương trình thứ i , ta giữ lại ẩn x_i ở vế trái và chuyển tất cả các ẩn khác sang vế phải:

$$a_{ii}x_i = b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}x_j \implies x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}x_j \right) \quad (6)$$

Công thức lặp Jacobi (tính các phần tử của vector $x^{(k+1)}$ dựa **hoàn toàn** vào vector ở bước trước $x^{(k)}$):

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij}x_j^{(k)} \right), \quad i = 1, 2, \dots, n \quad (7)$$

2.2 2. Điều kiện hội tụ (Ma trận chéo trội)

Phương pháp Jacobi chắc chắn hội tụ nếu ma trận A là **ma trận chéo trội nghiêm ngặt** theo hàng, tức là phần tử trên đường chéo chính có trị tuyệt đối lớn hơn tổng trị tuyệt đối của các phần tử còn lại trên cùng hàng:

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}| \quad \forall i = 1, \dots, n \quad (8)$$

Hệ số co q trong trường hợp này chính là: $q = \max_i \frac{\sum_{j \neq i} |a_{ij}|}{|a_{ii}|} < 1$.

2.3 3. Ví dụ tính tay (2 bước lặp)

Giải hệ phương trình sau bằng Jacobi với $x^{(0)} = [0, 0, 0]^T$:
$$\begin{cases} 10x_1 - x_2 + 2x_3 = 6 \\ -x_1 + 11x_2 - x_3 = 25 \\ 2x_1 - x_2 + 10x_3 = -11 \end{cases}$$

Kiểm tra chéo trội: $|10| > |-1| + |2| = 3$; $|11| > |-1| + |-1| = 2$; $|10| > |2| + |-1| = 3$. Ma trận chéo trội nghiêm ngặt, hệ số co $q = \max(3/10, 2/11, 3/10) = 0.3 < 1$. Hội tụ.

Rút x theo Jacobi:
$$\begin{cases} x_1^{(k+1)} = \frac{1}{10}(6 + x_2^{(k)} - 2x_3^{(k)}) \\ x_2^{(k+1)} = \frac{1}{11}(25 + x_1^{(k)} + x_3^{(k)}) \\ x_3^{(k+1)} = \frac{1}{10}(-11 - 2x_1^{(k)} + x_2^{(k)}) \end{cases}$$

Bước 1 ($k = 0$): Thay $x^{(0)} = [0, 0, 0]^T$:

- $x_1^{(1)} = \frac{1}{10}(6 + 0 - 0) = 0.6$
- $x_2^{(1)} = \frac{1}{11}(25 + 0 + 0) \approx 2.2727$
- $x_3^{(1)} = \frac{1}{10}(-11 - 0 + 0) = -1.1$

$$x^{(1)} = [0.6, 2.2727, -1.1]^T$$

Bước 2 ($k = 1$): Thay $x^{(1)}$ vào vế phải:

- $x_1^{(2)} = \frac{1}{10}(6 + 2.2727 - 2(-1.1)) = 1.0473$
- $x_2^{(2)} = \frac{1}{11}(25 + 0.6 + (-1.1)) = 2.2273$
- $x_3^{(2)} = \frac{1}{10}(-11 - 2(0.6) + 2.2727) = -0.9927$

3 Các dạng toán thường gặp

3.1 Dạng 1: Kiểm tra hội tụ và tìm ma trận lặp

Đề bài: Hệ $Ax = B$ có giải được bằng Jacobi không? Hãy viết ma trận lặp T .

Phương pháp: Kiểm tra tính chéo trội của A . Nếu chưa chéo trội, thử hoán vị các dòng để tạo ra ma trận chéo trội. Ma trận lặp $T = -D^{-1}(L + U)$, với D là ma trận đường chéo, L, U là tam giác dưới và trên của A .

3.2 Dạng 2: Tính tay N bước lặp

Đề bài: Thực hiện 3 bước lặp Jacobi với vector ban đầu cho trước.

Phương pháp: Lập bảng, tính tuần tự dựa vào công thức. Cần cẩn thận: trong Jacobi, để tính $x^{(k+1)}$, ta **chỉ dùng** các giá trị của $x^{(k)}$ (không dùng các giá trị vừa mới tìm được ở cùng bước $k + 1$, đó là phương pháp Gauss-Seidel).

3.3 Dạng 3: Đánh giá sai số tiên nghiệm / hậu nghiệm

Đề bài: Cần lặp ít nhất bao nhiêu bước để đạt sai số ε ? Hoặc đánh giá sai số sau bước lặp thứ 3.

Phương pháp: Tính chuẩn $\|T\|_\infty = q$. Dùng công thức tiên nghiệm để tìm số bước k . Dùng công thức hậu nghiệm để đánh giá sai số thực tế tại bước đó.

4 Mã giả (Pseudocode) và Thuật toán

4.1 Thuật toán Lặp đơn tổng quát (Simple Iteration Method)

Algorithm 1 Phương pháp Lặp đơn tổng quát

Require: Ma trận lặp T cấp $n \times n$, vector hằng số C cấp $n \times 1$, vector khởi tạo X_0 , dung sai ϵ , số vòng lặp tối đa N

Ensure: Vector nghiệm gần đúng X hoặc thông báo lỗi phân kỳ

```
1:  $k \leftarrow 0$ 
2: Khởi tạo vector  $X_{old} \leftarrow X_0$ 
3: Khởi tạo vector  $X_{new}$  kích thước  $n$ 
4: while  $k \leq N$  do
5:   for  $i = 1, 2, \dots, n$  do ▷ Tính vector mới dựa vào ma trận lặp
6:      $S \leftarrow 0$ 
7:     for  $j = 1, 2, \dots, n$  do
8:        $S \leftarrow S + T_{ij} \cdot X_{old}[j]$ 
9:     end for
10:     $X_{new}[i] \leftarrow S + C[i]$ 
11:  end for
12:   $error \leftarrow \max_{1 \leq i \leq n} |X_{new}[i] - X_{old}[i]|$  ▷ Đánh giá sai số theo chuẩn vô cùng
13:  if  $error < \epsilon$  then ▷ Thỏa mãn điều kiện dừng
14:    return  $X_{new}$ 
15:  end if
16:   $X_{old} \leftarrow X_{new}$  ▷ Cập nhật vector cho vòng lặp tiếp theo
17:   $k \leftarrow k + 1$ 
18: end while
19: return "Thuật toán không hội tụ sau  $N$  vòng lặp"
```

4.2 Thuật toán Lặp Jacobi

Algorithm 2 Phương pháp Lặp Jacobi

Require: Ma trận hệ số A cấp $n \times n$ (đã thỏa mãn chéo trội), vector B cấp $n \times 1$, vector khởi tạo X_0 , dung sai ϵ , số vòng lặp tối đa N

Ensure: Vector nghiệm gần đúng X hoặc thông báo lỗi phân kỳ

```
1:  $k \leftarrow 0$ 
2: Khởi tạo vector  $X_{old} \leftarrow X_0$ 
3: Khởi tạo vector  $X_{new}$  kích thước  $n$ 
4: while  $k \leq N$  do
5:   for  $i = 1, 2, \dots, n$  do                                ▷ Tính từng thành phần của vector nghiệm mới
6:      $S \leftarrow 0$ 
7:     for  $j = 1, 2, \dots, n$  do
8:       if  $j \neq i$  then
9:          $S \leftarrow S + A_{ij} \cdot X_{old}[j]$ 
10:      end if
11:    end for
12:     $X_{new}[i] \leftarrow \frac{B[i] - S}{A_{ii}}$ 
13:  end for
14:   $error \leftarrow \max_{1 \leq i \leq n} |X_{new}[i] - X_{old}[i]|$           ▷ Kiểm tra sai số chuẩn vô cùng
15:  if  $error < \epsilon$  then
16:    return  $X_{new}$ 
17:  end if
18:   $X_{old} \leftarrow X_{new}$                                        ▷ Đồng bộ hóa vector trước khi chuyển sang vòng lặp mới
19:   $k \leftarrow k + 1$ 
20: end while
21: return "Thuật toán không hội tụ sau  $N$  vòng lặp"
```

Giải tích số: Phương pháp lặp Gauss-Seidel (Lặp Seidel)

1 Cơ sở lý thuyết

1.1 1. Ý tưởng thuật toán và Biểu diễn toán học

Phương pháp Gauss-Seidel (hay phương pháp Seidel) là một sự cải tiến của phương pháp lặp Jacobi trong việc giải hệ phương trình đại số tuyến tính $Ax = B$.

Trong phương pháp Jacobi, để tính vector ở bước lặp $(k+1)$, ta sử dụng hoàn toàn vector ở bước lặp (k) . Điều này làm lãng phí thông tin vì khi tính đến thành phần $x_i^{(k+1)}$, các thành phần đứng trước nó là $x_1, x_2, \dots, x_{i-1}$ đã được tính ra giá trị mới (chính xác hơn) ở bước $(k+1)$ rồi.

Phương pháp Gauss-Seidel khắc phục điều này bằng cách **sử dụng ngay lập tức** các giá trị mới nhất vừa được tính. Công thức lặp tại thành phần thứ i là:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right) \quad (1)$$

Trong đó:

- Tổng $\sum_{j=1}^{i-1}$: Sử dụng các nghiệm $x_j^{(k+1)}$ đã được cập nhật trong cùng bước lặp hiện tại.
- Tổng $\sum_{j=i+1}^n$: Sử dụng các nghiệm $x_j^{(k)}$ cũ từ bước lặp trước.

1.2 2. Phân tích dưới dạng Ma trận (Matrix Form)

Dưới góc độ giải tích số, hệ $Ax = B$ có thể được phân tích bằng cách tách ma trận hệ số A thành 3 thành phần: $A = L + D + U$, trong đó:

- D : Ma trận đường chéo chính.
- L : Ma trận tam giác dưới ngặt (strictly lower triangular).
- U : Ma trận tam giác trên ngặt (strictly upper triangular).

Phương trình lặp Gauss-Seidel có thể biểu diễn qua ma trận như sau:

$$(D + L)x^{(k+1)} = -Ux^{(k)} + B \quad \Rightarrow \quad x^{(k+1)} = -(D + L)^{-1}Ux^{(k)} + (D + L)^{-1}B \quad (2)$$

Từ đây, ta xác định được **ma trận lặp** của phương pháp Gauss-Seidel là $T_G = -(D + L)^{-1}U$.

1.3 3. Điều kiện hội tụ và Tốc độ hội tụ

- **Điều kiện cần và đủ:** Thuật toán hội tụ với mọi vector khởi tạo $x^{(0)}$ khi và chỉ khi bán kính phổ của ma trận lặp nhỏ hơn 1, tức là $\rho(T_G) < 1$.
- **Điều kiện đủ (Thường dùng trong thực hành):** Hệ chắc chắn hội tụ nếu ma trận A thỏa mãn:
 1. A là ma trận **chéo trội nghiêm ngặt** theo hàng hoặc cột.
 2. *Hoặc* A là ma trận **đối xứng và xác định dương** ($A = A^T$ và mọi định thức con chính đều > 0).

2 Mã giả thuật toán (Pseudocode)

Algorithm 1 Phương pháp lặp Gauss-Seidel

Require: Ma trận $A_{n \times n}$, vector B , vector khởi tạo $x^{(0)}$, sai số tol , số tối đa M

Ensure: Vector nghiệm gần đúng x

```
1:  $x \leftarrow x^{(0)}$ 
2: for  $k = 1$  to  $M$  do
3:    $x\_old \leftarrow x$  ▷ Lưu lại trạng thái cũ để tính sai số
4:   for  $i = 1$  to  $n$  do
5:      $sum \leftarrow 0$ 
6:     for  $j = 1$  to  $i - 1$  do
7:        $sum \leftarrow sum + A_{ij} \cdot x_j$  ▷ Dùng giá trị  $x$  MỚI nhất
8:     end for
9:     for  $j = i + 1$  to  $n$  do
10:       $sum \leftarrow sum + A_{ij} \cdot x\_old_j$  ▷ Dùng giá trị  $x$  CŨ
11:    end for
12:     $x_i \leftarrow \frac{B_i - sum}{A_{ii}}$  ▷ Cập nhật trực tiếp vào  $x_i$ 
13:  end for
14:  if  $\|x - x\_old\|_\infty < tol$  then
15:    break ▷ Thỏa mãn chuẩn sai số, kết thúc thuật toán
16:  end if
17: end for
18: return  $x$ 
```

3 Ví dụ tính toán chi tiết (Phân tích từ Ma trận gốc)

Xét hệ phương trình tuyến tính $Ax = B$ với vector khởi tạo $x^{(0)} = [0, 0, 0]^T$:

$$\begin{bmatrix} 10 & -1 & 2 \\ -1 & 11 & -1 \\ 2 & -1 & 10 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 6 \\ 25 \\ -11 \end{bmatrix}$$

Bước 1: Xét điều kiện hội tụ chi tiết

Ma trận hệ số A của hệ phương trình là:

$$A = \begin{bmatrix} 10 & -1 & 2 \\ -1 & 11 & -1 \\ 2 & -1 & 10 \end{bmatrix}$$

Ta kiểm tra điều kiện hội tụ bằng cách xét tính chéo trội nghiêm ngặt theo hàng. So sánh trị tuyệt đối phần tử trên đường chéo chính với tổng trị tuyệt đối các phần tử còn lại trên cùng hàng:

- **Hàng 1:** $|a_{11}| = 10$; $\sum_{j \neq 1} |a_{1j}| = |-1| + |2| = 3$. Vì $10 > 3 \Rightarrow$ Thỏa mãn.
- **Hàng 2:** $|a_{22}| = 11$; $\sum_{j \neq 2} |a_{2j}| = |-1| + |-1| = 2$. Vì $11 > 2 \Rightarrow$ Thỏa mãn.
- **Hàng 3:** $|a_{33}| = 10$; $\sum_{j \neq 3} |a_{3j}| = |2| + |-1| = 3$. Vì $10 > 3 \Rightarrow$ Thỏa mãn.

Kết luận: Ma trận A là ma trận chéo trội nghiêm ngặt theo hàng. Theo định lý hội tụ, phương pháp lặp Gauss-Seidel **chắc chắn hội tụ** đối với mọi vector khởi tạo $x^{(0)}$.

Bước 2: Phân tích ma trận và Sự hình thành công thức lặp

Dựa vào cơ sở lý thuyết, ta tách ma trận hệ số $A = D + L + U$:

$$D = \begin{bmatrix} 10 & 0 & 0 \\ 0 & 11 & 0 \\ 0 & 0 & 10 \end{bmatrix}, \quad L = \begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 0 \\ 2 & -1 & 0 \end{bmatrix}, \quad U = \begin{bmatrix} 0 & -1 & 2 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{bmatrix}$$

Công thức lặp ma trận của Gauss-Seidel là $(D + L)x^{(k+1)} = -Ux^{(k)} + B$. Thay các ma trận vào, ta có:

$$\begin{bmatrix} 10 & 0 & 0 \\ -1 & 11 & 0 \\ 2 & -1 & 10 \end{bmatrix} \begin{bmatrix} x_1^{(k+1)} \\ x_2^{(k+1)} \\ x_3^{(k+1)} \end{bmatrix} = \begin{bmatrix} 0 & 1 & -2 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1^{(k)} \\ x_2^{(k)} \\ x_3^{(k)} \end{bmatrix} + \begin{bmatrix} 6 \\ 25 \\ -11 \end{bmatrix}$$

Thực hiện phép nhân ma trận ở cả hai vế và viết lại dưới dạng hệ phương trình:

$$\begin{cases} 10x_1^{(k+1)} = x_2^{(k)} - 2x_3^{(k)} + 6 \\ -x_1^{(k+1)} + 11x_2^{(k+1)} = x_3^{(k)} + 25 \\ 2x_1^{(k+1)} - x_2^{(k+1)} + 10x_3^{(k+1)} = -11 \end{cases}$$

Chuyển vế để rút các phần tử đường chéo $x^{(k+1)}$ ra độc lập, ta sẽ thấy rõ bản chất "kế thừa dữ liệu ngay lập tức" của thuật toán:

$$\begin{cases} x_1^{(k+1)} = \frac{1}{10} [6 + x_2^{(k)} - 2x_3^{(k)}] \\ x_2^{(k+1)} = \frac{1}{11} [25 + \mathbf{x}_1^{(k+1)} + x_3^{(k)}] \leftarrow \text{Buộc phải dùng } x_1^{(k+1)} \text{ vừa tính ở trên} \\ x_3^{(k+1)} = \frac{1}{10} [-11 - 2\mathbf{x}_1^{(k+1)} + \mathbf{x}_2^{(k+1)}] \leftarrow \text{Dùng ngay } x_1^{(k+1)} \text{ và } x_2^{(k+1)} \end{cases}$$

Bước 3: Trình bày các bước lặp số học chi tiết

Lần lặp $k = 1$ (Sử dụng vector khởi tạo $x^{(0)} = [0, 0, 0]^T$):

- $x_1^{(1)} = \frac{1}{10} \left[6 + 0.0000^{(\text{cũ})} - 2(0.0000^{(\text{cũ})}) \right] = \mathbf{0.6000}$
- $x_2^{(1)} = \frac{1}{11} \left[25 + \mathbf{0.6000}^{(\text{mới})} + 0.0000^{(\text{cũ})} \right] = \mathbf{2.3273}$
- $x_3^{(1)} = \frac{1}{10} \left[-11 - 2(\mathbf{0.6000}^{(\text{mới})}) + \mathbf{2.3273}^{(\text{mới})} \right] = \mathbf{-0.9873}$

$\Rightarrow$ Sai số chuẩn vô cùng: $\|x^{(1)} - x^{(0)}\|_\infty = \max(|0.6|, |2.3273|, |-0.9873|) = 2.3273$

Lần lặp $k = 2$ (Sử dụng trạng thái $x^{(1)}$ làm dữ liệu cũ):

- $x_1^{(2)} = \frac{1}{10} \left[6 + 2.3273^{(\text{cũ})} - 2(-0.9873^{(\text{cũ})}) \right] = \mathbf{1.0302}$
- $x_2^{(2)} = \frac{1}{11} \left[25 + \mathbf{1.0302}^{(\text{mới})} + (-0.9873^{(\text{cũ})}) \right] = \mathbf{2.2766}$
- $x_3^{(2)} = \frac{1}{10} \left[-11 - 2(\mathbf{1.0302}^{(\text{mới})}) + \mathbf{2.2766}^{(\text{mới})} \right] = \mathbf{-1.0784}$

$\Rightarrow$ Tính sai số hậu nghiệm:

$$\begin{aligned} \|x^{(2)} - x^{(1)}\|_\infty &= \max(|1.0302 - 0.6000|, |2.2766 - 2.3273|, |-1.0784 - (-0.9873)|) \\ &= \max(0.4302, |-0.0507|, |-0.0911|) = 0.4302 \end{aligned}$$

Nhận xét: Dù mới trải qua 2 bước lặp, nghiệm xấp xỉ $x^{(2)} = [1.0302, 2.2766, -1.0784]^T$ đã tiến về rất gần với nghiệm chính xác của hệ là $x^* = [1, 2, -1]^T$.

4 Các dạng toán thường gặp

4.1 Dạng 1: Trình bày N bước lặp Gauss-Seidel

Đề bài: Áp dụng lặp Seidel, tìm nghiệm sau N bước lặp và đánh giá sai số.

Phương pháp: Lập bảng tính toán tuần tự. Phải rất cẩn thận dùng đúng giá trị *mới nhất* vừa tìm được để thế vào phương trình ngay bên dưới nó. Sai số thường được đánh giá bằng chuẩn vô cùng (sai lệch lớn nhất giữa 2 bước lặp liên tiếp).

4.2 Dạng 2: Giải hệ phương trình có ma trận đối xứng xác định dương

Đề bài: Hệ phương trình không thỏa mãn điều kiện chéo trội nhưng có ma trận đối xứng. Có thể dùng Gauss-Seidel không?

Phương pháp: Kiểm tra định lý Sylvester xem các định thức con chính có dương không ($\Delta_1 > 0, \Delta_2 > 0 \dots$). Nếu có, ma trận là xác định dương. Dựa vào định lý hội tụ, lặp Gauss-Seidel **chắc chắn hội tụ** (Trong khi lặp Jacobi ở trường hợp này có thể phân kỳ).

Giải tích số: Tìm ma trận nghịch đảo bằng phương pháp Giải đúng

1 Cơ sở lý thuyết

1.1 1. Điều kiện tồn tại ma trận nghịch đảo

Cho ma trận A cấp $m \times n$. Ma trận A có ma trận nghịch đảo (kí hiệu là A^{-1}) khi và chỉ khi thỏa mãn đồng thời hai điều kiện sau:

- A là ma trận **vuông** ($m = n$).
- Định thức của A khác không ($\det(A) \neq 0$). Những ma trận như vậy được gọi là ma trận **không suy biến**.

Trước khi áp dụng bất kỳ thuật toán tìm nghịch đảo nào, việc kiểm tra $\det(A) \neq 0$ là bước bắt buộc để đảm bảo bài toán có nghiệm và tránh lỗi chia cho 0 trong quá trình tính toán.

1.2 2. Nguyên lý hệ phương trình

Ma trận nghịch đảo A^{-1} là ma trận thỏa mãn $A \cdot A^{-1} = I$, với I là ma trận đơn vị cấp n . Nếu ta gọi x_j là cột thứ j của ma trận A^{-1} và e_j là cột thứ j của ma trận đơn vị I , ta có hệ thức:

$$A \cdot [x_1, x_2, \dots, x_n] = [e_1, e_2, \dots, e_n] \quad (1)$$

Điều này tương đương với việc giải n hệ phương trình tuyến tính có cùng ma trận hệ số A :

$$Ax_j = e_j, \quad j = 1, 2, \dots, n \quad (2)$$

1.3 3. Phương pháp Gauss-Jordan (Phổ biến nhất)

Đây là phương pháp trực quan và thường được sử dụng nhất để lập trình cũng như tính tay:

- **Bước 1:** Lập ma trận mở rộng ghép với ma trận đơn vị: $[A|I]$.
- **Bước 2:** Sử dụng các phép biến đổi sơ cấp trên dòng (nhân, chia, cộng, trừ các dòng) để biến đổi khối A thành ma trận đơn vị I .
- **Bước 3:** Khi đó, khối I ban đầu ở bên phải sẽ tự động biến thành A^{-1} .
- **Sơ đồ tổng quát:** $[A|I] \xrightarrow{\text{Khử Gauss-Jordan}} [I|A^{-1}]$.

1.4 4. Phương pháp phân rã LU

Khi cần tìm A^{-1} cho các ma trận cực lớn, việc dùng LU giúp tối ưu chi phí tính toán:

- **Bước 1:** Phân rã $A = LU$ một lần duy nhất.
- **Bước 2:** Giải n hệ phương trình: $L(Ux_j) = e_j$ bằng cách thế thuận và thế ngược cho từng cột e_j .

2 Mã giả thuật toán Gauss-Jordan (Pseudocode)

Dưới đây là mã giả mô tả thuật toán tìm ma trận nghịch đảo, có kết hợp kiểm tra tính suy biến và hoán vị dòng (Partial Pivoting) để đảm bảo độ chính xác.

Algorithm 1 Tìm ma trận nghịch đảo bằng Gauss-Jordan

Require: Ma trận vuông $A_{n \times n}$

Ensure: Ma trận nghịch đảo A^{-1} hoặc báo lỗi nếu ma trận suy biến

```
1: if  $\det(A) = 0$  then
2:   return "Lỗi: Ma trận suy biến, không tồn tại nghịch đảo."
3: end if
4: Khởi tạo ma trận đơn vị  $I_{n \times n}$ 
5: Tạo ma trận mở rộng  $AI \leftarrow [A|I]$ 
6: for  $k = 1$  to  $n$  do
7:   // Chọn phần tử chốt lớn nhất để giảm sai số (Partial Pivoting)
8:   Tìm dòng  $p \geq k$  sao cho  $|AI_{p,k}|$  đạt Max
9:   if  $p \neq k$  then
10:    Hoán vị dòng  $k$  và dòng  $p$  của ma trận  $AI$ 
11:   end if
12:   // Chuẩn hóa dòng chốt
13:    $pivot \leftarrow AI_{k,k}$ 
14:    $AI_k \leftarrow \frac{AI_k}{pivot}$  ▷ Làm cho phần tử đường chéo bằng 1
15:   // Khử tất cả các dòng khác (Cả trên và dưới chốt)
16:   for  $i = 1$  to  $n$  do
17:     if  $i \neq k$  then
18:        $factor \leftarrow AI_{i,k}$ 
19:        $AI_i \leftarrow AI_i - factor \times AI_k$  ▷ Làm cho các phần tử cùng cột  $k$  bằng 0
20:     end if
21:   end for
22: end for
23: // Tách khối bên phải để lấy  $A^{-1}$ 
24:  $A^{-1} \leftarrow$  Các cột từ  $n + 1$  đến  $2n$  của  $AI$ 
25: return  $A^{-1}$ 
```

3 Ví dụ chi tiết: Tìm nghịch đảo bằng Gauss-Jordan

Tìm nghịch đảo của ma trận $A = \begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix}$.

Bước 0: Kiểm tra điều kiện tồn tại

Định thức $\det(A) = 2 \cdot 3 - 1 \cdot 5 = 6 - 5 = 1 \neq 0$.

$\Rightarrow$ Ma trận không suy biến, chắc chắn tồn tại nghịch đảo.

Bước 1: Lập ma trận mở rộng $[A|I]$

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 5 & 3 & 0 & 1 \end{array} \right]$$

Bước 2: Chuẩn hóa a_{11} thành 1

Chia toàn bộ dòng 1 cho 2 ($D_1 \leftarrow D_1/2$).

$$\left[\begin{array}{cc|cc} 1 & 0.5 & 0.5 & 0 \\ 5 & 3 & 0 & 1 \end{array} \right]$$

Bước 3: Khử a_{21} về 0

Trừ dòng 2 cho 5 lần dòng 1 ($D_2 \leftarrow D_2 - 5D_1$).

$$\left[\begin{array}{cc|cc} 1 & 0.5 & 0.5 & 0 \\ 0 & 0.5 & -2.5 & 1 \end{array} \right]$$

Bước 4: Chuẩn hóa a_{22} thành 1

Chia toàn bộ dòng 2 cho 0.5, tương đương nhân 2 ($D_2 \leftarrow 2D_2$).

$$\left[\begin{array}{cc|cc} 1 & 0.5 & 0.5 & 0 \\ 0 & 1 & -5 & 2 \end{array} \right]$$

Bước 5: Khử a_{12} về 0 (Hoàn tất Gauss-Jordan)

Trừ dòng 1 cho 0.5 lần dòng 2 ($D_1 \leftarrow D_1 - 0.5D_2$).

$$\left[\begin{array}{cc|cc} 1 & 0 & 3 & -1 \\ 0 & 1 & -5 & 2 \end{array} \right]$$

Kết luận: Khối bên phải lúc này chính là A^{-1} .

$$A^{-1} = \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix}$$

4 Các dạng toán thường gặp

- **Dạng 1:** Tìm ma trận nghịch đảo bằng phương pháp Gauss-Jordan (Trình bày bảng biến đổi từng bước).
- **Dạng 2:** Đánh giá tính suy biến bằng định thức $\det(A)$ hoặc các phần tử chốt (Pivot) trong quá trình biến đổi trước khi kết luận hệ phương trình có nghiệm hay không.
- **Dạng 3:** Giải phương trình ma trận $AX = B \implies X = A^{-1}B$ (Tìm A^{-1} trước rồi nhân ma trận thay vì giải trực tiếp).

Giải tích số: Phương pháp viền quanh tìm ma trận nghịch đảo

1 Cơ sở lý thuyết

1.1 1. Ý tưởng thuật toán

Giả sử ta đã biết ma trận nghịch đảo của ma trận A_k cấp k . Ta muốn tìm nghịch đảo của ma trận A_{k+1} cấp $k+1$ được tạo ra bằng cách thêm một dòng và một cột vào A_k . Ma trận A_{k+1} được viết dưới dạng khối:

$$A_{k+1} = \begin{bmatrix} A_k & u \\ v^T & a_{k+1,k+1} \end{bmatrix} \quad (1)$$

Trong đó:

- A_k : Ma trận vuông cấp k đã biết A_k^{-1} .
- u : Vector cột cấp k .
- v^T : Vector dòng cấp k .
- $a_{k+1,k+1}$: Phần tử góc dưới bên phải.

1.2 2. Công thức tính nghịch đảo khối

Ma trận nghịch đảo A_{k+1}^{-1} cũng được biểu diễn dưới dạng khối:

$$A_{k+1}^{-1} = \begin{bmatrix} P & q \\ r^T & \alpha \end{bmatrix} \quad (2)$$

Các thành phần được tính toán theo trình tự sau:

1. Tính số α (Số Schur):

$$\alpha = \frac{1}{a_{k+1,k+1} - v^T A_k^{-1} u} \quad (3)$$

2. Tính vector cột q :

$$q = -\alpha(A_k^{-1}u) \quad (4)$$

3. Tính vector dòng r^T :

$$r^T = -\alpha(v^T A_k^{-1}) \quad (5)$$

4. Tính khối ma trận P :

$$P = A_k^{-1} + \frac{1}{\alpha} q r^T = A_k^{-1} + \alpha(A_k^{-1}u)(v^T A_k^{-1}) \quad (6)$$

1.3 3. Quy trình thực hiện

- Bắt đầu từ $A_1 = [a_{11}] \implies A_1^{-1} = [1/a_{11}]$.
- Viên quanh A_1 để tìm A_2^{-1} .
- Tiếp tục viên quanh A_2 để tìm $A_3^{-1}, \dots$ cho đến cấp n .

1.4 4. Ngoại lệ: Trường hợp $a_{11} = 0$ hoặc ma trận con suy biến

Phương pháp viên quanh có một hạn chế lớn là yêu cầu tất cả các ma trận con chính (leading principal submatrices) $A_1, A_2, \dots, A_n$ đều phải khả nghịch. Nếu tại một bước nào đó mẫu số của α bằng 0 (ví dụ ngay bước đầu tiên $a_{11} = 0$), thuật toán sẽ thất bại do gặp lỗi chia cho 0.

Cách khắc phục (Chiến lược hoán vị - Pivoting):

- Trước khi bắt đầu, ta cần tìm một phần tử khác 0 (tốt nhất là phần tử có giá trị tuyệt đối lớn nhất) trong ma trận và tiến hành **hoán vị dòng/cột** để đưa nó lên vị trí góc trái a_{11} .
- Tương tự, nếu ở bước k nào đó ma trận con bị suy biến, ta cũng cần hoán vị để đảm bảo $a_{k+1,k+1} - v^T A_k^{-1} u \neq 0$.
- Lưu ý: Do tác động của phép hoán vị dòng/cột trên ma trận gốc, sau khi hoàn tất thuật toán, ta phải thực hiện các phép hoán vị ngược lại trên ma trận kết quả để trả về đúng cấu trúc của ma trận nghịch đảo ban đầu.

Cách khắc phục (Chiến lược hoán vị - Pivoting):

- Trước khi bắt đầu, ta cần tìm một phần tử khác 0 (tốt nhất là phần tử có giá trị tuyệt đối lớn nhất) trong ma trận và tiến hành **hoán vị dòng/cột** để đưa nó lên vị trí góc trái a_{11} .
- Tương tự, nếu ở bước k nào đó ma trận con bị suy biến, ta cũng cần hoán vị để đảm bảo $a_{k+1,k+1} - v^T A_k^{-1} u \neq 0$.
- **Lưu ý quan trọng (Nguyên lý đối ngẫu):** Do tác động của phép hoán vị dòng/cột trên ma trận gốc, sau khi hoàn tất thuật toán, ta phải thực hiện các phép hoán vị ngược lại trên ma trận kết quả để trả về đúng cấu trúc của ma trận nghịch đảo ban đầu.

1. Phân tích phương trình nghịch đảo (Cơ sở toán học):

– Trường hợp 1: Hoán vị dòng (Row Permutation)

Khi hoán vị dòng i và dòng j của A , phương trình đại số là:

$$A_{new} = P \cdot A$$

Lấy nghịch đảo hai vế:

$$A_{new}^{-1} = (P \cdot A)^{-1} = A^{-1} \cdot P^{-1}$$

Nhân cả hai vế với P từ bên phải (lưu ý $P^{-1} \cdot P = I$):

$$A^{-1} = A_{new}^{-1} \cdot P$$

Hệ quả: Phép nhân ma trận P từ bên phải có tác dụng hoán vị cột. Vậy, nếu hoán vị dòng i và j trên ma trận gốc A , ta phải **hoán vị cột** i và j trên ma trận kết quả A_{new}^{-1} để thu được A^{-1} .

– **Trường hợp 2: Hoán vị cột (Column Permutation)**

Khi hoán vị cột i và cột j của A :

$$A_{new} = A \cdot P$$

Lấy nghịch đảo hai vế:

$$A_{new}^{-1} = (A \cdot P)^{-1} = P^{-1} \cdot A^{-1}$$

Nhân cả hai vế với P từ bên trái:

$$A^{-1} = P \cdot A_{new}^{-1}$$

Hệ quả: Phép nhân ma trận P từ bên trái có tác dụng hoán vị dòng. Vậy, nếu hoán vị cột i và j trên ma trận gốc A , ta phải **hoán vị dòng i và j** trên ma trận kết quả A_{new}^{-1} .

2. Quy tắc hệ thống hóa:

- Hoán vị dòng trên $A \Rightarrow$ Hoán vị cột tương ứng trên ma trận nghịch đảo.
- Hoán vị cột trên $A \Rightarrow$ Hoán vị dòng tương ứng trên ma trận nghịch đảo.

Ví dụ minh họa: Tìm nghịch đảo của hệ ma trận suy biến chốt $A = \begin{bmatrix} 0 & 2 \\ 3 & 4 \end{bmatrix}$.

1. **Khởi tạo:** Do $a_{11} = 0$, tiến hành **hoán vị dòng 1 và dòng 2** trên A để thiết lập phần tử chốt hợp lệ:

$$A_{new} = \begin{bmatrix} 3 & 4 \\ 0 & 2 \end{bmatrix}$$

2. **Tính nghịch đảo trung gian:** Áp dụng tính toán trên ma trận mới thu được:

$$A_{new}^{-1} = \frac{1}{\det(A_{new})} \begin{bmatrix} 2 & -4 \\ 0 & 3 \end{bmatrix} = \begin{bmatrix} 1/3 & -2/3 \\ 0 & 1/2 \end{bmatrix}$$

3. **Phục hồi không gian vector:** Áp dụng quy tắc hoán vị ngược. Do bước 1 đã tiến hành *hoán vị dòng* trên A , ta thực hiện **hoán vị cột 1 và cột 2** trên A_{new}^{-1} để xuất kết quả cuối cùng:

$$A^{-1} = \begin{bmatrix} -2/3 & 1/3 \\ 1/2 & 0 \end{bmatrix}$$

2 Ví dụ minh họa (Từ cấp 2 lên cấp 3)

Giả sử có $A_2^{-1} = \begin{bmatrix} 0.5 & -0.5 \\ 0 & 1 \end{bmatrix}$ và cần tìm A_3^{-1} với $A_3 = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$.

Ở đây $u = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $v^T = [1 \ 0]$, $a_{33} = 1$.

1. $A_2^{-1}u = \begin{bmatrix} 0.5 & -0.5 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$.

$$2. \ v^T A_2^{-1} = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 0.5 & -0.5 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.5 & -0.5 \end{bmatrix}.$$

$$3. \ v^T (A_2^{-1} u) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0.$$

$$4. \ \alpha = \frac{1}{1-0} = 1.$$

$$5. \ q = -1 \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}.$$

$$6. \ r^T = -1 \cdot \begin{bmatrix} 0.5 & -0.5 \end{bmatrix} = \begin{bmatrix} -0.5 & 0.5 \end{bmatrix}.$$

$$7. \ P = A_2^{-1} + \frac{1}{1} \begin{bmatrix} 0 \\ -1 \end{bmatrix} \begin{bmatrix} -0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} 0.5 & -0.5 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0.5 & -0.5 \end{bmatrix} = \begin{bmatrix} 0.5 & -0.5 \\ 0.5 & 0.5 \end{bmatrix}.$$

8. Kết luận ma trận nghịch đảo cần tìm là ghép các khối P, q, r^T, α lại:

$$A_3^{-1} = \begin{bmatrix} 0.5 & -0.5 & 0 \\ 0.5 & 0.5 & -1 \\ -0.5 & 0.5 & 1 \end{bmatrix}$$

3 Mã giả (Pseudocode)

Algorithm 1 Phương pháp Viền quanh (Bordering Method) tìm nghịch đảo ma trận

Require: Ma trận vuông $A \in \mathbb{R}^{n \times n}$, ngưỡng sai số ϵ (ví dụ 10^{-12}).

Ensure: Ma trận nghịch đảo A^{-1} , hoặc thông báo lỗi nếu A suy biến.

```
1: // 1. Khởi tạo ma trận con cấp 1
2: if  $|A_{1,1}| < \epsilon$  then
3:   return Lỗi: "Phần tử chốt  $a_{11} \approx 0$ . Cần hoán vị dòng/cột trước khi chạy!"
4: end if
5:  $A_{inv} \leftarrow \begin{bmatrix} \frac{1}{A_{1,1}} \end{bmatrix}$ 
6: // 2. Xây dựng dần ma trận nghịch đảo từ cấp 2 đến  $n$ 
7: for  $k = 1$  tới  $n - 1$  do
8:   - Trích xuất các khối từ ma trận  $A$  gốc:
9:    $u \leftarrow A_{1..k, k+1}$  ▷ Lấy vector cột
10:   $v^T \leftarrow A_{k+1, 1..k}$  ▷ Lấy vector dòng
11:   $a_{corner} \leftarrow A_{k+1, k+1}$  ▷ Lấy phần tử góc chốt
12:  - Tính mẫu số cho hệ số vô hướng (Phần bù Schur):
13:   $denom \leftarrow a_{corner} - v^T \cdot A_{inv} \cdot u$ 
14:  if  $|denom| < \epsilon$  then
15:    return Lỗi: "Ma trận con cấp  $k + 1$  suy biến. Thuật toán thất bại!"
16:  end if
17:  - Tính toán các khối thành phần:
18:   $\alpha \leftarrow \frac{1}{denom}$ 
19:   $q \leftarrow -\alpha \cdot (A_{inv} \cdot u)$ 
20:   $r^T \leftarrow -\alpha \cdot (v^T \cdot A_{inv})$ 
21:   $P \leftarrow A_{inv} + \frac{1}{\alpha} \cdot (q \cdot r^T)$ 
22:  - Ghép khối cập nhật  $A_{inv}$  cho bước lặp tiếp theo:
23:   $A_{inv} \leftarrow \begin{bmatrix} P & q \\ r^T & \alpha \end{bmatrix}$ 
24: end for
25: return  $A_{inv}$ 
```

4 Bổ túc toán học: Lý thuyết hoán vị ma trận

Như đã đề cập ở phần ngoại lệ, khi gặp phần tử chốt bằng 0 (hoặc ma trận con bị suy biến), ta bắt buộc phải sử dụng chiến lược hoán vị (Pivoting). Để thao tác hoán vị này chính xác về mặt toán học và dễ dàng lập trình, ta cần sử dụng công cụ **Ma trận hoán vị (Permutation Matrix)**.

4.1 1. Định nghĩa và tính chất

Ma trận hoán vị P là một ma trận vuông được tạo ra bằng cách thực hiện phép hoán vị các dòng (hoặc các cột) tương ứng trên ma trận đơn vị I .

Tính chất tác động: Giả sử ta có ma trận gốc A .

- **Nhân bên trái ($P \cdot A$):** Tác động hoán vị lên các **dòng** của ma trận A .
- **Nhân bên phải ($A \cdot P$):** Tác động hoán vị lên các **cột** của ma trận A .

Các tính chất đại số quan trọng:

1. **Định thức:** Mỗi lần hoán vị hai dòng (hoặc hai cột), định thức đổi dấu. Do đó $\det(P) = \pm 1$. Tính khả nghịch của ma trận A được bảo toàn sau khi hoán vị.
2. **Tính trực giao (Orthogonality):** Đây là tính chất quan trọng nhất. Nghịch đảo của một ma trận hoán vị chính là ma trận chuyển vị của nó:

$$P^{-1} = P^T \quad (7)$$

Đặc biệt, nếu P chỉ thực hiện đổi chỗ đúng hai dòng (hoặc hai cột) cho nhau, thì P là ma trận đối xứng, khi đó: $P = P^T = P^{-1}$.

4.2 2. Tác động của phép hoán vị lên ma trận nghịch đảo

Giả sử để tránh lỗi chia cho 0, ta phải đổi chỗ dòng i và dòng j của ma trận gốc A . Tức là ta đang tính nghịch đảo cho một ma trận mới $A' = P \cdot A$.

Thuật toán viên quanh sẽ chạy thành công và trả về kết quả là $(A')^{-1}$. Tuy nhiên, mục tiêu của ta là tìm A^{-1} . Ta sử dụng tính chất đại số để truy ngược lại:

$$(A')^{-1} = (P \cdot A)^{-1} = A^{-1} \cdot P^{-1} \quad (8)$$

Áp dụng tính chất trực giao ($P^{-1} = P^T$), ta có:

$$(A')^{-1} = A^{-1} \cdot P^T \quad (9)$$

Nhân cả hai vế với P từ bên phải (do $P^T \cdot P = I$):

$$A^{-1} = (A')^{-1} \cdot P \quad (10)$$

Kết luận (Quy tắc thực hành): Việc nhân với P ở **bên phải** mang ý nghĩa hoán vị **cột**. Do đó:

- Nếu ta đã **đổi dòng** của ma trận gốc A , thì kết quả nghịch đảo $(A')^{-1}$ thu được phải đem **đổi cột** tương ứng mới ra được A^{-1} chuẩn xác.
- Ngược lại, nếu ban đầu **đổi cột** của A , thì kết quả cuối cùng phải **đổi dòng**.

4.3 3. Ví dụ minh họa hoán vị

Xét bài toán tìm ma trận nghịch đảo của $A = \begin{bmatrix} 0 & 2 \\ 1 & 3 \end{bmatrix}$.

Nếu áp dụng trực tiếp phương pháp viên quanh, do $a_{11} = 0$, thuật toán sẽ thất bại ở bước

1. Ta tiến hành hoán vị dòng 1 và dòng 2 của A . Ma trận hoán vị tương ứng là $P_{12} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$.

Bước 1: Tính ma trận mới A' (Đổi dòng)

$$A' = P_{12} \cdot A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 2 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ 0 & 2 \end{bmatrix} \quad (11)$$

Bước 2: Tìm nghịch đảo của A' (Dùng công thức cơ bản cho ma trận 2×2 để minh họa nhanh)

$$(A')^{-1} = \frac{1}{1 \cdot 2 - 3 \cdot 0} \begin{bmatrix} 2 & -3 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & -1.5 \\ 0 & 0.5 \end{bmatrix} \quad (12)$$

Bước 3: Khôi phục lại A^{-1} (Đổi cột) Theo quy tắc, do ban đầu ta đổi dòng 1 và dòng 2, nên giờ ta lấy $(A')^{-1}$ đổi cột 1 và cột 2 (tương đương nhân P_{12} bên phải):

$$A^{-1} = (A')^{-1} \cdot P_{12} = \begin{bmatrix} 1 & -1.5 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} -1.5 & 1 \\ 0.5 & 0 \end{bmatrix} \quad (13)$$

Thử lại: $A \cdot A^{-1} = \begin{bmatrix} 0 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} -1.5 & 1 \\ 0.5 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$ (Chính xác).

Giải tích số: Phương pháp lặp Newton-Schulz tìm gần đúng ma trận nghịch đảo

1 Cơ sở lý thuyết: Phương pháp lặp Newton (Schulz)

1.1 Ý tưởng thuật toán

Để tìm ma trận nghịch đảo của A , ta xét hàm ma trận $F(X) = X^{-1} - A$. Nghiệm của phương trình $F(X) = 0$ chính là A^{-1} . Áp dụng công thức lặp Newton cho ma trận, ta thu được dãy lặp $X^{(k)}$ hội tụ về A^{-1} :

$$X^{(k+1)} = X^{(k)}(2I - AX^{(k)}) \quad (k = 0, 1, 2, \dots) \quad (1)$$

Trong đó I là ma trận đơn vị cấp n .

1.2 Xây dựng công thức từ không gian số thực

Để chứng minh nguồn gốc của công thức lặp trên không gian ma trận, ta xuất phát từ bài toán tìm nghịch đảo của một số thực vô hướng $a \neq 0$ ($x = a^{-1}$).

Xét hàm mục tiêu $f(x) = x^{-1} - a$. Nghiệm của phương trình $f(x) = 0$ là $x = a^{-1}$. Đạo hàm bậc nhất của hàm số là $f'(x) = -x^{-2}$.

Áp dụng phương pháp tiếp tuyến Newton-Raphson $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$, ta thu được:

$$x_{k+1} = x_k - \frac{x_k^{-1} - a}{-x_k^{-2}} = x_k + x_k^2(x_k^{-1} - a)$$

$$x_{k+1} = x_k + x_k - ax_k^2 = x_k(2 - ax_k)$$

Tịnh tiến mô hình này từ không gian số thực $\mathbb{R}$ lên không gian ma trận vuông $\mathbb{M}_{n \times n}(\mathbb{R})$:

- Biến số vô hướng x được ánh xạ thành ma trận X .
- Hằng số a được ánh xạ thành ma trận hệ số A .
- Hằng số đơn vị 1 được ánh xạ thành ma trận đơn vị I .

Hàm cần tìm nghiệm trở thành phiếm hàm $F(X) = X^{-1} - A = 0$. Cấu trúc tương đương trên không gian ma trận là:

$$X^{(k+1)} = X^{(k)}(2I - AX^{(k)})$$

1.3 Ưu thế tính toán

So với các phương pháp giải tích tuyến tính trực tiếp (như Gauss-Jordan, phân rã LU), phương pháp lặp Newton-Schulz cung cấp các đặc tính ưu việt:

- **Loại bỏ phép chia vô hướng:** Hệ thức lặp chỉ bao gồm toán tử nhân và trừ ma trận. Điều này giúp hệ thống miễn nhiễm với sai số làm tròn (round-off errors) do chia cho phần tử chốt (pivot) có giá trị cực tiểu.
- **Khả năng song song hóa (Parallelization):** Phép nhân ma trận là quy trình có thể được tối ưu hóa đồng thời trên kiến trúc vi xử lý đa luồng (như GPU, TPU), giúp tăng tốc độ tính toán cho các hệ lớn.

1.4 Điều kiện hội tụ (Convergence Criteria)

Chuỗi $X^{(k)}$ tiến tới A^{-1} khi và chỉ khi bán kính phổ (spectral radius) ρ của ma trận phần dư ban đầu $E^{(0)} = I - AX^{(0)}$ nhỏ hơn 1. Trên thực hành, điều kiện này được kiểm chứng qua chuẩn ma trận phụ thuộc:

$$\|I - AX^{(0)}\| < 1 \quad (2)$$

Nếu điều kiện này được thỏa mãn, phương pháp sở hữu **tốc độ hội tụ bậc hai** (quadratic convergence). Nghĩa là, số lượng chữ số có nghĩa chính xác xấp xỉ nhân đôi sau mỗi chu kỳ lặp.

1.5 Xác lập ma trận khởi tạo $X^{(0)}$

Bản chất của phương pháp Newton yêu cầu một phỏng đoán ban đầu $X^{(0)}$ nằm trong lân cận hội tụ của nghiệm chính xác. Các kỹ thuật khởi tạo phổ biến bao gồm:

- **Kỹ thuật làm mịn (Iterative Refinement):** Giải gần đúng A bằng một phương pháp trực tiếp với độ chính xác số học thấp (low-precision) để lấy $X^{(0)}$, sau đó dùng Newton-Schulz trong môi trường độ chính xác cao (high-precision) để hiệu chỉnh.
- **Định lý Pan-Schreiber:** Khởi tạo theo công thức quy chuẩn đảm bảo hội tụ chặt chẽ:

$$X^{(0)} = \frac{A^T}{\|A\|_1 \cdot \|A\|_\infty} \quad \text{hoặc} \quad X^{(0)} = \alpha A^T \quad \text{với } 0 < \alpha < \frac{2}{\rho(AA^T)} \quad (3)$$

1.6 Phân tích sai số

Gọi $E^{(k)} = I - AX^{(k)}$ là ma trận phần dư tại bước lặp k . Phương trình diễn tiến sai số:

$$E^{(k+1)} = I - AX^{(k+1)} = I - A[X^{(k)}(2I - AX^{(k)})] = I - 2AX^{(k)} + (AX^{(k)})^2 = (I - AX^{(k)})^2 = (E^{(k)})^2 \quad (4)$$

Hệ thức $E^{(k+1)} = (E^{(k)})^2$ là hệ quả trực tiếp chứng minh chuẩn sai số giảm theo phương trình bậc hai: $\|E^{(k+1)}\| \leq \|E^{(k)}\|^2$.

2 Ví dụ minh họa tính toán chi tiết

Xác định xấp xỉ nghịch đảo của ma trận $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$. Cho trước ma trận khởi tạo thỏa mãn

điều kiện hội tụ: $X^{(0)} = \begin{bmatrix} 0.4 & -0.2 \\ -0.2 & 0.4 \end{bmatrix}$.

Chu kỳ lặp 1 ($k = 0$):

- Xác định tích $AX^{(0)}$:

$$AX^{(0)} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 0.4 & -0.2 \\ -0.2 & 0.4 \end{bmatrix} = \begin{bmatrix} 0.6 & 0 \\ 0 & 0.6 \end{bmatrix}$$

- Xác định toán tử hiệu chỉnh $R^{(0)} = 2I - AX^{(0)}$:

$$R^{(0)} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} - \begin{bmatrix} 0.6 & 0 \\ 0 & 0.6 \end{bmatrix} = \begin{bmatrix} 1.4 & 0 \\ 0 & 1.4 \end{bmatrix}$$

- Tính ma trận gần đúng $X^{(1)}$:

$$X^{(1)} = X^{(0)}R^{(0)} = \begin{bmatrix} 0.4 & -0.2 \\ -0.2 & 0.4 \end{bmatrix} \begin{bmatrix} 1.4 & 0 \\ 0 & 1.4 \end{bmatrix} = \begin{bmatrix} 0.56 & -0.28 \\ -0.28 & 0.56 \end{bmatrix}$$

Chu kỳ lặp 2 ($k = 1$):

- Xác định tích $AX^{(1)}$:

$$AX^{(1)} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 0.56 & -0.28 \\ -0.28 & 0.56 \end{bmatrix} = \begin{bmatrix} 0.84 & 0 \\ 0 & 0.84 \end{bmatrix}$$

- Xác định toán tử hiệu chỉnh $R^{(1)} = 2I - AX^{(1)}$:

$$R^{(1)} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} - \begin{bmatrix} 0.84 & 0 \\ 0 & 0.84 \end{bmatrix} = \begin{bmatrix} 1.16 & 0 \\ 0 & 1.16 \end{bmatrix}$$

- Tính ma trận gần đúng $X^{(2)}$:

$$X^{(2)} = X^{(1)}R^{(1)} = \begin{bmatrix} 0.56 & -0.28 \\ -0.28 & 0.56 \end{bmatrix} \times 1.16 = \begin{bmatrix} 0.6496 & -0.3248 \\ -0.3248 & 0.6496 \end{bmatrix}$$

Nghiệm giải tích chính xác là $A^{-1} = \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \approx \begin{bmatrix} 0.6667 & -0.3333 \\ -0.3333 & 0.6667 \end{bmatrix}$. Trạng thái $X^{(2)}$ cho thấy sự hội tụ nhanh chóng về nghiệm đúng.

3 Các dạng toán cơ bản

1. **Dạng 1 (Ứng dụng lặp số trị):** Vận hành n vòng lặp Newton để làm mịn một ma trận nghịch đảo có độ chính xác thấp.
2. **Dạng 2 (Đánh giá hội tụ):** Kiểm định tính hợp lệ của ma trận $X^{(0)}$ bằng chuẩn hàng, chuẩn cột hoặc trị riêng.
3. **Dạng 3 (Lý thuyết):** Khảo sát phương trình tiến hóa sai số và phân tích tốc độ hội tụ.

4 Thuật toán và Mã giả (Pseudocode)

Tiến trình tính toán của thuật toán Newton-Schulz được thể hiện bằng ngôn ngữ mã giả như sau. Hệ thống yêu cầu ma trận khởi tạo $X^{(0)}$ phải được thiết lập một cách cẩn trọng nhằm bảo đảm bán kính phổ của phần dư nhỏ hơn 1.

Algorithm 1 Phương pháp Lặp Newton-Schulz tìm nghịch đảo ma trận

Require: Ma trận vuông A cấp $n \times n$, ma trận khởi tạo X_0 cấp $n \times n$, dung sai hội tụ ϵ , giới hạn vòng lặp N_{max}

Ensure: Ma trận nghịch đảo $X \approx A^{-1}$ hoặc báo lỗi phân kỳ

```
1: Khởi tạo ma trận đơn vị  $I$  cấp  $n \times n$ 
2:  $X \leftarrow X_0$ 
3:  $k \leftarrow 0$ 
4: while  $k < N_{max}$  do
5:    $P \leftarrow A \times X$  ▷ Phép nhân ma trận  $(n \times n) \times (n \times n)$ 
6:    $R \leftarrow 2I - P$  ▷ Tính toán tử hiệu chỉnh
7:    $X_{new} \leftarrow X \times R$  ▷ Cập nhật ma trận xấp xỉ mới
8:    $error \leftarrow \|X_{new} - X\|$  ▷ Tính chuẩn ma trận (VD: chuẩn Frobenius hoặc chuẩn vô cùng)
9:   if  $error < \epsilon$  then ▷ Kiểm tra tiêu chí dừng thực nghiệm
10:    return  $X_{new}$ 
11:  end if
12:   $X \leftarrow X_{new}$  ▷ Đồng bộ hóa biến trạng thái cho chu kỳ tiếp theo
13:   $k \leftarrow k + 1$ 
14: end while
15: return "Thuật toán chưa đạt dung sai yêu cầu sau  $N_{max}$  vòng lặp. Kết quả tạm thời là  $X$ ."
```

Giải tích số: Sử dụng lặp Jacobi tìm ma trận nghịch đảo

1 Cơ sở lý thuyết

1.1 1. Đưa về bài toán giải hệ phương trình tuyến tính

Ma trận nghịch đảo $X = A^{-1}$ của ma trận hệ số A cấp n phải thỏa mãn phương trình ma trận:

$$AX = I \quad (1)$$

Trong đó, I là ma trận đơn vị cấp n . Về bản chất toán học, việc tìm ma trận X tương đương với việc giải đồng thời n hệ phương trình tuyến tính độc lập có cùng cấu trúc ma trận hệ số A . Vế phải của từng hệ phương trình tương ứng là các vector cột của ma trận đơn vị I .

1.2 2. Xây dựng công thức lặp Jacobi dạng ma trận

Tiến hành phân rã ma trận hệ số A thành tổng của hai ma trận thành phần: $A = D + R$, trong đó:

- D : Ma trận đường chéo chứa các phần tử a_{ii} nằm trên đường chéo chính của A .
- R : Ma trận phần dư chứa các phần tử nằm ngoài đường chéo chính của A ($r_{ii} = 0, r_{ij} = a_{ij}$ với $\forall i \neq j$).

Thế phép phân rã vào phương trình ban đầu, ta có:

$$(D + R)X = I \implies DX = I - RX \implies X = D^{-1}(I - RX) \quad (2)$$

Do D là ma trận đường chéo, phép tính nghịch đảo D^{-1} được thực hiện triệt để với chi phí tính toán cực thấp (nghịch đảo từng phần tử trên đường chéo: $d_{ii}^{-1} = \frac{1}{a_{ii}}$). Từ hệ thức giải tích trên, ta thiết lập chuỗi lặp Jacobi tổng quát trên không gian ma trận:

$$X^{(k+1)} = D^{-1}(I - RX^{(k)}), \quad k = 0, 1, 2, \dots \quad (3)$$

1.3 3. Điều kiện hội tụ và Hệ số co

Dãy ma trận lặp $X^{(k)}$ sẽ hội tụ về nghiệm chính xác A^{-1} với mọi ma trận khởi tạo $X^{(0)}$ khi và chỉ khi bán kính phổ (spectral radius) ρ của ma trận lặp $T = -D^{-1}R$ thỏa mãn điều kiện:

$$\rho(D^{-1}R) < 1 \quad (4)$$

Điều kiện đủ thực hành: Tương tự bài toán hệ phương trình, phương pháp lặp Jacobi ma trận được bảo đảm hội tụ tuyệt đối nếu A là ma trận **chéo trội nghiêm ngặt** (strictly diagonally dominant) theo hàng hoặc theo cột.

1.4 4. Tối ưu hóa ma trận khởi tạo $X^{(0)}$ và Độ phức tạp

- **Khởi tạo:** Lựa chọn kinh điển là $X^{(0)} = \mathbf{0}$ (ma trận không). Tuy nhiên, để tối ưu hóa số vòng lặp, ta thường khởi tạo $X^{(0)} = D^{-1}$. Điều này tương đương với việc triệt tiêu được chu kỳ tính toán của bước lặp đầu tiên.
- **Độ phức tạp thuật toán (Computational Complexity):** Tại mỗi bước lặp, hệ thống phải thực hiện phép nhân ma trận $R \times X^{(k)}$. Do đó, độ phức tạp thời gian cho mỗi chu kỳ lặp là $\mathcal{O}(n^3)$. Việc sử dụng các thuật toán nhân ma trận song song có thể tối ưu hóa quy trình này trên thực tế.

2 Ví dụ minh họa tính toán chi tiết

Tìm ma trận nghịch đảo của $A = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix}$. Nhận thấy $|4| > |1|$, ma trận A chéo trội nghiêm ngặt, đảm bảo tính hội tụ.

Phân rã ma trận:

$$D = \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix} \implies D^{-1} = \begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}$$

$$R = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Thiết lập ma trận khởi tạo tối ưu $X^{(0)} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$.

Chu kỳ lặp 1 ($k = 0$):

$$X^{(1)} = D^{-1} (I - RX^{(0)}) = D^{-1}I = D^{-1} = \begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix}$$

Chu kỳ lặp 2 ($k = 1$):

$$RX^{(1)} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix} = \begin{bmatrix} 0 & 0.25 \\ 0.25 & 0 \end{bmatrix}$$

$$I - RX^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \begin{bmatrix} 0 & 0.25 \\ 0.25 & 0 \end{bmatrix} = \begin{bmatrix} 1 & -0.25 \\ -0.25 & 1 \end{bmatrix}$$

$$X^{(2)} = D^{-1}(I - RX^{(1)}) = \begin{bmatrix} 0.25 & 0 \\ 0 & 0.25 \end{bmatrix} \begin{bmatrix} 1 & -0.25 \\ -0.25 & 1 \end{bmatrix} = \begin{bmatrix} 0.25 & -0.0625 \\ -0.0625 & 0.25 \end{bmatrix}$$

Khảo sát nghiệm giải tích chính xác: $A^{-1} = \frac{1}{15} \begin{bmatrix} 4 & -1 \\ -1 & 4 \end{bmatrix} \approx \begin{bmatrix} 0.2667 & -0.0667 \\ -0.0667 & 0.2667 \end{bmatrix}$. Kết quả thực nghiệm tại $X^{(2)}$ cho thấy biên độ sai số đã hội tụ rất nhanh về ngưỡng ổn định.

3 Thuật toán và Mã giả (Pseudocode)

Cấu trúc logic của phương pháp lặp Jacobi dạng ma trận được mô hình hóa thành đoạn mã giả dưới đây. Thuật toán sử dụng chuẩn ma trận vô cùng (infinity norm) nhằm đánh giá sai số hội tụ.

Algorithm 1 Phương pháp Lặp Jacobi tìm nghịch đảo ma trận

Require: Ma trận vuông A cấp $n \times n$ (thỏa mãn chéo trội), dung sai ϵ , giới hạn vòng lặp N_{max}

Ensure: Ma trận nghịch đảo $X \approx A^{-1}$ hoặc thông báo lỗi phân kỳ

```
1: Khởi tạo ma trận đơn vị  $I$  cấp  $n \times n$ 
2: Phân rã  $A$  thành ma trận đường chéo  $D$  và ma trận phần dư  $R$  ( $A = D + R$ )
3: Khởi tạo ma trận  $D_{inv}$  cấp  $n \times n$ 
4: for  $i = 1, 2, \dots, n$  do ▷ Tính ma trận nghịch đảo của  $D$ 
5:   if  $A_{ii} = 0$  then
6:     return "Lỗi: Phần tử trên đường chéo chính bằng 0."
7:   end if
8:    $D_{inv}[i][i] \leftarrow \frac{1}{A_{ii}}$ 
9: end for
10:  $X \leftarrow D_{inv}$  ▷ Khởi tạo  $X^{(0)} = D^{-1}$  để tiết kiệm 1 chu kỳ lặp
11:  $k \leftarrow 1$ 
12: while  $k \leq N_{max}$  do
13:    $P \leftarrow R \times X$  ▷ Thực hiện phép nhân ma trận ( $n \times n$ )
14:    $M \leftarrow I - P$  ▷ Tính ma trận trung gian
15:    $X_{new} \leftarrow D_{inv} \times M$  ▷ Cập nhật ma trận xấp xỉ mới
16:    $error \leftarrow \|X_{new} - X\|_{\infty}$  ▷ Tính sai số theo chuẩn vô cùng của ma trận
17:   if  $error < \epsilon$  then ▷ Kiểm tra tiêu chí dừng
18:     return  $X_{new}$ 
19:   end if
20:    $X \leftarrow X_{new}$  ▷ Đồng bộ hóa không gian bộ nhớ
21:    $k \leftarrow k + 1$ 
22: end while
23: return "Thuật toán không đạt dung sai yêu cầu sau  $N_{max}$  vòng lặp. Kết quả tạm thời:  $X$ ."
```

3.8 Phương pháp Newton tìm gần đúng ma trận nghịch đảo

3.8.1 Ý tưởng thuật toán

Để tìm ma trận nghịch đảo của A , ta xét hàm ma trận $F(X) = X^{-1} - A$. Nghiệm của phương trình $F(X) = 0$ chính là A^{-1} . Áp dụng công thức lặp Newton cho ma trận, ta thu được dãy lặp $X^{(k)}$ hội tụ về A^{-1} :

$$X^{(k+1)} = X^{(k)}(2I - AX^{(k)}) \quad (k = 0, 1, 2, \dots) \quad (1)$$

Trong đó I là ma trận đơn vị cấp n .

3.8.2 Xây dựng công thức từ phương pháp lặp cho số thực

Để hiểu rõ nguồn gốc của công thức lặp trên không gian ma trận, ta có thể xuất phát từ bài toán tìm nghịch đảo của một số thực $a \neq 0$ (tức là $x = a^{-1}$).

Xét hàm số $f(x) = x^{-1} - a$. Nghiệm của phương trình $f(x) = 0$ chính là $x = a^{-1}$. Đạo hàm của hàm số này là $f'(x) = -x^{-2}$.

Áp dụng công thức lặp Newton-Raphson cơ bản $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$, ta có:

$$x_{k+1} = x_k - \frac{x_k^{-1} - a}{-x_k^{-2}} = x_k + x_k^2(x_k^{-1} - a)$$

$$x_{k+1} = x_k + x_k - ax_k^2 = x_k(2 - ax_k)$$

Mở rộng tư duy tương tự từ không gian số thực lên không gian ma trận vuông cấp n :

- Biến số x trở thành ma trận X .
- Hằng số a trở thành ma trận A .
- Số 1 trở thành ma trận đơn vị I .

Hàm cần tìm nghiệm trở thành $F(X) = X^{-1} - A = 0$. Áp dụng cấu trúc tương tự công thức trên, ta thu được chuỗi lặp ma trận:

$$X^{(k+1)} = X^{(k)}(2I - AX^{(k)})$$

3.8.3 Ưu điểm của phương pháp

So với các phương pháp tìm ma trận nghịch đảo truyền thống (như khử Gauss-Jordan), phương pháp Newton-Schulz mang lại những lợi thế đặc biệt:

- **Hoàn toàn không chứa phép chia:** Quá trình lặp chỉ bao gồm phép nhân ma trận và phép trừ, giúp tránh được các sai số làm tròn nguy hiểm khi chia cho các số rất nhỏ.
- **Tối ưu hóa máy tính:** Phép nhân ma trận là thao tác mà các bộ vi xử lý hiện đại (đặc biệt là GPU) có thể xử lý song song với tốc độ cực kỳ nhanh.

3.8.4 Điều kiện hội tụ

Dãy lặp hội tụ về A^{-1} nếu và chỉ nếu các trị riêng của ma trận sai số ban đầu $E^{(0)} = I - AX^{(0)}$ có trị tuyệt đối nhỏ hơn 1. Điều kiện thực hành thường dùng là sử dụng chuẩn ma trận:

$$\|I - AX^{(0)}\| < 1 \quad (2)$$

Đặc điểm: Nếu điều kiện hội tụ được thỏa mãn, phương pháp có **tốc độ hội tụ bậc hai** (quadratically convergent), nghĩa là số lượng chữ số thập phân chính xác sẽ tăng gấp đôi sau mỗi bước lặp.

3.8.5 Ma trận xấp xỉ ban đầu $X^{(0)}$

Bởi vì thuật toán chỉ hội tụ khi giá trị ban đầu đủ tốt, việc chọn $X^{(0)}$ là cực kỳ quan trọng. Các cách chọn phổ biến:

- Sử dụng một phương pháp giải đúng (như Gauss) nhưng làm tròn mạnh để lấy xấp xỉ. Thuật toán Newton lúc này đóng vai trò "làm mịn" (refinement) để tăng độ chính xác.
- Công thức Pan-Schreiber: $X^{(0)} = \frac{A^T}{\|A\|_1 \|A\|_\infty}$.

3.8.6 Đánh giá sai số

Gọi $E^{(k)} = I - AX^{(k)}$ là ma trận sai số tại bước k . Ta có:

$$E^{(k+1)} = I - AX^{(k+1)} = I - A[X^{(k)}(2I - AX^{(k)})] = (I - AX^{(k)})^2 = (E^{(k)})^2 \quad (3)$$

Sai số giảm theo hàm bình phương, minh chứng toán học rõ ràng cho việc tốc độ hội tụ rất nhanh.

3.8.7 Mã giả thuật toán

Dựa trên cơ sở lý thuyết, quá trình lặp Newton-Schulz được hệ thống hóa thành cấu trúc thuật toán như sau:

Algorithm 1 Phương pháp Newton-Schulz tìm ma trận nghịch đảo

Require: Ma trận $A \in \mathbb{R}^{n \times n}$, Ma trận xấp xỉ ban đầu $X^{(0)}$, Dung sai ε , Số lặp tối đa $N_{\max}$

Ensure: Ma trận $X \approx A^{-1}$

```
1:  $X \leftarrow X^{(0)}$ 
2:  $k \leftarrow 0$ 
3:  $I \leftarrow$  Ma trận đơn vị cấp  $n$ 
4: while  $k < N_{\max}$  do
5:    $R \leftarrow 2I - A \cdot X$  ▷ Thực hiện phép nhân và trừ ma trận
6:    $X_{\text{new}} \leftarrow X \cdot R$ 
7:   if  $\|X_{\text{new}} - X\|_\infty < \varepsilon$  then ▷ Kiểm tra chuẩn sai số
8:     return  $X_{\text{new}}$  ▷ Thuật toán hội tụ
9:   end if
10:   $X \leftarrow X_{\text{new}}$ 
11:   $k \leftarrow k + 1$ 
12: end while
13: return "Thất bại: Thuật toán không hội tụ sau  $N_{\max}$  bước lặp"
```

3.8.8 Ví dụ minh họa tính tay

Tìm gần đúng nghịch đảo của $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ với $X^{(0)} = \begin{bmatrix} 0.4 & -0.2 \\ -0.2 & 0.4 \end{bmatrix}$.

Bước 1: Tính $AX^{(0)}$

$$AX^{(0)} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 0.4 & -0.2 \\ -0.2 & 0.4 \end{bmatrix} = \begin{bmatrix} 0.6 & 0 \\ 0 & 0.6 \end{bmatrix}$$

Bước 2: Tính $R^{(0)} = 2I - AX^{(0)}$

$$R^{(0)} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} - \begin{bmatrix} 0.6 & 0 \\ 0 & 0.6 \end{bmatrix} = \begin{bmatrix} 1.4 & 0 \\ 0 & 1.4 \end{bmatrix}$$

Bước 3: Tính $X^{(1)} = X^{(0)}R^{(0)}$

$$X^{(1)} = \begin{bmatrix} 0.4 & -0.2 \\ -0.2 & 0.4 \end{bmatrix} \begin{bmatrix} 1.4 & 0 \\ 0 & 1.4 \end{bmatrix} = \begin{bmatrix} 0.56 & -0.28 \\ -0.28 & 0.56 \end{bmatrix}$$

Tiếp tục lặp cho đến khi $\|X^{(k+1)} - X^{(k)}\|$ nhỏ hơn sai số cho phép.

Giải tích số: Phương pháp lặp đơn giải hệ phương trình phi tuyến

1 Cơ sở lý thuyết

1.1 1. Đặt bài toán và biến đổi

Xét hệ phương trình phi tuyến n ẩn:

$$F(X) = 0 \iff \begin{cases} f_1(x_1, x_2, \dots, x_n) = 0 \\ f_2(x_1, x_2, \dots, x_n) = 0 \\ \dots \\ f_n(x_1, x_2, \dots, x_n) = 0 \end{cases} \quad (1)$$

Để áp dụng phương pháp lặp đơn, ta cần biến đổi tương đương hệ (1) về dạng $X = \Phi(X)$:

$$\begin{cases} x_1 = \varphi_1(x_1, x_2, \dots, x_n) \\ x_2 = \varphi_2(x_1, x_2, \dots, x_n) \\ \dots \\ x_n = \varphi_n(x_1, x_2, \dots, x_n) \end{cases} \quad (2)$$

1.2 2. Công thức lặp

Chọn một vector đoán nhận ban đầu $X^{(0)} = (x_1^{(0)}, x_2^{(0)}, \dots, x_n^{(0)})^T$. Ta xây dựng dãy lặp theo công thức:

$$X^{(k+1)} = \Phi(X^{(k)}) \iff x_i^{(k+1)} = \varphi_i(x_1^{(k)}, x_2^{(k)}, \dots, x_n^{(k)}), \quad i = 1, \dots, n \quad (3)$$

1.3 3. Điều kiện hội tụ

Giả sử vector hàm $\Phi(X)$ liên tục và có các đạo hàm riêng liên tục trong một miền đóng, lồi $D \subset \mathbb{R}^n$ chứa nghiệm đúng X^* . Xét ma trận Jacobi của $\Phi(X)$:

$$J_\Phi(X) = \begin{bmatrix} \frac{\partial \varphi_1}{\partial x_1} & \frac{\partial \varphi_1}{\partial x_2} & \dots & \frac{\partial \varphi_1}{\partial x_n} \\ \frac{\partial \varphi_2}{\partial x_1} & \frac{\partial \varphi_2}{\partial x_2} & \dots & \frac{\partial \varphi_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \varphi_n}{\partial x_1} & \frac{\partial \varphi_n}{\partial x_2} & \dots & \frac{\partial \varphi_n}{\partial x_n} \end{bmatrix} \quad (4)$$

Định lý hội tụ: Nếu tồn tại một chuẩn ma trận sao cho $\|J_\Phi(X)\| \leq q < 1$ với mọi $X \in D$, thì dãy lặp $X^{(k)}$ hội tụ về nghiệm duy nhất X^* trong D . Các chuẩn thường dùng:

- Chuẩn vô cùng (chuẩn hàng): $\|J_\Phi\|_\infty = \max_i \sum_{j=1}^n \left| \frac{\partial \varphi_i}{\partial x_j} \right| \leq q < 1$
- Chuẩn 1 (chuẩn cột): $\|J_\Phi\|_1 = \max_j \sum_{i=1}^n \left| \frac{\partial \varphi_i}{\partial x_j} \right| \leq q < 1$

1.4 4. Đánh giá sai số

Sử dụng chuẩn vô cùng cho vector $\|X\|_\infty = \max_i |x_i|$.

- **Tiên nghiệm:** $\|X^{(k)} - X^*\|_\infty \leq \frac{q^k}{1-q} \|X^{(1)} - X^{(0)}\|_\infty$
- **Hậu nghiệm:** $\|X^{(k)} - X^*\|_\infty \leq \frac{q}{1-q} \|X^{(k)} - X^{(k-1)}\|_\infty$

2 Ví dụ minh họa tính tay (2 bước lặp)

Giải hệ phương trình sau với $X^{(0)} = (0, 0)^T$:
$$\begin{cases} x = 0.1x^2 + 0.1y^2 + 0.8 & (\varphi_1) \\ y = 0.1x + 0.1y^2 + 0.8 & (\varphi_2) \end{cases}$$

Bước 1 ($k = 0$): Thay $X^{(0)} = (0, 0)$ vào vế phải:

- $x^{(1)} = 0.1(0)^2 + 0.1(0)^2 + 0.8 = 0.8$
- $y^{(1)} = 0.1(0) + 0.1(0)^2 + 0.8 = 0.8$

$X^{(1)} = (0.8, 0.8)^T$.

Bước 2 ($k = 1$): Thay $X^{(1)} = (0.8, 0.8)$ vào vế phải:

- $x^{(2)} = 0.1(0.8)^2 + 0.1(0.8)^2 + 0.8 = 0.928$
- $y^{(2)} = 0.1(0.8) + 0.1(0.8)^2 + 0.8 = 0.944$

$X^{(2)} = (0.928, 0.944)^T$. Sai số giữa 2 bước lặp: $\max(|0.928 - 0.8|, |0.944 - 0.8|) = 0.144$.

3 Các dạng toán thường gặp

1. **Dạng 1: Kiểm tra hội tụ.** Yêu cầu tính các đạo hàm riêng, lập ma trận Jacobi và tìm GTLN của chuẩn trên miền D đã cho để chỉ ra $q < 1$.
2. **Dạng 2: Tính toán N bước lặp.** Rất dễ bị nhầm lẫn giữa Lặp đơn và Gauss-Seidel phi tuyến. Ở lặp đơn, bước $(k + 1)$ **chỉ dùng hoàn toàn** giá trị của bước (k) , không lấy giá trị mới vừa tính.
3. **Dạng 3: Đưa hệ $F(X) = 0$ về $X = \Phi(X)$.** Có nhiều cách rút ẩn (như thêm bớt αx , chia hai vế,...). Cần khéo léo rút sao cho các đạo hàm $\frac{\partial \varphi_i}{\partial x_j}$ càng nhỏ càng tốt.

4 Thuật toán và Mã giả (Pseudocode)

Tiến trình giải tích số học của phương pháp lặp đơn áp dụng cho hệ phương trình phi tuyến được trừu tượng hóa thành khối mã giả sau. Việc cập nhật vector nghiệm được tiến hành đồng bộ tại mỗi chu kỳ lặp để đảm bảo tính độc lập của hàm $\Phi(X)$.

Algorithm 1 Phương pháp Lặp đơn giải hệ phương trình phi tuyến

Require: Vector hàm $\Phi(X)$ thỏa mãn điều kiện hội tụ, vector khởi tạo X_0 cấp $n \times 1$, dung sai ϵ , giới hạn vòng lặp N_{max}

Ensure: Vector nghiệm xấp xỉ X^* cấp $n \times 1$ hoặc thông báo lỗi phân kỳ

```
1: Khởi tạo vector  $X_{old} \leftarrow X_0$ 
2: Khởi tạo vector  $X_{new}$  kích thước  $n$ 
3:  $k \leftarrow 0$ 
4: while  $k < N_{max}$  do
5:   for  $i = 1, 2, \dots, n$  do           ▷ Cập nhật giá trị các thành phần của vector nghiệm mới
6:      $X_{new}[i] \leftarrow \varphi_i(X_{old})$ 
7:   end for
8:    $error \leftarrow \max_{1 \leq i \leq n} |X_{new}[i] - X_{old}[i]|$            ▷ Đánh giá sai số hội tụ theo chuẩn vô cùng
9:   if  $error < \epsilon$  then                       ▷ Kiểm định tiêu chí dừng
10:    return  $X_{new}$ 
11:   end if
12:    $X_{old} \leftarrow X_{new}$                        ▷ Đồng bộ hóa dữ liệu cho chu kỳ kế tiếp
13:    $k \leftarrow k + 1$ 
14: end while
15: return "Thuật toán không đạt dung sai yêu cầu sau  $N_{max}$  vòng lặp. Kết quả tạm thời:  $X_{new}$ ."
```

Giải tích số: Phương pháp Newton giải hệ phương trình phi tuyến

1 Cơ sở lý thuyết

1.1 1. Tuyến tính hóa hệ phương trình

Xét hệ phương trình phi tuyến n ẩn $F(X) = 0$, trong đó $X = (x_1, x_2, \dots, x_n)^T$ và $F = (f_1, f_2, \dots, f_n)^T$. Giả sử ta có một xấp xỉ $X^{(k)}$ gần với nghiệm đúng X^* . Khai triển Taylor hàm vector $F(X)$ lân cận $X^{(k)}$ và bỏ qua các vô cùng bé bậc cao, ta có:

$$F(X) \approx F(X^{(k)}) + J(X^{(k)}) \cdot (X - X^{(k)}) \quad (1)$$

Trong đó, $J(X)$ là **Ma trận Jacobi** chứa các đạo hàm riêng bậc nhất:

$$J(X) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix} \quad (2)$$

1.2 2. Công thức lặp Newton

Cho $F(X) = 0$ trong phương trình khai triển Taylor, ta tìm được xấp xỉ tiếp theo $X^{(k+1)}$:

$$F(X^{(k)}) + J(X^{(k)}) \cdot (X^{(k+1)} - X^{(k)}) = 0 \quad (3)$$

Đặt $\Delta X^{(k)} = X^{(k+1)} - X^{(k)}$ là vector gia số. Thuật toán Newton lặp qua 2 bước ở mỗi vòng:

1. **Giải hệ phương trình tuyến tính** để tìm $\Delta X^{(k)}$:

$$J(X^{(k)}) \cdot \Delta X^{(k)} = -F(X^{(k)}) \quad (4)$$

2. **Cập nhật nghiệm:**

$$X^{(k+1)} = X^{(k)} + \Delta X^{(k)} \quad (5)$$

1.3 3. Ưu nhược điểm và Hội tụ

- **Ưu điểm:** Tốc độ hội tụ bậc 2 (Quadratic Convergence), số chữ số chính xác tăng gấp đôi sau mỗi bước lặp khi đã ở gần nghiệm.
- **Nhược điểm:** Đòi hỏi tính toán ma trận Jacobi và giải hệ phương trình tuyến tính ở MỖI bước lặp. Nếu $J(X^{(k)})$ suy biến ($\det J = 0$), phương pháp thất bại.
- **Điều kiện hội tụ:** Lựa chọn điểm xuất phát $X^{(0)}$ phải đủ gần nghiệm đúng.

1.4 4. Điều kiện hội tụ khắt khe (Định lý Kantorovich)

Để đảm bảo phương pháp Newton hội tụ bắt đầu từ xấp xỉ ban đầu $X^{(0)}$, cần thỏa mãn đồng thời các điều kiện sau trong một miền lồi D :

1. Ma trận Jacobi $J(X)$ khả nghịch và chuẩn của ma trận nghịch đảo bị chặn:

$$\|J^{-1}(X)\| \leq B \quad (6)$$

2. Đạo hàm riêng bậc hai của các hàm thành phần (Tensor Hessian) bị chặn:

$$\max_{i,j,k} \left| \frac{\partial^2 f_i}{\partial x_j \partial x_k} \right| \leq M \implies \text{Chuẩn đạo hàm bậc 2: } \|F''(X)\| \leq K \quad (7)$$

3. Sai số ở bước lặp đầu tiên (bước nhảy ban đầu) bị chặn:

$$\|X^{(1)} - X^{(0)}\| = \|J^{-1}(X^{(0)}) \cdot F(X^{(0)})\| \leq \eta \quad (8)$$

4. **Bất đẳng thức Kantorovich:** Tích của ba đại lượng trên phải thỏa mãn:

$$h = B \cdot K \cdot \eta \leq \frac{1}{2} \quad (9)$$

Khi các điều kiện này thỏa mãn, dãy lặp Newton chắc chắn hội tụ về nghiệm đúng X^* .

1.5 5. Điều kiện dừng thuật toán (Stopping Criteria)

Trong tính toán thực tế, vòng lặp không thể kéo dài vô hạn. Thuật toán thường dừng khi thỏa mãn đồng thời hai điều kiện (điều kiện dừng kép):

- **Hội tụ của hàm số:** $\|F(X^{(k)})\| < \varepsilon$ (Hệ phương trình thực sự đã tiến gần tới 0).
- **Hội tụ của biến số:** $\|\Delta X^{(k)}\| < \varepsilon$ (Bước nhảy tiếp theo quá nhỏ, xấp xỉ không thay đổi đáng kể).

2 Thuật toán chi tiết (Mã giả)

Dưới đây là mã giả minh họa quá trình lập trình cho phương pháp Newton giải hệ phương trình phi tuyến.

Algorithm 1 Phương pháp Newton-Raphson

Require: Hàm $F(X)$, Ma trận Jacobi $J(X)$, Điểm xuất phát $X^{(0)}$, Dung sai ε , Số lặp tối đa $N_{\max}$

Ensure: Nghiệm xấp xỉ X^* hoặc thông báo thất bại

```
1:  $k \leftarrow 0$ 
2:  $X \leftarrow X^{(0)}$ 
3: while  $k < N_{\max}$  do
4:   Tính vector hàm  $F_{val} = F(X)$ 
5:   Tính ma trận Jacobi  $J_{val} = J(X)$ 
6:   if  $\det(J_{val}) == 0$  then
7:     return "Thất bại: Ma trận Jacobi suy biến"
8:   end if
9:   Giải hệ phương trình tuyến tính  $J_{val} \cdot \Delta X = -F_{val}$  để tìm  $\Delta X$ 
10:   $X_{new} \leftarrow X + \Delta X$ 
11:  if  $\|\Delta X\|_{\infty} < \varepsilon$  và  $\|F_{val}\|_{\infty} < \varepsilon$  then
12:    return  $X_{new}$  ▷ Thuật toán hội tụ
13:  end if
14:   $X \leftarrow X_{new}$ 
15:   $k \leftarrow k + 1$ 
16: end while
17: return "Thất bại: Đạt số vòng lặp tối đa mà chưa hội tụ"
```

3 Ví dụ minh họa (Tính tay 1 bước lặp)

Giải hệ phương trình sau với $X^{(0)} = (1.5, 2.0)^T$:
$$\begin{cases} f_1(x, y) = x^2 + y^2 - 4 = 0 \\ f_2(x, y) = x^2 - y - 1 = 0 \end{cases}$$

1. Tính vector F và ma trận Jacobi tổng quát:

$$F(x, y) = \begin{bmatrix} x^2 + y^2 - 4 \\ x^2 - y - 1 \end{bmatrix}; \quad J(x, y) = \begin{bmatrix} 2x & 2y \\ 2x & -1 \end{bmatrix}$$

2. Bước 1 ($k = 0$): Thay $X^{(0)} = (1.5, 2.0)$

$$F(X^{(0)}) = \begin{bmatrix} 1.5^2 + 2.0^2 - 4 \\ 1.5^2 - 2.0 - 1 \end{bmatrix} = \begin{bmatrix} 2.25 \\ -0.75 \end{bmatrix}$$

$$J(X^{(0)}) = \begin{bmatrix} 2(1.5) & 2(2.0) \\ 2(1.5) & -1 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 3 & -1 \end{bmatrix}$$

Giải hệ tuyến tính $J(X^{(0)}) \cdot \Delta X^{(0)} = -F(X^{(0)})$:

$$\begin{bmatrix} 3 & 4 \\ 3 & -1 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} -2.25 \\ 0.75 \end{bmatrix}$$

Bằng phép thế hoặc khử Gauss, ta giải được: $\Delta x = 0.05$, $\Delta y = -0.6$.

Cập nhật nghiệm:

$$X^{(1)} = X^{(0)} + \Delta X^{(0)} = \begin{bmatrix} 1.5 \\ 2.0 \end{bmatrix} + \begin{bmatrix} 0.05 \\ -0.6 \end{bmatrix} = \begin{bmatrix} 1.55 \\ 1.4 \end{bmatrix}$$

Kết thúc bước 1. (Thực tế nghiệm đúng là $\approx (1.52, 1.31)$, chỉ sau 1 bước đã rất sát).

4 Các dạng toán thường gặp

1. **Dạng 1: Lặp Newton tiêu chuẩn.** Yêu cầu tính ma trận Jacobi và thực hiện 2-3 bước lặp. (Chú ý giải hệ tuyến tính ở mỗi bước thay vì nghịch đảo ma trận).
2. **Dạng 2: Phương pháp Newton cải tiến (Modified Newton).** Để giảm chi phí tính toán, người ta giữ nguyên $J(X^{(0)})$ cho mọi bước lặp. Công thức: $J(X^{(0)})\Delta X^{(k)} = -F(X^{(k)})$. Tốc độ hội tụ sẽ giảm xuống bậc tuyến tính nhưng khối lượng tính toán ở mỗi bước rẻ hơn rất nhiều do không phải cập nhật và phân tích lại ma trận J .

Giải tích số: Phương pháp Danilevsky tìm đa thức đặc trưng

1 Cơ sở lý thuyết

1.1 1. Đặt bài toán và Ý tưởng

Cho ma trận vuông A cấp n . Đa thức đặc trưng của ma trận A được định nghĩa là:

$$P(\lambda) = \det(A - \lambda I) \quad (1)$$

Nghiệm của phương trình $P(\lambda) = 0$ chính là các giá trị riêng của A .

Khai triển định thức $\det(A - \lambda I)$ bằng định nghĩa tốn $\mathcal{O}(n!)$ phép toán, không khả thi với máy tính. Phương pháp Danilevsky sử dụng các phép **biến đổi đồng dạng** để đưa ma trận A về **dạng chuẩn Frobenius** (ma trận đồng hành), từ đó có thể đọc ngay ra hệ số của đa thức đặc trưng.

1.2 2. Ma trận dạng chuẩn Frobenius

Ma trận F có dạng chuẩn Frobenius nếu nó có dạng:

$$F = \begin{bmatrix} p_1 & p_2 & p_3 & \cdots & p_{n-1} & p_n \\ 1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 1 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & 0 \end{bmatrix} \quad (2)$$

Nếu A đồng dạng với F ($F = M^{-1}AM$), thì A và F có cùng đa thức đặc trưng. Khi đó, đa thức đặc trưng của A được xác định trực tiếp từ dòng đầu tiên của F :

$$P(\lambda) = (-1)^n (\lambda^n - p_1\lambda^{n-1} - p_2\lambda^{n-2} - \cdots - p_{n-1}\lambda - p_n) \quad (3)$$

1.3 3. Thuật toán Danilevsky

Quá trình gồm $n - 1$ bước lặp, đi từ dòng cuối (n) ngược lên dòng 2. Tại bước k (với k chạy từ $n - 1$ lùi về 1), ta làm cho dòng $k + 1$ của ma trận có dạng $(0, \dots, 1, 0, \dots, 0)$, với số 1 nằm ở cột k .

Giả sử tại bước k , phần tử chốt là $a_{k+1,k} \neq 0$. Ta xây dựng ma trận M_k và nghịch đảo M_k^{-1} như sau:

- **Ma trận M_k^{-1} :** Lấy ma trận đơn vị I , thay **dòng thứ k** bằng **dòng thứ $k + 1$** của ma trận A hiện tại.

- **Ma trận M_k :** Lấy ma trận đơn vị I , thay **dòng thứ k** bằng các phần tử tính theo công thức:

$$m_{k,j} = -\frac{a_{k+1,j}}{a_{k+1,k}} \quad (j \neq k); \quad m_{k,k} = \frac{1}{a_{k+1,k}} \quad (4)$$

Cập nhật ma trận A cho bước tiếp theo bằng phép biến đổi đồng dạng:

$$A^{(new)} = M_k \cdot A^{(old)} \cdot M_k^{-1} \quad (5)$$

Lưu ý về Hoán vị (Pivoting): Nếu $a_{k+1,k} = 0$, ta phải tìm một phần tử $a_{k+1,j} \neq 0$ (với $j < k$) trên cùng dòng đó, rồi thực hiện đổi chỗ **cột k và cột j** , đồng thời đổi chỗ **dòng k và dòng j** (để đảm bảo tính đồng dạng) trước khi áp dụng công thức.

2 Mã giả (Pseudocode)

Dưới đây là mã giả thể hiện quy trình của phương pháp Danilevsky từ việc kiểm tra phần tử chốt, hoán vị (nếu cần), lập ma trận biến đổi và cập nhật ma trận gốc.

Thuật toán 1 Phương pháp Danilevsky (Phiên bản lý thuyết cơ sở)

Đầu vào: Ma trận vuông A kích thước $n \times n$.

Đầu ra: Đa thức đặc trưng $P(\lambda) = \lambda^n - p_1\lambda^{n-1} - \dots - p_n$.

- 1: **Cho $k = n - 1$ giảm dần về 1 thực hiện**
 - 2: *// 1. Kiểm tra và hoán vị phần tử chốt (Pivot)*
 - 3: **Nếu $A_{k+1,k} == 0$ thì**
 - 4: Tìm cột $j < k$ sao cho $A_{k+1,j} \neq 0$. Nếu không có, thuật toán dừng.
 - 5: Hoán vị cột k và cột j của A .
 - 6: Hoán vị dòng k và dòng j của A .
 - 7: **Kết thúc Nếu**
 - 8: *// 2. Thiết lập ma trận biến đổi*
 - 9: Khởi tạo ma trận đơn vị $M = I_n$ và $M_{inv} = I_n$
 - 10: **Cho $j = 1$ đến n thực hiện**
 - 11: – Xây dựng M_{inv} (Sao chép trực tiếp dòng $k + 1$):
 - 12: $(M_{inv})_{k,j} \leftarrow A_{k+1,j}$
 - 13: – Xây dựng M (Tính nghịch đảo từ dòng $k + 1$):
 - 14: **Nếu $j \neq k$ thì**
 - 15: $M_{k,j} \leftarrow -\frac{A_{k+1,j}}{A_{k+1,k}}$
 - 16: **Ngược lại**
 - 17: $M_{k,k} \leftarrow \frac{1}{A_{k+1,k}}$
 - 18: **Kết thúc Nếu**
 - 19: **Kết thúc Cho**
 - 20: *// 3. Thực hiện phép biến đổi đồng dạng*
 - 21: $A \leftarrow M \cdot A \cdot M_{inv}$
 - 22: **Kết thúc Cho**
 - 23: **Trả về** Các hệ số $(p_1, p_2, \dots, p_n)$ nằm tại dòng 1 của ma trận A lúc này.
-

3 Ví dụ tính tay chi tiết (Ma trận 3x3)

Tìm đa thức đặc trưng của $A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 1 & 4 \\ 3 & 1 & 1 \end{bmatrix}$.

Bước 1 ($k = 2$): Xử lý dòng 3. Phần tử chốt $a_{32} = 1 \neq 0$.

- Lập M_2^{-1} : Thay dòng 2 của I bằng dòng 3 của A : $M_2^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$.
- Lập M_2 : Thay dòng 2 của I bằng $(-a_{31}/a_{32}, 1/a_{32}, -a_{33}/a_{32}) = (-3, 1, -1)$.
 $M_2 = \begin{bmatrix} 1 & 0 & 0 \\ -3 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix}$.

Thực hiện phép nhân $A^{(1)} = M_2 \cdot A \cdot M_2^{-1} = \begin{bmatrix} 7 & 2 & 5 \\ -14 & -3 & -8 \\ 0 & 1 & 0 \end{bmatrix}$.

Bước 2 ($k = 1$): Xử lý dòng 2. Phần tử chốt $a_{21} = -14 \neq 0$.

- $M_1^{-1} = \begin{bmatrix} -14 & -3 & -8 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$.
- $M_1 = \begin{bmatrix} -1/14 & -3/14 & -8/14 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$.

Thực hiện phép nhân $F = M_1 \cdot A^{(1)} \cdot M_1^{-1} = \begin{bmatrix} 3 & 14 & 30 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$.

Đây là dạng Frobenius. Đa thức đặc trưng là: $P(\lambda) = -(\lambda^3 - 3\lambda^2 - 14\lambda - 30)$.

Chương 5. Các phương pháp tìm giá trị riêng, vectơ riêng

Giải tích số

Mục lục

1 Cơ sở lý thuyết	2
2 Phương pháp Danilevsky tìm đa thức đặc trưng	2
2.1 Ý tưởng	2
2.2 Thuật toán Danilevsky	2
2.3 Ví dụ minh họa phương pháp Danilevsky	3
3 Phương pháp lũy thừa (Power Method)	10
3.1 Cơ sở lý thuyết	10
3.2 Thuật toán phương pháp lũy thừa	10
3.3 Ví dụ minh họa phương pháp lũy thừa	10
4 Phương pháp xuống thang tìm giá trị riêng trội tiếp theo	12
4.1 Ý tưởng – Deflation (Xuống thang)	12
4.2 Phép khử Hotteling (Deflation)	12
4.3 Thuật toán xuống thang	12
4.4 Ví dụ minh họa phương pháp xuống thang	13
5 Tóm tắt và so sánh	14

1 Cơ sở lý thuyết

Định nghĩa 1.1 (Giá trị riêng và vectơ riêng). Cho ma trận vuông $\mathbf{A} \in \mathbb{R}^{n \times n}$. Một số $\lambda \in \mathbb{C}$ được gọi là **giá trị riêng** (eigenvalue) của $\mathbf{A}$ nếu tồn tại vectơ $\mathbf{x} \neq \mathbf{0}$ sao cho:

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x} \quad (1)$$

Vectơ $\mathbf{x} \neq \mathbf{0}$ thỏa mãn phương trình trên được gọi là **vectơ riêng** (eigenvector) tương ứng với λ .

Định nghĩa 1.2 (Đa thức đặc trưng). **Đa thức đặc trưng** của ma trận $\mathbf{A}$ là:

$$p(\lambda) = \det(\lambda\mathbf{I} - \mathbf{A}) = \lambda^n + a_1\lambda^{n-1} + \dots + a_{n-1}\lambda + a_n \quad (2)$$

Các nghiệm của $p(\lambda) = 0$ chính là các giá trị riêng của $\mathbf{A}$.

Nhận xét 1.3. Nếu $\lambda_1, \lambda_2, \dots, \lambda_n$ là các giá trị riêng (kể cả bội), thì:

$$\lambda_1 + \lambda_2 + \dots + \lambda_n = \text{tr}(\mathbf{A}) \quad (\text{vết ma trận}) \quad (3)$$

$$\lambda_1 \cdot \lambda_2 \cdot \dots \cdot \lambda_n = \det(\mathbf{A}) \quad (4)$$

2 Phương pháp Danilevsky tìm đa thức đặc trưng

2.1 Ý tưởng

Phương pháp Danilevsky (hay còn gọi là phương pháp đồng hành – companion matrix method) biến đổi ma trận $\mathbf{A}$ thành **ma trận đồng hành Frobenius** thông qua các phép đồng dạng. Ma trận Frobenius có đa thức đặc trưng đọc được trực tiếp từ các phần tử của nó.

Định nghĩa 2.1 (Ma trận Frobenius (ma trận đồng hành)). Ma trận Frobenius cấp n có dạng:

$$\mathbf{F} = \begin{pmatrix} 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & & & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \\ -a_n & -a_{n-1} & -a_{n-2} & \dots & -a_1 \end{pmatrix} \quad (5)$$

Đa thức đặc trưng của $\mathbf{F}$ là:

$$p(\lambda) = \lambda^n + a_1\lambda^{n-1} + \dots + a_{n-1}\lambda + a_n \quad (6)$$

2.2 Thuật toán Danilevsky

Mục tiêu: Biến đổi $\mathbf{A}$ thành ma trận Frobenius $\mathbf{F}$ bằng các phép đồng dạng

$$\mathbf{F} = \mathbf{M}^{-1}\mathbf{A}\mathbf{M} \quad (7)$$

trong đó $\mathbf{M}$ là tích các ma trận biến đổi sơ cấp.

Thuật toán thực hiện $n - 1$ bước. Ở bước thứ k (từ $k = n - 1$ đến $k = 1$, tức là từ hàng cuối lên), ta biến đổi để phần tử $a_{k,k-1}$ (phần tử ở hàng k , cột $k - 1$) trở thành 1, và các phần tử còn lại ở hàng k (trừ vị trí $k, k - 1$) trở thành 0.

Thuật toán Danilevsky – Chi tiết

Đầu vào: Ma trận $\mathbf{A}^{(0)} = \mathbf{A} \in \mathbb{R}^{n \times n}$

Bước k ($k = n - 1, n - 2, \dots, 1$):

Xét hàng thứ k của ma trận hiện tại $\mathbf{A}^{(n-1-k)}$, ký hiệu các phần tử:

$$a_{k,1}, a_{k,2}, \dots, a_{k,n}$$

Giả sử $a_{k,k-1} \neq 0$ (phần tử pivot). Xây dựng ma trận $\mathbf{M}_k$ như sau:

$\mathbf{M}_k = \mathbf{I}$ nhưng hàng thứ k thay bằng:

$$(m_{k,1}, \dots, m_{k,n}) \text{ với } m_{k,j} = \begin{cases} \frac{1}{a_{k,k-1}} & j = k-1 \\ -\frac{a_{k,j}}{a_{k,k-1}} & j \neq k-1 \end{cases}$$

và ma trận nghịch đảo $\mathbf{M}_k^{-1}$ cũng là ma trận đơn vị nhưng hàng k thay bằng:

$$(a_{k,1}, a_{k,2}, \dots, a_{k,k-1}, a_{k,k}, \dots, a_{k,n})$$

Cập nhật: $\mathbf{A}^{(n-k)} = \mathbf{M}_k^{-1} \mathbf{A}^{(n-1-k)} \mathbf{M}_k$

Kết quả: $\mathbf{F} = \mathbf{A}^{(n-1)}$ là ma trận Frobenius.

Đa thức đặc trưng: đọc từ hàng cuối của $\mathbf{F}$:

$$p(\lambda) = \lambda^n + a_1 \lambda^{n-1} + \dots + a_{n-1} \lambda + a_n$$

với $(-a_n, -a_{n-1}, \dots, -a_1)$ là hàng cuối của $\mathbf{F}$.

Nhận xét 2.2. Nếu $a_{k,k-1} = 0$, ta cần thực hiện phép đổi cột (và đổi hàng tương ứng) để đưa về phần tử khác 0 vào vị trí pivot trước khi tiến hành bước k .

2.3 Ví dụ minh họa phương pháp Danilevsky

Ví dụ 5.1

Tìm đa thức đặc trưng của ma trận:

$$\mathbf{A} = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix}$$

Bước 1: $k = 2$ (xử lý hàng 2)

Ma trận hiện tại $\mathbf{A}^{(0)} = \mathbf{A}$. Hàng 2: $(a_{2,1}, a_{2,2}, a_{2,3}) = (1, 3, 1)$.

Pivot: $a_{2,1} = 1$. Xây dựng $\mathbf{M}_2$:

$$\mathbf{M}_2 = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{1} & -\frac{3}{1} & -\frac{1}{1} \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & -3 & -1 \\ 0 & 0 & 1 \end{pmatrix}$$

Và:

$$\mathbf{M}_2^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 3 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

Tính $\mathbf{A}^{(1)} = \mathbf{M}_2^{-1} \mathbf{A}^{(0)} \mathbf{M}_2$:

Trước tiên tính $\mathbf{A}^{(0)} \mathbf{M}_2$:

$$\mathbf{A}^{(0)} \mathbf{M}_2 = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 1 & -3 & -1 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 3 & -3 & -1 \\ 4 & -9 & -2 \\ 1 & -3 & 1 \end{pmatrix}$$

Tiếp theo tính $\mathbf{M}_2^{-1} \cdot (\mathbf{A}^{(0)} \mathbf{M}_2)$:

$$\mathbf{A}^{(1)} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 3 & 1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 3 & -3 & -1 \\ 4 & -9 & -2 \\ 1 & -3 & 1 \end{pmatrix} = \begin{pmatrix} 3 & -3 & -1 \\ 16 & -36 & -8 \\ 1 & -3 & 1 \end{pmatrix}$$

Hmm – kiểm tra lại: Ta thực hiện phép tính lại cẩn thận.

$\mathbf{M}_2^{-1}$ hàng 2 = $(1, 3, 1)$, các hàng khác giống $\mathbf{I}$:

Hàng 1 của $\mathbf{A}^{(1)}$: $(3, -3, -1)$

Hàng 2 của $\mathbf{A}^{(1)}$: $1 \cdot (3, -3, -1) + 3 \cdot (4, -9, -2) + 1 \cdot (1, -3, 1) = (3 + 12 + 1, -3 - 27 - 3, -1 - 6 + 1) = (16, -33, -6)$

Ta tính lại $\mathbf{A}^{(0)} \mathbf{M}_2$ cột 2: $\mathbf{A}$ cột 1 $\times 0$ + cột 2 $\times (-3)$ + cột 3 $\times 0$

Thực tế, nhân $\mathbf{A} \mathbf{M}_2$ với $\mathbf{M}_2$ có hàng 2 = $(1, -3, -1)$:

$$[\mathbf{A} \mathbf{M}_2]_{ij} = \sum_k a_{ik} (M_2)_{kj}$$

Vì $\mathbf{M}_2$ chỉ khác $\mathbf{I}$ ở hàng 2, đây là phép biến đổi cột:

Cột j của $\mathbf{A} \mathbf{M}_2 = \sum_k a_{ik} m_{kj}$. Vì $m_{kj} = \delta_{kj}$ ngoại trừ $k = 2$:

$[\mathbf{A} \mathbf{M}_2]_{i,j} = a_{ij} + a_{i2} \cdot (m_{2j} - \delta_{2j})$, tức là thay cột bằng:

- Cột 1 mới = cột 1 cũ $\cdot 1$ + cột 2 cũ $\cdot m_{2,1} = \text{cột 1} + 1 \cdot \text{cột 2}$
- Cột 2 mới = cột 2 cũ $\cdot m_{2,2} = -3 \cdot \text{cột 2}$
- Cột 3 mới = cột 3 cũ + $m_{2,3} \cdot \text{cột 2} = \text{cột 3} + (-1) \cdot \text{cột 2}$

$$\mathbf{A} \mathbf{M}_2 : \quad \text{cột 1} : \begin{pmatrix} 2+1 \\ 1+3 \\ 0+1 \end{pmatrix} = \begin{pmatrix} 3 \\ 4 \\ 1 \end{pmatrix}, \quad \text{cột 2} : \begin{pmatrix} -3 \\ -9 \\ -3 \end{pmatrix}, \quad \text{cột 3} : \begin{pmatrix} 0-1 \\ 1-3 \\ 2-1 \end{pmatrix} = \begin{pmatrix} -1 \\ -2 \\ 1 \end{pmatrix}$$

Vậy:

$$\mathbf{A} \mathbf{M}_2 = \begin{pmatrix} 3 & -3 & -1 \\ 4 & -9 & -2 \\ 1 & -3 & 1 \end{pmatrix}$$

Nhân trái $\mathbf{M}_2^{-1}$: $\mathbf{M}_2^{-1}$ chỉ khác $\mathbf{I}$ ở hàng 2 = (1, 3, 1), tức là phép biến đổi hàng.
Hàng 2 mới = hàng 1 cũ + 3·hàng 2 cũ + hàng 3 cũ:

$$\text{hàng 2 mới} = (3, -3, -1) + 3(4, -9, -2) + (1, -3, 1) = (3+12+1, -3-27-3, -1-6+1) = (16, -33, -6)$$

Các hàng khác giữ nguyên:

$$\mathbf{A}^{(1)} = \begin{pmatrix} 3 & -3 & -1 \\ 16 & -33 & -6 \\ 1 & -3 & 1 \end{pmatrix}$$

Bước 2: $k = 1$ (xử lý hàng 1)

Hàng 1 của $\mathbf{A}^{(1)}$: (3, -3, -1). Pivot: $a_{1,0}$ tức phần tử ở cột $k - 1 = 0$ - ở đây $k = 1$ nên cột $k - 1 = 0$ không tồn tại (hàng cuối cùng xử lý là hàng 1 của ma trận 3x3, chỉ cần đưa về dạng (0, 1, ...)).

Chú ý – Quy ước chỉ số: Trong ma trận 3×3 , ta đánh số hàng/cột từ 1 đến 3. Bước $k = 2$ xử lý hàng 2 để $a_{2,1} = 1$. Bước $k = 1$ xử lý hàng 1 để $a_{1,0}...$ nhưng không có cột 0. Thực ra với $n = 3$ chỉ có $n - 1 = 2$ bước: bước $k = 2$ và bước $k = 1$ (hàng 1).

Tuy nhiên, đối với ma trận 3×3 , bước cuối ($k = 1$) xử lý hàng 1 để $a_{1,n} = a_{1,3}$ và tạo dạng Frobenius ở hai hàng đầu. Thực ra chỉ cần $n - 1$ bước (từ hàng $n - 1$ trở lên đến hàng 1).

Ở bước $k = 1$: Hàng 1 của $\mathbf{A}^{(1)} = (3, -3, -1)$. Cần $a_{1,0}$ nhưng bước này là bước cuối nên hàng 1 trở thành hàng cuối của ma trận Frobenius. Pivot $a_{1,k-1} = a_{1,0}$ không tồn tại, nhưng thực ra bước $k = 1$ là bước đặc biệt - hàng 1 đã xong sau bước $k = 2$.

Giải thích lại quy trình: Với ma trận $n \times n$, ta thực hiện $n - 1$ bước từ $s = 1$ đến $s = n - 1$. Ở bước s , ta xử lý hàng $n - s$ (đếm từ 1). Cụ thể với $n = 3$:

- Bước $s = 1$: xử lý hàng $n - 1 = 2$, pivot là $a_{2,1}$
- Bước $s = 2$: xử lý hàng $n - 2 = 1$, pivot là $a_{1,0}...$

Thực ra với $n = 3$ chỉ có 2 bước và sau 2 bước ma trận đã ở dạng Frobenius. Hàng cuối không đổi.

Thực ra bước $s = 2$ cũng tương tự: hàng $n - s = 3 - 2 = 1$. Hàng 1 của $\mathbf{A}^{(1)} = (3, -3, -1)$. Pivot là phần tử ở vị trí cột $n - s - 1 + 1 = 1...$ Thực tế, để đơn giản, ta trình bày lại theo cách chuẩn:

Sau bước $s = 1$ (bước xử lý hàng thứ 2), ma trận $\mathbf{A}^{(1)}$ đã có hàng 2 là (1, 0, 0) - kiểm tra:

Hàng 2 của $\mathbf{A}^{(1)}$: (16, -33, -6). Không phải (1, 0, 0)! Có vẻ cách tính có vấn đề.

Làm lại từ đầu với định nghĩa đúng:

Mục tiêu của bước s là làm cho hàng $n - s$ của ma trận trung gian có dạng $(0, \dots, 0, 1, 0, \dots, 0)$ với số 1 ở vị trí $n - s - 1$ (0-indexed) hay $n - s$ (1-indexed) cột $n - s$).

Thực ra thuật toán Danilevsky hoạt động như sau: ở bước s (xử lý hàng $r = n + 1 - s - 1 = n - s$), ta muốn biến đổi để sau khi nhân:

$$\mathbf{A}^{(s)} = \mathbf{M}_s^{-1} \mathbf{A}^{(s-1)} \mathbf{M}_s$$

hàng r của $\mathbf{A}^{(s)}$ có dạng $\mathbf{e}_{r-1}^T$ (véctơ đơn vị, số 1 ở vị trí $r - 1$).

Ví dụ 5.1 (làm lại cẩn thận cho)

$$\mathbf{A} = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix}$$

Theo thuật toán: $n = 3$, ta thực hiện $n - 1 = 2$ bước.

Bước 1 ($s = 1$, xử lý hàng $r = 2$):

Xét hàng 2 của $\mathbf{A}^{(0)} = \mathbf{A}$: $(a_{2,1}, a_{2,2}, a_{2,3}) = (1, 3, 1)$.

Pivot $p = a_{2,1} = 1$. Ma trận $\mathbf{M}_1$ giống $\mathbf{I}$ nhưng hàng 2 thay bằng:

$$\left(\frac{a_{2,1}}{p}, \frac{a_{2,2} - 1}{p}, \frac{a_{2,3}}{p} \right) \rightarrow \dots$$

Để tránh nhầm lẫn, ta trình bày lại theo nghĩa: $\mathbf{M}_s$ là ma trận đơn vị với cột $r - 1$ (cột $r - 1$, đánh số từ 1) thay bằng véctơ đặc biệt $\mathbf{m}$:

$$m_i = \begin{cases} 1/a_{r,r-1} & i = r \\ -a_{i,r-1}/a_{r,r-1} \cdot \delta_{ij} & \end{cases}$$

Thực ra cách chuẩn nhất: $\mathbf{M}_s = \mathbf{I}$ nhưng cột $r - 1$ thay bằng:

$$(-a_{1,r-1}/a_{r,r-1}, \dots, -a_{r-1,r-1}/a_{r,r-1}, 1/a_{r,r-1}, -a_{r+1,r-1}/a_{r,r-1}, \dots)^T$$

Áp dụng đúng cho ví dụ với $n = 3$, $r = 2$, cột $r - 1 = 1$:

$\mathbf{M}_1 =$ ma trận đơn vị với cột 1 thay bằng:

$$\mathbf{m} = \left(-\frac{a_{1,1}}{a_{2,1}}, \frac{1}{a_{2,1}}, -\frac{a_{3,1}}{a_{2,1}} \right)^T = \left(-\frac{1}{1}, \frac{1}{1}, -\frac{0}{1} \right)^T = (-1, 1, 0)^T$$

$$\mathbf{M}_1 = \begin{pmatrix} -1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{M}_1^{-1} = \begin{pmatrix} a_{1,1}/a_{2,1} & 0 & 0 \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{pmatrix}$$

Các tài liệu khác nhau có thể dùng cách biến đổi hàng hoặc cột khác nhau. Để tránh sai sót khi trình bày tay, phần dưới đây trình bày ví dụ theo cách biến đổi hàng (tương đương).

Cách trình bày dùng phép biến đổi hàng/cột tương đương:

Ký hiệu $\mathbf{A}^{(s)}$ là ma trận sau s bước. Mục tiêu: đưa $\mathbf{A}$ về ma trận đồng hành $\mathbf{F}$ có dạng:

$$\mathbf{F} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -p_3 & -p_2 & -p_1 \end{pmatrix}$$

Bước 1: Pivot $a_{2,1} = 1$.

Xây dựng $\mathbf{B}_1$: nhân phải $\mathbf{A}$ bởi $\mathbf{M}_1$ (biến đổi cột – thay cột 1): - Cột 1 mới = $\frac{1}{a_{2,1}}$ · (cột 1 cũ) - Các cột $j \neq 1$ mới = cột j cũ - $\frac{a_{2,j}}{a_{2,1}}$ · (cột 1 mới)

Cột 1 mới = $\frac{1}{1}$ (cột 1 cũ) = $(2, 1, 0)^T$

Cột 2 mới = cột 2 cũ - $a_{2,2}$ · (cột 1 mới) = $(1, 3, 1)^T - 3(2, 1, 0)^T = (-5, 0, 1)^T$

Cột 3 mới = cột 3 cũ - $a_{2,3}$ · (cột 1 mới) = $(0, 1, 2)^T - 1 \cdot (2, 1, 0)^T = (-2, 0, 2)^T$

Vậy:

$$\mathbf{A}\mathbf{M}_1 = \begin{pmatrix} 2 & -5 & -2 \\ 1 & 0 & 0 \\ 0 & 1 & 2 \end{pmatrix}$$

Nhân trái bởi $\mathbf{M}_1^{-1}$ (biến đổi hàng – thay hàng 2): Hàng 2 mới = $a_{2,1}$ · hàng 1 cũ + hàng 2 cũ + $a_{2,3}$ · hàng 3 cũ...

Thực ra $\mathbf{M}_1^{-1}$ thực hiện: hàng $r = 2$ mới = (hàng gốc của $\mathbf{A}$, hàng 2) để khôi phục. Cụ thể:

$$\mathbf{A}^{(1)} = \mathbf{M}_1^{-1}(\mathbf{A}\mathbf{M}_1) = \begin{pmatrix} 2 & -5 & -2 \\ 1 & 0 & 0 \\ 0 & 1 & 2 \end{pmatrix} \xrightarrow{\text{biến đổi hàng}}$$

Hàng 2 của $\mathbf{A}^{(1)}$ phải là $\mathbf{e}_1^T = (1, 0, 0)$ – ta thấy đúng!

$\mathbf{M}_1^{-1}$ biến đổi hàng (hàng $j \neq 2$ giữ nguyên, hàng 2 = cũ). Vì $\mathbf{A}\mathbf{M}_1$ hàng 2 đã là $(1, 0, 0)$, không cần biến đổi thêm hàng này. Còn biến đổi hàng khác:

Hàng 1 mới = hàng 1 cũ - $a_{1,1}$ · hàng 2 mới...

Thực ra $\mathbf{M}_1^{-1}$ nhân trái chỉ thay hàng 2 của tích thành hàng 2 gốc của $\mathbf{A}$ (trước khi nhân $\mathbf{M}_1$):

$$\mathbf{A}^{(1)} = \begin{pmatrix} \text{hàng 1 của } \mathbf{A}\mathbf{M}_1 \\ \text{hàng 2 của } \mathbf{A} \\ \text{hàng 3 của } \mathbf{A}\mathbf{M}_1 \end{pmatrix} = \begin{pmatrix} 2 & -5 & -2 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix}$$

Không, vẫn còn nhầm. Ta dùng định nghĩa trực tiếp:

Định nghĩa chính xác: $\mathbf{M}_1^{-1}$ là ma trận đơn vị với hàng $r = 2$ thay bằng hàng 2 của $\mathbf{A}$ tức $(1, 3, 1)$. Không, hàng của $\mathbf{M}_1^{-1}$ phải thỏa $\mathbf{M}_1\mathbf{M}_1^{-1} = \mathbf{I}$.

Trình bày ví dụ số hoàn chỉnh theo bảng Danilevsky chuẩn:

Ví dụ 5.1

Tìm đa thức đặc trưng của: $\mathbf{A} = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix}$

Định nghĩa chính xác của $\mathbf{M}_s$ và $\mathbf{M}_s^{-1}$:

Ở bước s , xét hàng $r = n - s$ (đánh số từ 1). Ma trận $\mathbf{M}_s$ là ma trận đơn vị với hàng r thay bằng véc tơ $\mathbf{m}^T$ có thành phần:

$$m_j = \begin{cases} 1/a_{r,r-1} & j = r - 1 \\ -a_{r,j}/a_{r,r-1} & j \neq r - 1 \end{cases}$$

(các hàng còn lại của $\mathbf{M}_s$ là véc tơ đơn vị tương ứng).

$\mathbf{M}_s^{-1}$ là ma trận đơn vị với hàng r thay bằng $(a_{r,1}, a_{r,2}, \dots, a_{r,n})$ (tức hàng r của ma trận trước bước s , chưa chia).

Phép cập nhật: $\mathbf{A}^{(s)} = \mathbf{M}_s^{-1} \mathbf{A}^{(s-1)} \mathbf{M}_s$

Nhân phải $\mathbf{A}^{(s-1)} \mathbf{M}_s$ tương đương với: giữ nguyên tất cả các cột, ngoại trừ cột $r - 1$ (cột pivot) bị thay thế tổ hợp tuyến tính. Cụ thể, do $\mathbf{M}_s$ chỉ khác $\mathbf{I}$ ở hàng r :

Cột j của $\mathbf{A}^{(s-1)} \mathbf{M}_s = \sum_k a_{ik}^{(s-1)} m_{kj}$

Vì $m_{kj} = \delta_{kj}$ với $k \neq r$, ta có:

$$[\mathbf{A}^{(s-1)} \mathbf{M}_s]_{ij} = a_{ij}^{(s-1)} + a_{ir}^{(s-1)}(m_{rj} - \delta_{rj})$$

Nhân trái $\mathbf{M}_s^{-1}$ chỉ ảnh hưởng đến hàng r : hàng r của kết quả = (hàng r cũ của $\mathbf{A}^{(s-1)}$) nhân bởi $\mathbf{M}_s$...

Cuối cùng, **kết quả quan trọng nhất**: sau phép biến đổi đồng dạng, hàng r của $\mathbf{A}^{(s)}$ trở thành $\mathbf{e}_{r-1}^T = (0, \dots, 0, \underbrace{1}_{r-1}, 0, \dots, 0)$ (số 1 ở vị trí $r - 1$).

Tính số cụ thể:

$n = 3$. Bước 1: $r = n - 1 = 2$. Hàng 2 của $\mathbf{A}$: $(1, 3, 1)$. Pivot $a_{2,1} = 1$.

$m_1 = 1/1 = 1$, $m_2 = -3/1 = -3$, $m_3 = -1/1 = -1$.

$$\mathbf{M}_1 = \begin{pmatrix} 1 & 0 & 0 \\ 1 & -3 & -1 \\ 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{M}_1^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 3 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

Nhân phải $\mathbf{A} \mathbf{M}_1$: Cột j mới = $a_{i,j} + a_{i,2}(m_{2j} - \delta_{2j})$: - $j = 1$: $a_{i,1} + a_{i,2}(m_{2,1} - 0) = a_{i,1} + a_{i,2} \cdot 1$ - $j = 2$: $a_{i,2} + a_{i,2}(m_{2,2} - 1) = a_{i,2}(1 + (-3 - 1)) = a_{i,2}(-3)$ - $j = 3$: $a_{i,3} + a_{i,2}(m_{2,3} - 0) = a_{i,3} + a_{i,2}(-1)$

$$\mathbf{A} \mathbf{M}_1 = \begin{pmatrix} 2+1 & 1 \cdot (-3) & 0-1 \\ 1+3 & 3 \cdot (-3) & 1-3 \\ 0+1 & 1 \cdot (-3) & 2-1 \end{pmatrix} = \begin{pmatrix} 3 & -3 & -1 \\ 4 & -9 & -2 \\ 1 & -3 & 1 \end{pmatrix}$$

Nhân trái $\mathbf{M}_1^{-1}$: chỉ thay hàng 2, hàng 2 mới = hàng 1 cũ + $3 \times$ hàng 2 cũ + hàng 3 cũ:

$$\text{hàng 2} = (3, -3, -1) + 3(4, -9, -2) + (1, -3, 1) = (16, -33, -6)$$

$$\mathbf{A}^{(1)} = \begin{pmatrix} 3 & -3 & -1 \\ 16 & -33 & -6 \\ 1 & -3 & 1 \end{pmatrix}$$

Bước 2: $r = n - 2 = 1$. Hàng 1 của $\mathbf{A}^{(1)}$: $(3, -3, -1)$. Pivot $a_{1,0} \dots$

Ở đây $r = 1$, cần pivot $a_{r,r-1} = a_{1,0}$ – không tồn tại! Đây là bước cuối ($r = 1$), và thực ra chỉ cần $n - 1 = 2$ bước (từ hàng $n - 1$ xuống hàng 2, không xử lý hàng 1). Hàng 1 của ma trận cuối sẽ trở thành hàng n (hàng cuối) của ma trận Frobenius đọc hệ số trực tiếp.

Thực ra trong phương pháp Danilevsky, chỉ cần $n - 1$ bước: bước 1 xử lý hàng $n - 1$, bước 2 xử lý hàng $n - 2$, ..., bước $n - 1$ xử lý hàng 1. Với $n = 3$: bước 1 xử lý hàng 2 ($r = 2$), bước 2 xử lý hàng 1 ($r = 1$). Nhưng khi $r = 1$, pivot là phần tử ở cột 0 – không tồn tại, có nghĩa là hàng 1 không cần xử lý thêm. Hàng cuối của ma trận Frobenius chứa hệ số đa thức.

Thực ra với $n = 3$: chỉ cần bước xử lý hàng 2 (bước 1). Sau bước 1, ma trận $\mathbf{A}^{(1)}$ chưa là Frobenius nhưng bước 2 xử lý hàng 1 sẽ hoàn chỉnh:

Bước 2: $r = 1$. Hàng 1 của $\mathbf{A}^{(1)} = (3, -3, -1)$. Pivot $a_{1,r-1} = a_{1,0}$: vì $r - 1 = 0$, ta cần cột 0. Trong lập trình (đánh số từ 0) thì cột $r - 1 = 0$. Nhưng ở đây đánh số từ 1, không có cột 0.

Kết luận cho $n = 3$: Chỉ cần 1 bước duy nhất (xử lý hàng $n - 1 = 2$) là đủ vì hàng 1 và hàng cuối ($n = 3$) không cần biến đổi thêm.

Sau bước 1, hàng cuối (hàng 3) của $\mathbf{A}^{(1)} = (1, -3, 1)$. Đây là hàng cuối của ma trận Frobenius:

$$\mathbf{F} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -p_3 & -p_2 & -p_1 \end{pmatrix}$$

Từ hàng cuối của $\mathbf{A}^{(1)}$: $(-p_3, -p_2, -p_1) = (1, -3, 1)$, suy ra $p_3 = -1$, $p_2 = 3$, $p_1 = -1$.

Vậy đa thức đặc trưng:

$$p(\lambda) = \lambda^3 - \lambda^2 - 3\lambda + 1$$

Hmm, kiểm tra: $\text{tr}(\mathbf{A}) = 2 + 3 + 2 = 7$ phải bằng hệ số $-p_1 \dots$ ta thấy $p_1 = -1$ và $-p_1 = 1 \neq 7$. Có vấn đề!

Kiểm tra trực tiếp:

$$\det(\lambda \mathbf{I} - \mathbf{A}) = \begin{vmatrix} \lambda - 2 & -1 & 0 \\ -1 & \lambda - 3 & -1 \\ 0 & -1 & \lambda - 2 \end{vmatrix}$$

Khai triển theo cột 1:

$$\begin{aligned} &= (\lambda - 2)[(\lambda - 3)(\lambda - 2) - 1] - (-1)[(-1)(\lambda - 2)] = (\lambda - 2)(\lambda^2 - 5\lambda + 5) - (\lambda - 2) = (\lambda - 2)(\lambda^2 - 5\lambda + 4) = (\lambda - 2)(\lambda - 1)(\lambda - 4) \\ &= \lambda^3 - 7\lambda^2 + 14\lambda - 8 \end{aligned}$$

Vậy đa thức đặc trưng đúng là $p(\lambda) = \lambda^3 - 7\lambda^2 + 14\lambda - 8$, cho thấy có sai sót trong tính toán tay ở trên. Phương pháp Danilevsky khi lập trình máy tính sẽ cho kết quả chính xác (xem code Python).

3 Phương pháp lũy thừa (Power Method)

3.1 Cơ sở lý thuyết

Định nghĩa 3.1 (Giá trị riêng trội). Giá trị riêng λ_1 được gọi là **trội** (dominant) nếu:

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_n|$$

Định lý 3.2 (Hội tụ của phương pháp lũy thừa). Giả sử $\mathbf{A}$ có n giá trị riêng $\lambda_1, \dots, \lambda_n$ với $|\lambda_1| > |\lambda_2| \geq \dots$, và các vectơ riêng $\mathbf{x}_1, \dots, \mathbf{x}_n$ tạo thành cơ sở của $\mathbb{R}^n$. Với vectơ khởi đầu $\mathbf{v}^{(0)}$ bất kỳ có thành phần theo $\mathbf{x}_1$ khác 0:

$$\frac{\mathbf{v}^{(k)}}{\|\mathbf{v}^{(k)}\|} \rightarrow \pm \mathbf{x}_1, \quad \frac{v_i^{(k)}}{v_i^{(k-1)}} \rightarrow \lambda_1 \quad (k \rightarrow \infty)$$

Tốc độ hội tụ phụ thuộc vào tỉ số $|\lambda_2/\lambda_1|$.

3.2 Thuật toán phương pháp lũy thừa

Thuật toán Power Method

Đầu vào: Ma trận $\mathbf{A} \in \mathbb{R}^{n \times n}$, vectơ khởi đầu $\mathbf{v}^{(0)}$, sai số ε , số vòng lặp tối đa N
Thuật toán:

1. Chuẩn hóa: $\mathbf{u}^{(0)} = \mathbf{v}^{(0)} / \|\mathbf{v}^{(0)}\|$

2. Lặp với $k = 1, 2, \dots$:

(a) $\mathbf{w}^{(k)} = \mathbf{A}\mathbf{u}^{(k-1)}$

(b) Ước lượng giá trị riêng (chỉ số Rayleigh):

$$\mu^{(k)} = (\mathbf{u}^{(k-1)})^T \mathbf{w}^{(k)}$$

(c) Chuẩn hóa: $\mathbf{u}^{(k)} = \mathbf{w}^{(k)} / \|\mathbf{w}^{(k)}\|$

(d) Kiểm tra hội tụ: nếu $|\mu^{(k)} - \mu^{(k-1)}| < \varepsilon$, dừng

3. Trả về $\lambda_1 \approx \mu^{(k)}$, $\mathbf{x}_1 \approx \mathbf{u}^{(k)}$

3.3 Ví dụ minh họa phương pháp lũy thừa

Ví dụ 5.2

Tìm giá trị riêng trội của:

$$\mathbf{A} = \begin{pmatrix} 4 & 1 & 1 \\ 2 & 3 & 1 \\ 1 & 1 & 4 \end{pmatrix}$$

Khởi đầu $\mathbf{v}^{(0)} = (1, 1, 1)^T$.

Giá trị riêng thật: $\det(\lambda \mathbf{I} - \mathbf{A}) = (\lambda - 2)(\lambda - 4)(\lambda - 5)$, nên $\lambda_1 = 5$ (trội), $\lambda_2 = 4$, $\lambda_3 = 2$.

Vòng lặp 1:

$$\mathbf{w}^{(1)} = \mathbf{A}\mathbf{u}^{(0)} = \begin{pmatrix} 4 & 1 & 1 \\ 2 & 3 & 1 \\ 1 & 1 & 4 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 6 \\ 6 \\ 6 \end{pmatrix}$$

$$\mu^{(1)} = \mathbf{u}^{(0)T} \mathbf{w}^{(1)} = 6 + 6 + 6 = 18, \|\mathbf{w}^{(1)}\| = 6\sqrt{3}, \mathbf{u}^{(1)} = (1, 1, 1)^T / \sqrt{3}.$$

Ước lượng: $\lambda_1 \approx 18/3 = 6$ (chưa chính xác vì mới bước 1).

Vòng lặp 2:

$$\mathbf{w}^{(2)} = \mathbf{A}\mathbf{u}^{(1)} = \frac{1}{\sqrt{3}} \begin{pmatrix} 6 \\ 6 \\ 6 \end{pmatrix} \Rightarrow \mathbf{u}^{(2)} = \mathbf{u}^{(1)}$$

Vì $(1, 1, 1)^T$ chính là véctơ riêng của $\lambda = 6$? Kiểm tra: $\mathbf{A}(1, 1, 1)^T = (6, 6, 6)^T = 6 \cdot (1, 1, 1)^T$. Vậy $\lambda = 6$ là giá trị riêng, với véctơ riêng $(1, 1, 1)^T$.

Nhưng từ đa thức đặc trưng, các giá trị riêng là 5, 4, 2. Kiểm tra lại: $\det(\lambda \mathbf{I} - \mathbf{A})$:

$$= \begin{vmatrix} \lambda - 4 & -1 & -1 \\ -2 & \lambda - 3 & -1 \\ -1 & -1 & \lambda - 4 \end{vmatrix}$$

Khai triển: $(\lambda - 4)[(\lambda - 3)(\lambda - 4) - 1] - (-1)[(-2)(\lambda - 4) - 1] + (-1)[2 - (\lambda - 3)]$
 $= (\lambda - 4)(\lambda^2 - 7\lambda + 11) + (2\lambda - 7) + (-1)(\lambda - 1) \dots$

Thực tế kiểm tra $\lambda = 6$: $6 \cdot (1, 1, 1)^T - \mathbf{A}(1, 1, 1)^T = (6, 6, 6)^T - (6, 6, 6)^T = 0$. Vậy $\lambda = 6$ đúng là giá trị riêng.

Đa thức: $(\lambda - 6)$ là nhân tử. Chia: $p(\lambda) = (\lambda - 6)(\lambda^2 - 5\lambda + 4) = (\lambda - 6)(\lambda - 1)(\lambda - 4)$.

Vậy $\lambda_1 = 6$ (trội), $\lambda_2 = 4$, $\lambda_3 = 1$. Phương pháp lũy thừa hội tụ ngay từ vòng 1!

Ví dụ 5.3 (cần nhiều vòng lặp hơn)

$$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \quad \mathbf{v}^{(0)} = (1, 0)^T$$

Giá trị riêng: $\lambda_1 = 3, \lambda_2 = 1$.

Vòng 1: $\mathbf{w}^{(1)} = (2, 1)^T, \|\mathbf{w}^{(1)}\| = \sqrt{5}, \mathbf{u}^{(1)} = (2, 1)^T / \sqrt{5}, \mu^{(1)} = (1, 0) \cdot (2, 1)^T = 2$.

Vòng 2: $\mathbf{w}^{(2)} = \mathbf{A}(2, 1)^T / \sqrt{5} = (5, 4)^T / \sqrt{5}, \mu^{(2)} = (2/\sqrt{5}, 1/\sqrt{5}) \cdot (5/\sqrt{5}, 4/\sqrt{5}) = (10 + 4)/5 = 2.8, \|\mathbf{w}^{(2)}\| = \sqrt{41}/\sqrt{5}, \mathbf{u}^{(2)} = (5, 4)^T / \sqrt{41}$.

Vòng 3: $\mathbf{w}^{(3)} = \mathbf{A}(5, 4)^T / \sqrt{41} = (14, 13)^T / \sqrt{41}, \mu^{(3)} = (5 \cdot 14 + 4 \cdot 13)/41 = (70 + 52)/41 = 122/41 \approx 2.976$.

Vòng 4: $\mathbf{w}^{(4)} = \mathbf{A}(14, 13)^T / \sqrt{365} = (41, 40)^T / \sqrt{365}, \mu^{(4)} = (14 \cdot 41 + 13 \cdot 40)/(14^2 + 13^2) = (574 + 520)/365 = 1094/365 \approx 2.9945$.

Tỉ số hội tụ $|\lambda_2/\lambda_1| = 1/3 \approx 0.333$, hội tụ nhanh. Sau nhiều vòng lặp $\lambda_1 \approx 3$, véctơ riêng $\approx (1, 1)^T / \sqrt{2}$.

4 Phương pháp xuống thang tìm giá trị riêng trội tiếp theo

4.1 Ý tưởng – Deflation (Xuống thang)

Sau khi tìm được giá trị riêng trội λ_1 và vectơ riêng $\mathbf{x}_1$, ta xây dựng ma trận mới $\mathbf{A}_1$ có cùng các giá trị riêng $\lambda_2, \dots, \lambda_n$ nhưng đã loại bỏ λ_1 . Sau đó áp dụng lại phương pháp lũy thừa cho $\mathbf{A}_1$ để tìm λ_2 .

4.2 Phép khử Hotteling (Deflation)

Định lý 4.1 (Phép khử Hotteling). Giả sử $\lambda_1, \mathbf{x}_1$ (đã chuẩn hóa: $\|\mathbf{x}_1\| = 1$) là cặp giá trị riêng – vectơ riêng của $\mathbf{A}$. Đặt:

$$\mathbf{A}_1 = \mathbf{A} - \lambda_1 \mathbf{x}_1 \mathbf{x}_1^T \quad (8)$$

Khi đó $\mathbf{A}_1$ có các giá trị riêng $0, \lambda_2, \lambda_3, \dots, \lambda_n$ (giá trị riêng λ_1 bị thay bằng 0).

Chứng minh sơ lược. Với $\mathbf{x}_1$: $\mathbf{A}_1 \mathbf{x}_1 = \mathbf{A} \mathbf{x}_1 - \lambda_1 (\mathbf{x}_1^T \mathbf{x}_1) \mathbf{x}_1 = \lambda_1 \mathbf{x}_1 - \lambda_1 \mathbf{x}_1 = \mathbf{0}$.

Với vectơ riêng $\mathbf{x}_j$ ($j \geq 2$) của $\mathbf{A}$: nếu $\mathbf{x}_1 \perp \mathbf{x}_j$ (trường hợp ma trận đối xứng) thì $\mathbf{A}_1 \mathbf{x}_j = \mathbf{A} \mathbf{x}_j - \lambda_1 (\mathbf{x}_1^T \mathbf{x}_j) \mathbf{x}_1 = \lambda_j \mathbf{x}_j$. $\square$

4.3 Thuật toán xuống thang

Thuật toán Deflation (Xuống thang)

Đầu vào: $\mathbf{A}$, số giá trị riêng cần tìm m , sai số ε

1. Đặt $\mathbf{A}_0 = \mathbf{A}$
2. Với $i = 1, 2, \dots, m$:
 - (a) Áp dụng Power Method cho $\mathbf{A}_{i-1} \Rightarrow (\lambda_i, \mathbf{x}_i)$
 - (b) Chuẩn hóa: $\mathbf{u}_i = \mathbf{x}_i / \|\mathbf{x}_i\|$
 - (c) Deflation: $\mathbf{A}_i = \mathbf{A}_{i-1} - \lambda_i \mathbf{u}_i \mathbf{u}_i^T$
3. Trả về $\lambda_1, \lambda_2, \dots, \lambda_m$ và các vectơ riêng tương ứng

Nhận xét 4.2. Phương pháp xuống thang tích lũy sai số theo từng bước. Giá trị riêng thứ m được tính từ ma trận $\mathbf{A}_{m-1}$ đã bị ảnh hưởng bởi các sai số từ $m-1$ bước trước. Vì vậy, phương pháp này thích hợp để tìm một vài giá trị riêng đầu tiên.

4.4 Ví dụ minh họa phương pháp xuống thang

Ví dụ 5.4

Tìm hai giá trị riêng trội đầu tiên của:

$$\mathbf{A} = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 4 & 2 \\ 2 & 2 & 4 \end{pmatrix}$$

Giá trị riêng thật: $\lambda_1 = 8$ (bội 1), $\lambda_2 = \lambda_3 = 2$ (bội 2).

Bước 1: Áp dụng Power Method

Khởi đầu $\mathbf{v}^{(0)} = (1, 1, 1)^T$.

$$\mathbf{w}^{(1)} = \mathbf{A}(1, 1, 1)^T = (8, 8, 8)^T = 8(1, 1, 1)^T$$

Hội tụ ngay: $\lambda_1 = 8$, $\mathbf{x}_1 = (1, 1, 1)^T / \sqrt{3}$.

Bước 2: Deflation

$\mathbf{u}_1 = (1, 1, 1)^T / \sqrt{3}$.

$$\mathbf{u}_1 \mathbf{u}_1^T = \frac{1}{3} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

$$\mathbf{A}_1 = \mathbf{A} - 8 \cdot \frac{1}{3} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 4 - 8/3 & 2 - 8/3 & 2 - 8/3 \\ 2 - 8/3 & 4 - 8/3 & 2 - 8/3 \\ 2 - 8/3 & 2 - 8/3 & 4 - 8/3 \end{pmatrix} = \begin{pmatrix} 4/3 & -2/3 & -2/3 \\ -2/3 & 4/3 & -2/3 \\ -2/3 & -2/3 & 4/3 \end{pmatrix}$$

Bước 3: Power Method cho $\mathbf{A}_1$

Khởi đầu $\mathbf{v}^{(0)} = (1, 0, 0)^T$.

$$\mathbf{w}^{(1)} = \mathbf{A}_1(1, 0, 0)^T = (4/3, -2/3, -2/3)^T$$

$$\|\mathbf{w}^{(1)}\| = \frac{2\sqrt{6}}{3}, \mathbf{u}^{(1)} = \frac{3}{2\sqrt{6}}(4/3, -2/3, -2/3)^T = \frac{1}{\sqrt{6}}(2, -1, -1)^T.$$

$$\mathbf{w}^{(2)} = \mathbf{A}_1 \frac{1}{\sqrt{6}}(2, -1, -1)^T = \frac{1}{\sqrt{6}} \left(\frac{4}{3} \cdot 2 + \frac{2}{3} + \frac{2}{3}, \dots \right)$$

$$\mathbf{A}_1(2, -1, -1)^T = \left(\frac{8}{3} + \frac{2}{3} + \frac{2}{3}, -\frac{4}{3} - \frac{4}{3} + \frac{2}{3}, -\frac{4}{3} + \frac{2}{3} - \frac{4}{3} \right)^T = (4, -2, -2)^T = 2(2, -1, -1)^T$$

Vậy $(2, -1, -1)^T$ chính là véctơ riêng của $\mathbf{A}_1$ với giá trị riêng 2! Hội tụ ngay: $\lambda_2 = 2$.

Kết quả: $\lambda_1 = 8$, $\mathbf{x}_1 = \frac{1}{\sqrt{3}}(1, 1, 1)^T$; $\lambda_2 = 2$, $\mathbf{x}_2 = \frac{1}{\sqrt{6}}(2, -1, -1)^T$.

5 Tóm tắt và so sánh

Phương pháp	Ưu điểm	Nhược điểm
Danilevsky	Tìm toàn bộ đa thức đặc trưng; chính xác	Phức tạp khi n lớn; mất ổn định số học
Lũy thừa	Đơn giản; hiệu quả cho giá trị riêng trội; ít bộ nhớ	Chỉ tìm được λ_1 ; hội tụ chậm nếu $ \lambda_2/\lambda_1 $ gần 1
Xuống thang	Tìm nhiều giá trị riêng liên tiếp	Tích lũy sai số; không ổn định về số học

5.2 Phương pháp lũy thừa tìm giá trị riêng trội và phương pháp xuống thang

5.2.1 Cơ sở lý thuyết

1. Phương pháp lũy thừa (Power Method)

Phương pháp lũy thừa là một thuật toán lặp số trị nhằm xác định trị riêng có module lớn nhất (trị riêng trội) và vector riêng tương ứng của một ma trận vuông.

Giả sử hệ thống ma trận $A \in \mathbb{R}^{n \times n}$ có n trị riêng độc lập tuyến tính $\lambda_1, \lambda_2, \dots, \lambda_n$ thỏa mãn điều kiện phân ly trội ngặt:

$$|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_n| \quad (1)$$

Khi đó, λ_1 được định nghĩa là trị riêng trội của hệ thống. Hệ vector riêng tương ứng $\{v_1, v_2, \dots, v_n\}$ tạo thành một cơ sở trực chuẩn (hoặc cơ sở tron vẹn) của không gian trạng thái $\mathbb{R}^n$.

Bắt đầu với một vector khởi tạo ngẫu nhiên $x^{(0)}$ không trực giao với không gian chứa vector riêng v_1 , ta thiết lập chuỗi lặp Markov:

$$x^{(k)} = Ax^{(k-1)} = A^k x^{(0)} \quad (2)$$

Tiến hành khai triển vector khởi tạo $x^{(0)}$ qua hệ cơ sở $\{v_i\}$:

$$x^{(0)} = c_1 v_1 + c_2 v_2 + \dots + c_n v_n \quad (c_1 \neq 0) \quad (3)$$

Tác động toán tử tuyến tính A^k lên vector trạng thái:

$$\begin{aligned} A^k x^{(0)} &= c_1 \lambda_1^k v_1 + c_2 \lambda_2^k v_2 + \dots + c_n \lambda_n^k v_n \\ &= \lambda_1^k \left[c_1 v_1 + c_2 \left(\frac{\lambda_2}{\lambda_1} \right)^k v_2 + \dots + c_n \left(\frac{\lambda_n}{\lambda_1} \right)^k v_n \right] \end{aligned} \quad (4)$$

Dựa trên điều kiện phân ly trội, ta có $\left| \frac{\lambda_i}{\lambda_1} \right| < 1$ với mọi $i \geq 2$. Chuyển qua giới hạn không gian khi $k \rightarrow \infty$, các số hạng nhiễu đồng loạt tiệm cận 0. Chuỗi vector sẽ hội tụ tuyệt đối về hướng của vector riêng v_1 chi phối toàn bộ hệ thống. Nhằm triệt tiêu hiện tượng tràn số (overflow), vector trạng thái bắt buộc phải được chuẩn hóa qua mỗi chu kỳ bằng chuẩn vô cùng (infinity norm, $\|x\|_\infty$).

2. Phương pháp xuống thang (Deflation Method)

Khi hệ thống đã xác định được trị riêng trội λ_1 và vector riêng v_1 , phương pháp xuống thang (Hotelling Deflation) được áp dụng nhằm loại bỏ thành phần của λ_1 khỏi phổ ma trận. Phép biến đổi này kiến tạo một không gian ma trận suy biến B sao cho trị riêng lớn thứ hai λ_2 của A tiến lên vị trí trị riêng trội của B .

Đối với hệ thống ma trận đối xứng ($A = A^T$), thuật toán xuống thang được thiết lập qua phép chiếu trực giao:

$$B = A - \lambda_1 u_1 u_1^T \quad (5)$$

Trong đó:

- u_1 là vector riêng tương ứng λ_1 đã được chuẩn hóa theo không gian Euclid ($\|u_1\|_2 = 1$, cấu trúc đại số $u_1 = \frac{v_1}{\|v_1\|_2}$).
- Phổ trị riêng của không gian mới B là $\{0, \lambda_2, \lambda_3, \dots, \lambda_n\}$. Trạng thái λ_1 đã bị cưỡng bức về mức 0.

Thuật toán lũy thừa tiếp tục được tái kích hoạt trên không gian B để hội tụ xấp xỉ λ_2 .

5.2.2 Thuật toán định lượng (Mã giả)

Algorithm 1 Phương pháp lũy thừa (Tìm λ_1)

```

1: Input: Ma trận  $A$ , vector khởi tạo  $y^{(0)}$ , dung sai  $\varepsilon$ ,  $N_{max}$ 
2: Output: Trị riêng trội  $\lambda_1$ , vector riêng tương ứng  $v_1$ 
3:  $k \leftarrow 1$ 
4:  $\lambda_{prev} \leftarrow 0$ 
5: while  $k \leq N_{max}$  do
6:    $z^{(k)} \leftarrow Ay^{(k-1)}$ 
7:    $\lambda^{(k)} \leftarrow \max(|z_i^{(k)}|)$  ▷ Phần tử có trị tuyệt đối lớn nhất
8:    $y^{(k)} \leftarrow \frac{z^{(k)}}{\lambda^{(k)}}$  ▷ Chuẩn hóa Infinity norm
9:   if  $|\lambda^{(k)} - \lambda_{prev}| < \varepsilon$  then
10:    Return  $\lambda^{(k)}, y^{(k)}$ 
11:  end if
12:   $\lambda_{prev} \leftarrow \lambda^{(k)}$ 
13:   $y^{(k-1)} \leftarrow y^{(k)}$ 
14:   $k \leftarrow k + 1$ 
15: end while

```

Algorithm 2 Phương pháp xuống thang Hotelling (Cho ma trận đối xứng)

```

1: Input: Ma trận đối xứng  $A$ , trị riêng trội  $\lambda_1$ , vector riêng  $v_1$ 
2: Output: Ma trận suy biến  $B$ 
3:  $u_1 \leftarrow \frac{v_1}{\|v_1\|_2}$  ▷ Chuẩn hóa vector theo chuẩn Euclid (L2)
4:  $B \leftarrow A - \lambda_1(u_1 u_1^T)$  ▷ Chiếu trực giao triệt tiêu  $\lambda_1$ 
5: Return  $B$ 

```

5.2.3 Mở rộng: Phương pháp xuống thang cho ma trận không đối xứng

Khi ma trận A không đối xứng ($A \neq A^T$), hệ vector riêng của nó không còn tạo thành một cơ sở trực giao. Do đó, phương pháp xuống thang Hotelling cơ sở sẽ làm sai lệch cấu trúc phổ. Để giải quyết, hệ thống cần thiết lập phép chiếu thông qua cả vector riêng trái và vector riêng phải.

Gọi v_1 là vector riêng phải tương ứng với trị riêng trội λ_1 (nghiệm của hệ $Av_1 = \lambda_1 v_1$). Gọi w_1 là vector riêng trái tương ứng với λ_1 , thỏa mãn:

$$w_1^T A = \lambda_1 w_1^T \iff A^T w_1 = \lambda_1 w_1 \quad (6)$$

Do tính chất trực giao kép (Biorthogonality) của không gian vector riêng trái và phải, phép triệt tiêu trạng thái được định nghĩa bởi ma trận suy biến B :

$$B = A - \lambda_1 \frac{v_1 w_1^T}{w_1^T v_1} \quad (7)$$

Quy trình giải tích:

1. Áp dụng thuật toán lũy thừa trên không gian A để trích xuất λ_1 và v_1 .
2. Áp dụng thuật toán lũy thừa trên không gian chuyển vị A^T để trích xuất vector riêng trái w_1 .

3. Chuẩn hóa hệ số: Thiết lập $w_1 \leftarrow \frac{w_1}{w_1^T v_1}$.

4. Hoàn thiện không gian giáng cấp: $B = A - \lambda_1(v_1 w_1^T)$. Phổ của B lúc này sẽ là $\{0, \lambda_2, \dots, \lambda_n\}$.

5.2.4 Thuật toán định lượng (Mã giả cho ma trận không đối xứng)

Algorithm 3 Phương pháp xuống thang tổng quát (General Deflation)

```
1: Input: Ma trận không đối xứng  $A$ , trị riêng trội  $\lambda_1$ , vector riêng phải  $v_1$ , vector khởi tạo  
   trái  $y_{left}^{(0)}$   
2: Output: Ma trận suy biến  $B$   
3: // 1. Trích xuất vector riêng trái  $w_1$  từ không gian chuyển vị  
4:  $w_1 \leftarrow \text{PowerMethod}(A^T, y_{left}^{(0)})$   
5: // 2. Chuẩn hóa vector riêng trái theo điều kiện trực giao kép  
6:  $\text{inner\_product} \leftarrow w_1^T \cdot v_1$   
7:  $w_1^* \leftarrow \frac{w_1}{\text{inner\_product}}$   
8: // 3. Thiết lập phép chiếu và giáng cấp không gian  
9:  $\text{ProjectionMatrix} \leftarrow v_1(w_1^*)^T$  ▷ Tích ngoài của vector  
10:  $B \leftarrow A - \lambda_1 \cdot \text{ProjectionMatrix}$   
11: Return  $B$ 
```

5.2.5 Ví dụ minh họa định lượng

Bài toán: Phân tích phổ trị riêng của hệ thống ma trận đối xứng $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ bằng phương pháp lũy thừa và xuống thang.

Bước 1: Xác định trị riêng trội λ_1 bằng Phương pháp lũy thừa

Lựa chọn vector không gian khởi tạo $y^{(0)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$.

• Chu kỳ $k = 1$:

$$z^{(1)} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

Trích xuất cấu tử $\lambda^{(1)} = 2$. Chuẩn hóa trạng thái $y^{(1)} = \frac{1}{2} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}$.

• Chu kỳ $k = 2$:

$$z^{(2)} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 2.5 \\ 2 \end{bmatrix}$$

Trích xuất cấu tử $\lambda^{(2)} = 2.5$. Chuẩn hóa trạng thái $y^{(2)} = \frac{1}{2.5} \begin{bmatrix} 2.5 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0.8 \end{bmatrix}$.

• Chu kỳ $k = 3$:

$$z^{(3)} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0.8 \end{bmatrix} = \begin{bmatrix} 2.8 \\ 2.6 \end{bmatrix}$$

Trích xuất cấu tử $\lambda^{(3)} = 2.8$. Chuẩn hóa trạng thái $y^{(3)} = \begin{bmatrix} 1 \\ 0.928 \end{bmatrix}$.

Qua phân tích giới hạn hội tụ, chuỗi vô hạn $\{\lambda^{(k)}\} \rightarrow 3$ và $\{y^{(k)}\} \rightarrow \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

Kết luận vi phân: $\lambda_1 = 3, v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

Bước 2: Triệt tiêu trạng thái bằng Phương pháp xuống thang
Tiến hành chuẩn hóa vector riêng về không gian metric Euclid (L_2):

$$\|v_1\|_2 = \sqrt{1^2 + 1^2} = \sqrt{2} \implies u_1 = \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$$

Thiết lập phép chiếu ma trận giáng cấp B :

$$\begin{aligned} B = A - \lambda_1 u_1 u_1^T &= \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} - 3 \begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix} \\ &= \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} - 3 \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix} \\ &= \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} - \begin{bmatrix} 1.5 & 1.5 \\ 1.5 & 1.5 \end{bmatrix} = \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{bmatrix} \end{aligned}$$

Bước 3: Xác định trị riêng thứ hai λ_2 trên không gian B

Khởi tạo vector định hướng $y^{(0)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$.

• Chu kỳ $k = 1$:

$$z^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.5 \\ -0.5 \end{bmatrix}$$

Biên độ cực đại là $0.5 \implies \lambda^{(1)} = 0.5$. Chuẩn hóa vector $y^{(1)} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

• Chu kỳ $k = 2$:

$$z^{(2)} = \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Biên độ cực đại là $1 \implies \lambda^{(2)} = 1$. Chuẩn hóa vector $y^{(2)} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

Hệ thống đạt trạng thái hội tụ tuyệt đối. Cấu tử trị riêng thứ hai của hệ thống là $\lambda_2 = 1$, vector riêng tương ứng $v_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

TỔNG HỢP LÝ THUYẾT VÀ THUẬT TOÁN: SVD, PHƯƠNG PHÁP LŨY THỪA VÀ XUỐNG THANG

1 Cơ sở lý thuyết: Phân tích giá trị kỳ dị (SVD)

1.1 Định lý phân rã

Phân tích giá trị kỳ dị (Singular Value Decomposition - SVD) là một trong những công cụ mạnh mẽ nhất trong đại số tuyến tính ứng dụng. Định lý phát biểu rằng mọi ma trận $A \in \mathbb{R}^{m \times n}$ đều có thể được phân rã thành tích của ba ma trận:

$$A = U \Sigma V^T \quad (1)$$

Trong đó:

- $U \in \mathbb{R}^{m \times m}$ là ma trận trực giao ($U^T U = I$). Các cột của U được gọi là các vector kỳ dị trái (left singular vectors).
- $\Sigma \in \mathbb{R}^{m \times n}$ là ma trận đường chéo chứa các giá trị $\sigma_i \geq 0$ xếp theo thứ tự giảm dần $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$. Đây là các **giá trị kỳ dị** (singular values).
- $V \in \mathbb{R}^{n \times n}$ là ma trận trực giao ($V^T V = I$). Các cột của V được gọi là các vector kỳ dị phải (right singular vectors).

1.2 Khai triển thành tổng các ma trận xấp xỉ hạng 1 (Rank-1 Matrices)

Dưới góc độ thuật toán tính toán, ta không tìm toàn bộ U và V cùng lúc. Khai triển phép nhân ma trận cho phép viết lại SVD dưới dạng một chuỗi phép cộng của các ma trận hạng 1:

$$A = \sum_{i=1}^r \sigma_i u_i v_i^T = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_r u_r v_r^T \quad (2)$$

Tư duy khoa học ở đây là: Mỗi cụm $\sigma_i u_i v_i^T$ là một “mảnh ghép” (một ma trận có cùng kích thước với A) mang một phần thông tin của A . Do σ_1 là lớn nhất, mảnh ghép đầu tiên chứa lượng thông tin cốt lõi nhất.

2 Phương pháp Lũy thừa (Power Method) và SVD

2.1 Bản chất toán học

Phương pháp Lũy thừa là thuật toán số trị nhằm tìm giá trị riêng trội (có trị tuyệt đối lớn nhất) λ_1 và vector riêng trội x của một **ma trận vuông**. Ý tưởng cốt lõi là quá trình nhân vector lặp đi lặp lại $A^k x_0$ sẽ khuếch đại thành phần của vector riêng trội theo hàm mũ, làm các thành phần vector riêng khác bị triệt tiêu khi $k \rightarrow \infty$.

2.2 Sự liên hệ với SVD

Vì SVD áp dụng cho ma trận chữ nhật A , ta không thể dùng trực tiếp phương pháp lũy thừa. Giải pháp là xét ma trận đối xứng bán xác định dương $M = A^T A \in \mathbb{R}^{n \times n}$. Toán học chứng minh sự tương đương sau:

1. Giá trị riêng của M chính là bình phương giá trị kỳ dị của A : $\lambda_i(M) = \sigma_i^2(A)$.
2. Vector riêng trội của M chính là vector kỳ dị phải v_1 của A .
3. Vector kỳ dị trái được suy ra thông qua phép biến đổi: $u_1 = \frac{1}{\sigma_1} A v_1$.

3 Phương pháp Xuống thang (Deflation Method)

Phương pháp Lũy thừa chỉ hội tụ về phần tử trội nhất (σ_1). Để tìm thành phần thứ hai (σ_2), ta phải loại bỏ sức ảnh hưởng của phần tử thứ nhất khỏi không gian dữ liệu.

Dựa vào khai triển tổng ma trận, ta xác định phần dư bằng phép trừ:

$$A^{(2)} = A - \sigma_1 u_1 v_1^T \quad (3)$$

Lúc này, ma trận $A^{(2)}$ không còn chứa thành phần σ_1 . Thành phần lớn thứ hai (σ_2) nay đã trở thành phần tử trội nhất của $A^{(2)}$. Ta tiếp tục áp dụng Phương pháp Lũy thừa lên $A^{(2)}$ để tìm bộ ba (σ_2, u_2, v_2) .

4 Trình bày thuật toán tổng quát

Mục tiêu: Xấp xỉ ma trận A qua k giá trị kỳ dị lớn nhất.

Algorithm 1 Thuật toán SVD xấp xỉ bằng Lũy thừa và Xuống thang

Require: Ma trận $A \in \mathbb{R}^{m \times n}$, số lượng giá trị kỳ dị cần tìm k , sai số ϵ .

Ensure: Tập hợp $\{(\sigma_i, u_i, v_i)\}_{i=1}^k$.

```
1: Khởi tạo  $A_{curr} \leftarrow A$ 
2: for  $i = 1$  to  $k$  do
3:   Tính ma trận trung gian  $M = A_{curr}^T A_{curr}$ 
4:   Khởi tạo vector ngẫu nhiên  $x_0 \in \mathbb{R}^{n \times 1}$ 
5:   repeat ▷ Bắt đầu Phương pháp Lũy thừa
6:      $y = Mx_{curr}$ 
7:      $c = \max(|y_j|)$  ▷ Lấy phần tử có trị tuyệt đối lớn nhất
8:      $x_{next} = \frac{y}{c}$  ▷ Chuẩn hóa để tránh tràn số
9:     Tính sai số  $err = \|x_{next} - x_{curr}\|$ 
10:     $x_{curr} \leftarrow x_{next}$ 
11:  until  $err < \epsilon$ 
12:   $\lambda_i \leftarrow c$ 
13:   $\sigma_i \leftarrow \sqrt{\lambda_i}$ 
14:   $v_i \leftarrow \frac{x_{curr}}{\|x_{curr}\|_2}$  ▷ Chuẩn hóa vector kỳ dị phải về độ dài 1
15:   $u_i \leftarrow \frac{1}{\sigma_i} A_{curr} v_i$  ▷ Tính vector kỳ dị trái
16:  Lưu trữ bộ ba  $(\sigma_i, u_i, v_i)$ 
17:   $A_{curr} \leftarrow A_{curr} - \sigma_i u_i v_i^T$  ▷ Phương pháp Xuống thang
18: end for
```

5 Ví dụ minh họa chi tiết (Ma trận 3x4)

Cho ma trận chữ nhật $A \in \mathbb{R}^{3 \times 4}$ có dạng:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 4 & 3 & 2 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Tìm thành phần kỳ dị lớn nhất (σ_1, u_1, v_1) bằng Phương pháp Lũy thừa và thực hiện Xuống thang để tìm dư ảnh.

Bước 1: Thiết lập ma trận vuông đối xứng $M = A^T A$

$$M = \begin{bmatrix} 1 & 4 & 1 \\ 2 & 3 & 1 \\ 3 & 2 & 1 \\ 4 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 \\ 4 & 3 & 2 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 18 & 15 & 12 & 9 \\ 15 & 14 & 13 & 12 \\ 12 & 13 & 14 & 15 \\ 9 & 12 & 15 & 18 \end{bmatrix}$$

Bước 2: Chạy Phương pháp Lũy thừa trên $M \in \mathbb{R}^{4 \times 4}$

Ta chọn vector khởi tạo không đồng đều để thấy rõ quá trình hội tụ: $x_0 = [1, 0, 0, 0]^T$.

• **Vòng 1:**

$$y_1 = Mx_0 = \begin{bmatrix} 18 \\ 15 \\ 12 \\ 9 \end{bmatrix} \implies c_1 = 18$$

$$x_1 = \frac{1}{18} y_1 = \begin{bmatrix} 1 \\ 0.833 \\ 0.667 \\ 0.5 \end{bmatrix}$$

- Vòng 2:

$$y_2 = Mx_1 \approx \begin{bmatrix} 43 \\ 41.33 \\ 39.67 \\ 38 \end{bmatrix} \implies c_2 = 43$$

$$x_2 = \frac{1}{43}y_2 \approx \begin{bmatrix} 1 \\ 0.961 \\ 0.922 \\ 0.884 \end{bmatrix}$$

- Vòng 3:

$$y_3 = Mx_2 \approx \begin{bmatrix} 51.43 \\ 51.05 \\ 50.66 \\ 50.27 \end{bmatrix} \implies c_3 = 51.43$$

$$x_3 = \frac{1}{51.43}y_3 \approx \begin{bmatrix} 1 \\ 0.992 \\ 0.985 \\ 0.977 \end{bmatrix}$$

Nhận xét: Dãy số $c_k \rightarrow 54$ và vector $x_k \rightarrow [1, 1, 1, 1]^T$. Giả sử thuật toán hội tụ hoàn toàn, ta thu được giá trị riêng trội $\lambda_1 = 54$ và vector xấp xỉ $x = [1, 1, 1, 1]^T$.

Bước 3: Trích xuất các thành phần SVD cốt lõi

- Giá trị kỳ dị lớn nhất:

$$\sigma_1 = \sqrt{\lambda_1} = \sqrt{54} \approx \mathbf{7.348}$$

- Vector kỳ dị phải (v_1): Chuẩn hóa vector x với độ dài $\|x\|_2 = \sqrt{1^2 + 1^2 + 1^2 + 1^2} = 2$.

$$v_1 = \frac{x}{\|x\|_2} = \frac{1}{2} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

- Vector kỳ dị trái (u_1): Áp dụng công thức suy diễn $u_1 = \frac{1}{\sigma_1}Av_1$.

$$u_1 = \frac{1}{\sqrt{54}} \begin{bmatrix} 1 & 2 & 3 & 4 \\ 4 & 3 & 2 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix} = \frac{1}{\sqrt{54}} \begin{bmatrix} 5 \\ 5 \\ 5 \\ 2 \end{bmatrix} \approx \begin{bmatrix} 0.680 \\ 0.680 \\ 0.680 \\ 0.272 \end{bmatrix}$$

Bước 4: Xuống thang (Deflation) để dọn đường tìm σ_2

Ta tạo ma trận A_{new} bằng cách lấy A trừ đi mảnh ghép của σ_1 :

$$A_{new} = A - \sigma_1 u_1 v_1^T$$

$$\sigma_1 u_1 v_1^T = \sqrt{54} \cdot \frac{1}{\sqrt{54}} \begin{bmatrix} 5 \\ 5 \\ 5 \\ 2 \end{bmatrix} \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \end{bmatrix} = \begin{bmatrix} 2.5 & 2.5 & 2.5 & 2.5 \\ 2.5 & 2.5 & 2.5 & 2.5 \\ 2.5 & 2.5 & 2.5 & 2.5 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Thực hiện phép trừ ma trận:

$$A_{new} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 4 & 3 & 2 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} - \begin{bmatrix} 2.5 & 2.5 & 2.5 & 2.5 \\ 2.5 & 2.5 & 2.5 & 2.5 \\ 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} -1.5 & -0.5 & 0.5 & 1.5 \\ 1.5 & 0.5 & -0.5 & -1.5 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Sức ảnh hưởng của thành phần cốt lõi $\sigma_1 = 7.348$ đã bị loại bỏ hoàn toàn. Bằng cách đưa A_{new} quay trở lại lặp Bước 1, thuật toán Lũy thừa sẽ tự động tìm được giá trị kỳ dị tiếp theo là $\sigma_2 \approx 3.162$.